PREMIUM HANDWRITTEN STUDY EDITION

JAVA + SPRING

Spring Boot Zero to Hero

A visual interview notebook.

Short answers. Real examples. Explained follow-ups.



180 Original Questions • 180 Follow-up Answers

24 topic chapters • 12 coding exercises • 30-day study plan

Java 17+ / Spring Boot 3.x baseline | Architecture & Production Scenarios

Your study playbook

Each original question is reproduced with exactly the same wording. Explain it in **20 seconds**, trace the supplied example, then answer the follow-up without looking.

1 • EASY ANSWER	A short and accurate response in plain English.
2 • THE PICTURE	A small diagram showing the relevant Spring flow or system boundary.
3 • SOURCE EXAMPLE	The Java, XML, YAML, or conceptual snippet from the supplied guide.
4 • WHY IT WORKS	The rule or behavior described by the source material.
5 • COMMON TRAP	What an interviewer wants you to avoid claiming.
6 • FOLLOW-UP	A direct, practical answer, not an unanswered practice prompt.

> Version boundary: Most snippets are focused fragments, not independently runnable Spring Boot applications. Use Java 17+ and a compatible Boot 3.x stack. Java 21 and Boot 4 discussions must be interpreted with their stated version context.

24 chapters at a glance

01 Spring Boot foundations
02 Application startup and context
03 Dependency injection and bean selection
04 Scopes, lifecycle and proxies
05 External configuration and profiles
06 Auto-configuration and custom starters
07 Spring MVC request processing
08 REST contracts and API design
09 Validation and exception responses
10 Spring Data JPA and entity ownership
11 Transactions and consistency
12 Database performance and concurrency
13 Spring Security fundamentals
14 JWT, OAuth 2.0 and OpenID Connect
15 Testing from units to real boundaries
16 HTTP clients, deadlines and resilience
17 Async execution, scheduling and caching
18 Messaging, Kafka and reliable events
19 WebFlux, Reactor and virtual threads
20 Actuator, observability and operational health
21 Packaging, Docker and Kubernetes deployment
22 Modern Spring and migration choices
23 Advanced design and interview distinctions
24 Senior production diagnosis

CHAPTER 01 • SPRING BOOT FOUNDATIONS

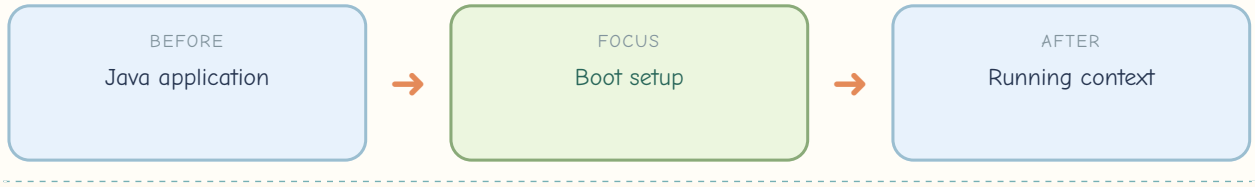
QUESTION 001 / 180

What is Spring Boot?

01 / THE SHORT INTERVIEW ANSWER

> **Spring Boot builds on Spring to provide opinionated dependency management, auto-configuration, executable packaging, and operational features.**

DRAW THE IDEA — Question focus



02 / THE ORIGINAL EXAMPLE

```
@SpringBootApplication
public class App {
  public static void main(String[] args) {
    SpringApplication.run(App.class, args);
  }
}
```

03 / WHY THIS WORKS

The launcher creates a Spring application context and, for a web application, starts its configured server. Boot reduces setup; your business behavior still belongs in application code.

04 / WATCH OUT

> Do not describe Boot as a replacement for Spring.

05 / INTERVIEWER FOLLOW-UP

Which decisions remain your responsibility?

Answer: You still design domain rules, API contracts, security, transactions, data ownership, limits, testing and operations. Boot configures infrastructure; it does not decide the right business behavior.

Source guide p. 12 • Beginner • References: B01, B02, B03

CHAPTER 01 • SPRING BOOT FOUNDATIONS

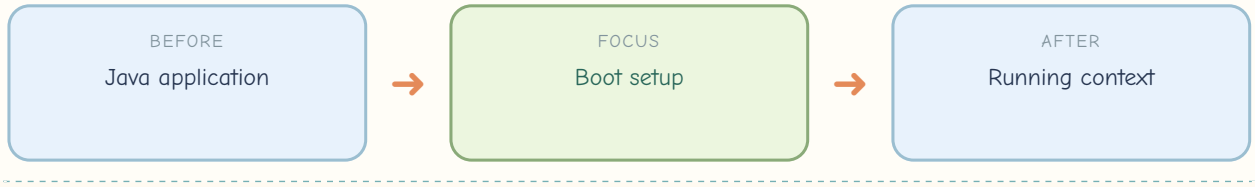
QUESTION 002 / 180

How are Spring, Spring MVC and Spring Boot different?

01 / THE SHORT INTERVIEW ANSWER

> **Spring supplies dependency injection and infrastructure; MVC handles servlet web requests; Boot assembles and configures applications using them.**

DRAW THE IDEA — MVC different



02 / THE ORIGINAL EXAMPLE

```
spring-context -> bean container
spring-webmvc  -> DispatcherServlet and controllers
Spring Boot    -> application assembly and defaults
```

03 / WHY THIS WORKS

A Boot MVC service uses all three roles. Boot can also run a command-line job with no MVC server. These names identify different layers, not competing frameworks.

04 / WATCH OUT

> Boot does not require REST or microservices.

05 / INTERVIEWER FOLLOW-UP

Can Boot run without an embedded web server?

Answer: Yes. Set spring.main.web-application-type=none or configure SpringApplication with WebApplicationType.NONE. The context can run batch jobs or command-line tasks without starting a web server.

Source guide p. 13 • Beginner • References: B01, B02, B03

CHAPTER 01 • SPRING BOOT FOUNDATIONS

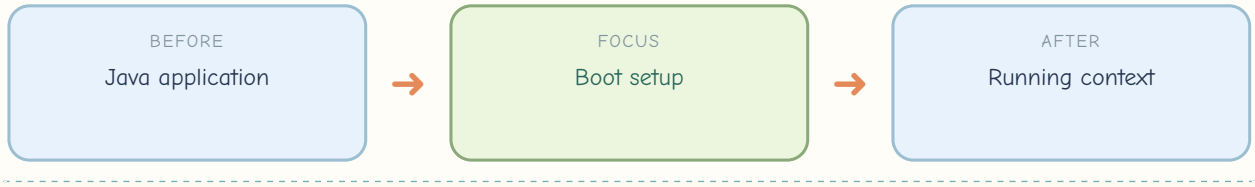
QUESTION 003 / 180

What does @SpringBootApplication combine?

01 / THE SHORT INTERVIEW ANSWER

> **It combines @SpringBootConfiguration, @EnableAutoConfiguration, and @ComponentScan.**

DRAW THE IDEA — @SpringBootApplication combine



02 / THE ORIGINAL EXAMPLE

```
@SpringBootApplication
class App { /* main method omitted */ }
// Configuration + auto-configuration + component scanning
```

03 / WHY THIS WORKS

The application class can declare bean methods, Boot can contribute conditional infrastructure, and scanning discovers components below the application package. These mechanisms can also be configured separately.

04 / WATCH OUT

> Scanning does not search every package in every dependency.

05 / INTERVIEWER FOLLOW-UP

Why put the application class in a root package?

Answer: Component scanning normally starts at the application class package and its children. A root package places controllers and services inside that default scan area.

Source guide p. 13 • Beginner • References: B01, B02, B03

CHAPTER 01 • SPRING BOOT FOUNDATIONS

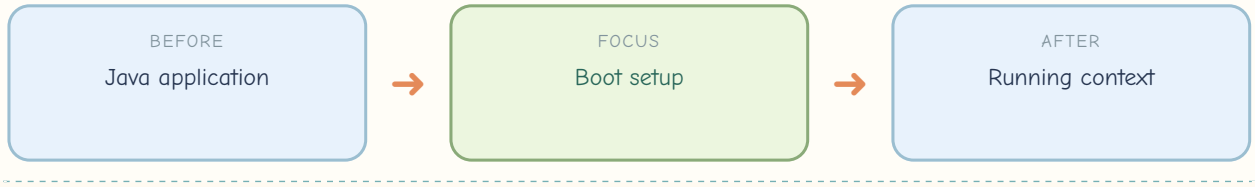
QUESTION 004 / 180

What is a starter?

01 / THE SHORT INTERVIEW ANSWER

> **A starter is a convenient dependency descriptor for a capability; it is not itself the entire runtime implementation.**

DRAW THE IDEA — *starter*



02 / THE ORIGINAL EXAMPLE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

03 / WHY THIS WORKS

For the Boot 3.x baseline, the web starter brings MVC, an embedded servlet server and supporting libraries through transitive dependencies. Boot's managed dependency versions keep the selected ecosystem aligned.

04 / WATCH OUT

A dependency starter and an auto-configuration module are distinct roles.

05 / INTERVIEWER FOLLOW-UP

How do you replace the default embedded server?

Answer: Exclude the default Tomcat starter and add the matching Jetty or Undertow starter for your compatible Boot version. Check the dependency tree and run an HTTP smoke test.

Source guide p. 14 • Beginner • References: B01, B02, B03

CHAPTER 01 • SPRING BOOT FOUNDATIONS

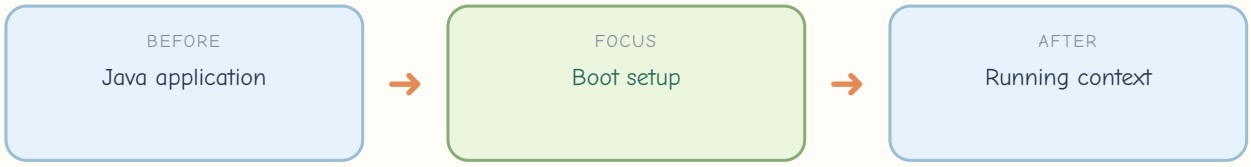
QUESTION 005 / 180

Why use Boot dependency management instead of assigning every version?

01 / THE SHORT INTERVIEW ANSWER

Its tested version set reduces incompatible combinations; override a dependency only with compatibility evidence.

DRAW THE IDEA — dependency management instead assigning



02 / THE ORIGINAL EXAMPLE

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>${chosen-boot-3.5-patch}</version>
</parent>
```

03 / WHY THIS WORKS

This version is a selection placeholder, not a ready-to-run POM. The parent supplies dependency and plugin conventions. A BOM import supplies dependency management when another parent must be retained.

04 / WATCH OUT

A BOM does not automatically apply every build-plugin convention.

05 / INTERVIEWER FOLLOW-UP

How would you investigate a dependency conflict?

Answer: Inspect mvn dependency:tree or Gradle dependencyInsight, find the selected transitive version, compare Boot BOM versions, then test a compatible exclusion or override.

Source guide p. 14 • Beginner • References: B01, B02, B03

CHAPTER 01 • SPRING BOOT FOUNDATIONS

QUESTION 006 / 180

Does Boot generate application source code at runtime?

01 / THE SHORT INTERVIEW ANSWER

> **Boot configures objects through Spring infrastructure; ordinary auto-configuration does not generate your business classes.**

DRAW THE IDEA — generate source code runtime



02 / THE ORIGINAL EXAMPLE

```
Dependency present + condition matches
  -> configuration contributes bean definitions
Context refresh -> beans created and wired
```

03 / WHY THIS WORKS

Adding a JDBC dependency can lead to a DataSource when the required configuration and conditions are satisfied. It does not generate your repository's business logic.

04 / WATCH OUT

> AOT build-time generation is a separate feature.

05 / INTERVIEWER FOLLOW-UP

How can you see why an auto-configuration matched?

Answer: Start with --debug or inspect the Actuator conditions endpoint in a secured environment. The condition evaluation report explains matched, unmatched and excluded configurations.

Source guide p. 15 • Beginner • References: B01, B02, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

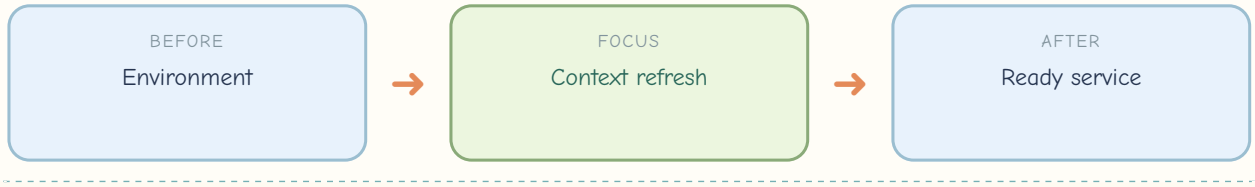
QUESTION 007 / 180

What does SpringApplication.run do?

01 / THE SHORT INTERVIEW ANSWER

> It prepares the environment, creates and refreshes an appropriate application context, and completes startup callbacks.

DRAW THE IDEA — SpringApplication.run



02 / THE ORIGINAL EXAMPLE

```
ConfigurableApplicationContext context =
    SpringApplication.run(App.class, args);
OrderService service = context.getBean(OrderService.class);
```

03 / WHY THIS WORKS

The returned context owns managed beans and their lifecycle. A web context also manages server integration. If a required bean cannot be created, run fails instead of returning a healthy application.

04 / WATCH OUT

Startup involves more than invoking main.

05 / INTERVIEWER FOLLOW-UP

Where would a missing required property fail?

Answer: Missing required configuration usually fails during property binding, validation or bean creation as the context refreshes. The root cause tells which property was absent.

Source guide p. 16 • Beginner • References: B01, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

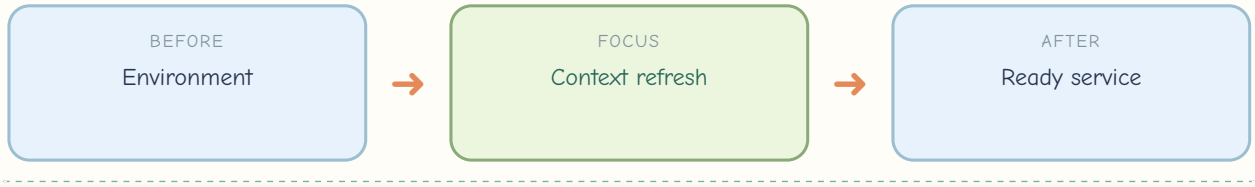
QUESTION 008 / 180

What is ApplicationContext?

01 / THE SHORT INTERVIEW ANSWER

> **It is Spring's bean container with additional services such as events, resources, messages, and environment access.**

DRAW THE IDEA — ApplicationContext



02 / THE ORIGINAL EXAMPLE

```
@Service
class PricingService { BigDecimal price() { return BigDecimal.TEN; } }
// context.getBean(PricingService.class) returns the managed bean.
```

03 / WHY THIS WORKS

The container constructs and wires managed objects according to definitions. Calling new PricingService manually creates an ordinary object outside those lifecycle and proxy rules.

04 / WATCH OUT

A Java object is not managed merely because its class has an annotation.

05 / INTERVIEWER FOLLOW-UP

How can a library object become a bean?

Answer: Declare it with a @Bean factory method in a @Configuration class, or register a BeanDefinition. Spring then owns its creation and lifecycle.

Source guide p. 17 • Beginner • References: B01, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

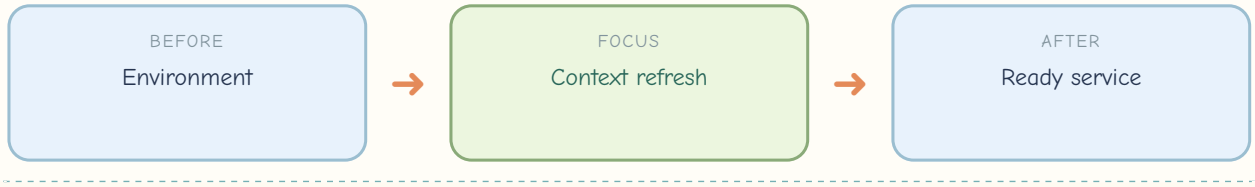
QUESTION 009 / 180

How does BeanFactory differ from ApplicationContext?

01 / THE SHORT INTERVIEW ANSWER

BeanFactory supplies core bean creation and lookup; ApplicationContext adds application-level facilities and commonly creates non-lazy singletons during refresh.

DRAW THE IDEA — BeanFactory ApplicationContext



02 / THE ORIGINAL EXAMPLE

```
BeanFactory -> create and resolve beans
ApplicationContext -> BeanFactory capabilities + events,
                      environment, resources, integration
```

03 / WHY THIS WORKS

Boot normally works with an ApplicationContext. Lazy behavior depends on the actual context and bean definitions; avoid treating every factory implementation as having one identical startup policy.

04 / WATCH OUT

The terms are related, not interchangeable in every API.

05 / INTERVIEWER FOLLOW-UP

Why do eager singletons help detect errors early?

Answer: Creating required non-lazy singletons at startup exposes missing dependencies and invalid configuration before production requests arrive, rather than at the first use.

Source guide p. 17 • Beginner • References: B01, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

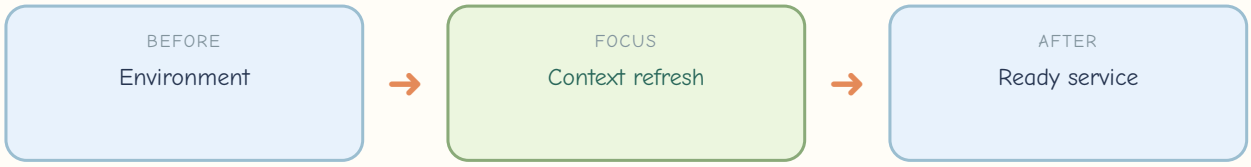
QUESTION 010 / 180

How do ApplicationRunner and CommandLineRunner differ?

01 / THE SHORT INTERVIEW ANSWER

Both run after context refresh; ApplicationRunner exposes parsed ApplicationArguments, while CommandLineRunner receives raw strings.

DRAW THE IDEA — ApplicationRunner CommandLineRunner



02 / THE ORIGINAL EXAMPLE

```
@Bean
ApplicationRunner warmup() {
  return args -> System.out.println(args.containsOption("warmup"));
}
// java -jar app.jar --warmup=true -> prints true
```

03 / WHY THIS WORKS

Use runner callbacks for controlled startup work. An exception can fail startup, and slow work postpones completion. @Order can order runners but does not create a general dependency graph.

04 / WATCH OUT

Do not hide endless background loops in a runner.

05 / INTERVIEWER FOLLOW-UP

When should warmup finish before readiness?

Answer: If a warmup result is required before safe traffic, make it a bounded startup action and do not report readiness until it completes. Optional warmup should not block indefinitely.

Source guide p. 18 • Beginner • References: B01, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

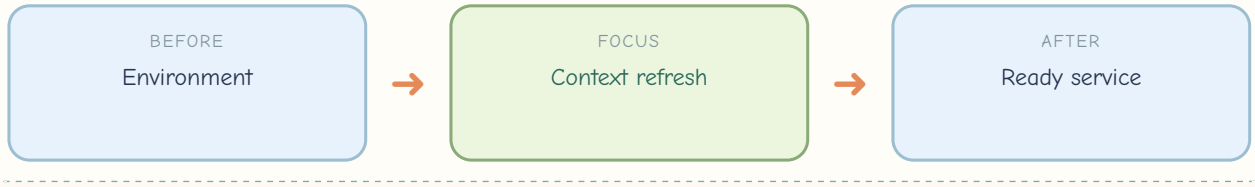
QUESTION 011 / 180

What is the difference between started and ready events?

01 / THE SHORT INTERVIEW ANSWER

> **ApplicationStartedEvent occurs after refresh before runners; ApplicationReadyEvent follows successful runner completion.**

DRAW THE IDEA — difference started ready events



02 / THE ORIGINAL EXAMPLE

```
Refresh complete -> ApplicationStartedEvent
Run application and command-line runners
Successful completion -> ApplicationReadyEvent
```

03 / WHY THIS WORKS

A server may already be bound before all warmup completes. Use readiness state to control traffic admission rather than assuming a listening port means business readiness.

04 / WATCH OUT

> An event callback is not a distributed deployment coordinator.

05 / INTERVIEWER FOLLOW-UP

What if a runner throws?

Answer: The application startup fails and ApplicationReadyEvent is not published. Record the cause and leave traffic readiness disabled instead of accepting requests with incomplete initialization.

Source guide p. 18 • Beginner • References: B01, B03

CHAPTER 02 • APPLICATION STARTUP AND CONTEXT

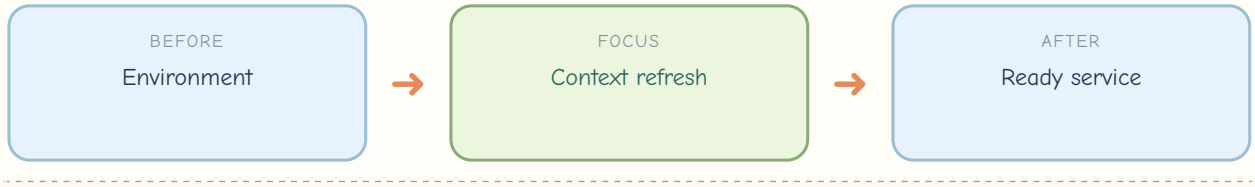
QUESTION 012 / 180

How do you diagnose startup failure?

01 / THE SHORT INTERVIEW ANSWER

Find the first meaningful cause, then check definitions, conditions, configuration, versions, and external resource availability.

DRAW THE IDEA — diagnose startup failure



02 / THE ORIGINAL EXAMPLE

```
java -jar app.jar --debug
# Inspect the condition evaluation report and failure analysis.
# Check the deepest relevant cause rather than the final wrapper.
```

03 / WHY THIS WORKS

An UnsatisfiedDependencyException can wrap a connection or binding failure. The outer exception describes where construction failed; the cause explains why. Keep debug logs controlled because configuration diagnostics can contain sensitive context.

04 / WATCH OUT

Repeatedly adding @ComponentScan can mask the real issue.

05 / INTERVIEWER FOLLOW-UP

How do you distinguish missing bean from failed bean creation?

Answer: A missing bean often causes NoSuchBeanDefinitionException; a found bean that fails initialization yields BeanCreationException with a nested cause. Trace the first useful root cause.

Source guide p. 19 • Beginner • References: B01, B03

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

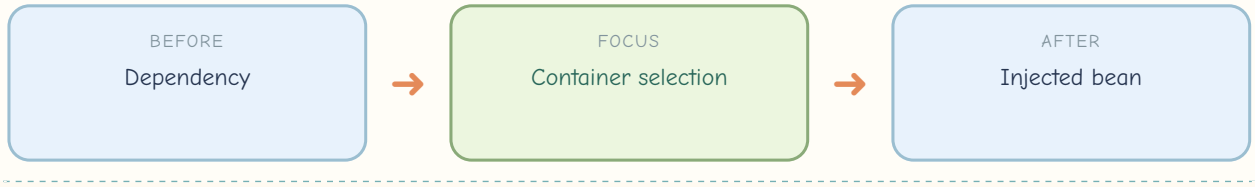
QUESTION 013 / 180

Why prefer constructor injection?

01 / THE SHORT INTERVIEW ANSWER

> **It makes required dependencies explicit, supports final fields, and permits plain unit testing without a Spring context.**

DRAW THE IDEA — prefer constructor injection



02 / THE ORIGINAL EXAMPLE

```
@Service
class Checkout {
  private final PaymentGateway gateway;
  Checkout(PaymentGateway gateway) { this.gateway = gateway; }
}
```

03 / WHY THIS WORKS

Spring resolves the constructor dependency. A test can pass a fake gateway directly. With one constructor, @Autowired is usually unnecessary. The object cannot be constructed successfully without its required collaborator.

04 / WATCH OUT

> Constructor injection does not make dependency methods thread safe.

05 / INTERVIEWER FOLLOW-UP

When is setter injection appropriate?

Answer: Setter injection fits genuinely optional or reconfigurable collaborators; document the absent-state behavior. Use constructors for required dependencies.

Source guide p. 20 • Beginner • References: F01, F02

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

QUESTION 014 / 180

What is the difference between @Component, @Service and @Repository?

01 / THE SHORT INTERVIEW ANSWER

All can register scanned components; @Service communicates business intent, while @Repository also supports configured exception translation.

DRAW THE IDEA — difference @Component @Service @Reposit



02 / THE ORIGINAL EXAMPLE

```
@Repository
class JdbcCustomerStore { /* data access */ }
@Service
class CustomerService { /* business policy */ }
```

03 / WHY THIS WORKS

Stereotypes communicate architecture. Repository exception translation depends on the relevant post-processor and persistence integration; it is not a blanket translation of every arbitrary exception.

04 / WATCH OUT

@Service does not automatically start a transaction.

05 / INTERVIEWER FOLLOW-UP

What annotation defines a transaction boundary?

Answer: @Transactional marks the intended transaction boundary, subject to the configured transaction manager and proxy/weaving invocation path. @Service by itself does not start one.

Source guide p. 21 • Beginner • References: F01, F02

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

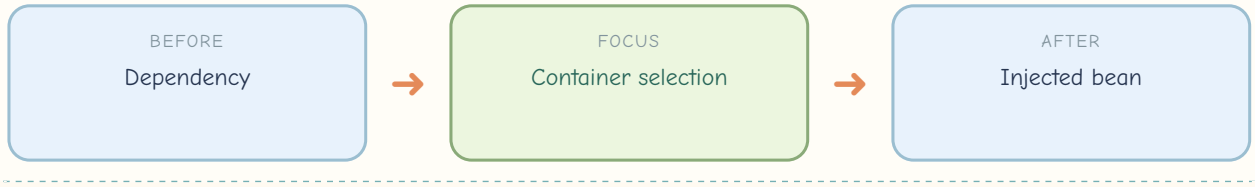
QUESTION 015 / 180

How does @Bean differ from @Component?

01 / THE SHORT INTERVIEW ANSWER

> **@Component discovers a class through scanning; @Bean defines a managed object through a configuration method.**

DRAW THE IDEA — @Bean @Component



02 / THE ORIGINAL EXAMPLE

```
@Configuration
class ClockConfig {
  @Bean Clock clock() { return Clock.systemUTC(); }
}
```

03 / WHY THIS WORKS

Clock is a library type you need not annotate. Its factory result becomes a managed bean. @Bean is also useful when construction needs arguments or policy that cannot be expressed by component scanning alone.

04 / WATCH OUT

Calling a factory outside Spring can create an unmanaged object.

05 / INTERVIEWER FOLLOW-UP

How do bean names get chosen?

Answer: Default scanned bean names usually derive from the class name; @Bean uses the method name. You can assign explicit names and qualifiers to remove ambiguity.

Source guide p. 21 • Beginner • References: F01, F02

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

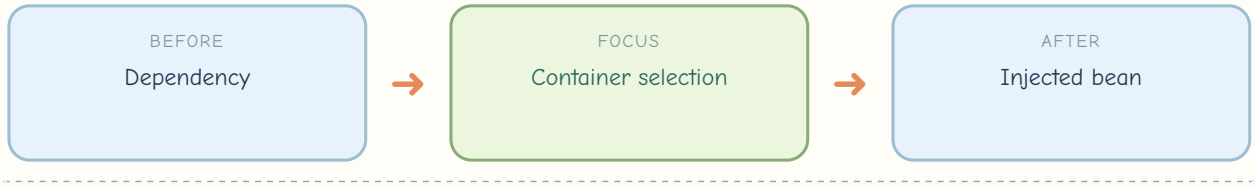
QUESTION 016 / 180

How do @Primary and @Qualifier resolve multiple beans?

01 / THE SHORT INTERVIEW ANSWER

> **@Primary supplies a preferred candidate; @Qualifier narrows a dependency to matching qualified candidates.**

DRAW THE IDEA — @Primary @Qualifier resolve multiple



02 / THE ORIGINAL EXAMPLE

```
Checkout(@Qualifier("upiGateway") PaymentGateway gateway) {
  this.gateway = gateway;
}
// Both cardGateway and upiGateway may implement PaymentGateway.
```

03 / WHY THIS WORKS

The constructor chooses the UPI-qualified bean rather than guessing by implementation type. Primary is useful for a default choice; qualifiers express dependency-specific selection.

04 / WATCH OUT

Qualifiers can be custom annotations, not only bean-name strings.

05 / INTERVIEWER FOLLOW-UP

What if no candidate matches?

Answer: A required injection usually fails with NoSuchBeanDefinitionException, often wrapped by UnsatisfiedDependencyException. Either register a matching bean or fix the qualifier.

Source guide p. 22 • Beginner • References: F01, F02

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

QUESTION 017 / 180

How should optional dependencies be represented?

01 / THE SHORT INTERVIEW ANSWER

> **Use an explicit optional provider or Optional only when absence is a supported behavior.**

DRAW THE IDEA — optional dependencies represented



02 / THE ORIGINAL EXAMPLE

```
class Reporter {
  Reporter(ObjectProvider<AuditSink> sinks) {
    AuditSink sink = sinks.getIfAvailable();
    // Define a deliberate no-sink policy.
  }
}
```

03 / WHY THIS WORKS

The provider defers lookup and can support optional or scoped access. Required security or payment collaborators should generally fail startup when absent. Multiple candidates may still need resolution.

04 / WATCH OUT

> Optional wiring must not silently weaken a required invariant.

05 / INTERVIEWER FOLLOW-UP

When would getIfUnique be useful?

Answer: getIfUnique returns the only matching bean, if one exists; it is useful when a default is optional and multiple candidates must not be chosen arbitrarily.

Source guide p. 22 • Beginner • References: F01, F02

CHAPTER 03 • DEPENDENCY INJECTION AND BEAN SELECTION

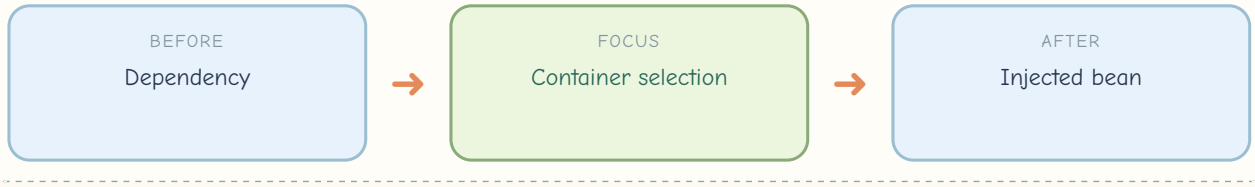
QUESTION 018 / 180

How do you handle circular dependencies?

01 / THE SHORT INTERVIEW ANSWER

> **Treat a cycle as a design signal and separate responsibilities; do not assume every circular graph can or should be auto-resolved.**

DRAW THE IDEA — handle circular dependencies



02 / THE ORIGINAL EXAMPLE

```
Before: OrderService -> PaymentService -> OrderService
After:  CheckoutCoordinator -> OrderService
                            -> PaymentService
```

03 / WHY THIS WORKS

Constructor cycles prevent constructing either dependency first. A coordinator or event boundary can remove the cycle. Lazy wiring can postpone resolution but leaves the architectural dependency intact.

04 / WATCH OUT

Enabling circular references is not a universal repair.

05 / INTERVIEWER FOLLOW-UP

Can event callbacks introduce another cycle?

Answer: Yes. Event listeners that call back into the publishing service can make a logical cycle even if constructor injection is acyclic. Map the full call and event path.

Source guide p. 23 • Beginner • References: F01, F02

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

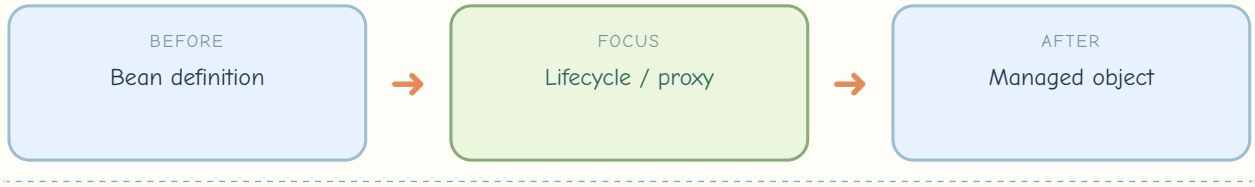
QUESTION 019 / 180

What does singleton scope mean in Spring?

01 / THE SHORT INTERVIEW ANSWER

A singleton is one bean instance per definition per container, not necessarily one object per JVM.

DRAW THE IDEA — singleton scope mean



02 / THE ORIGINAL EXAMPLE

```
@Service
class Counter {
  private int count;
  void increment() { count++; }
}
```

03 / WHY THIS WORKS

Concurrent requests can reach this same Counter. Its scope supplies sharing, not atomicity. Use request-local variables, immutable state, or suitable synchronization. Several application contexts can have separate singleton instances.

04 / WATCH OUT

Singleton scope does not imply thread safety.

05 / INTERVIEWER FOLLOW-UP

Why is mutable request state unsafe in a singleton field?

Answer: Two requests may overwrite one another's user or order data because the same singleton instance receives both calls. Keep request values local or properly request-scoped.

Source guide p. 24 • Beginner • References: F02, F03

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

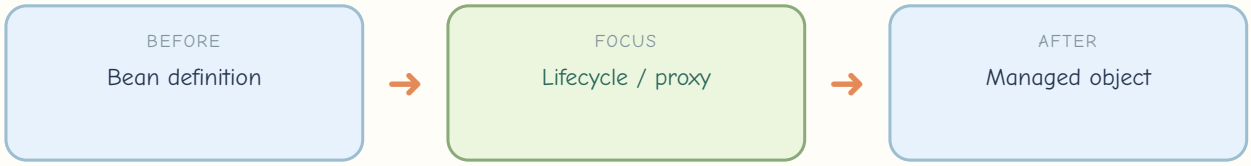
QUESTION 020 / 180

When do prototype and request scopes help?

01 / THE SHORT INTERVIEW ANSWER

> **Prototype creates a fresh instance per container lookup; request scope supplies state associated with one web request.**

DRAW THE IDEA — prototype request scopes help



02 / THE ORIGINAL EXAMPLE

```
@Component
@RequestScope
class RequestData { String correlationId; }
// Injection into a singleton can use a scoped proxy.
```

03 / WHY THIS WORKS

A request-scoped proxy routes each request to its current target. Injecting a prototype once into a singleton instead fixes that one prototype instance at injection time unless lookup is deferred.

04 / WATCH OUT

Spring does not manage complete destruction of ordinary prototypes.

05 / INTERVIEWER FOLLOW-UP

How would ObjectProvider supply a new prototype?

Answer: Inject ObjectProvider<MyPrototype>, then call getObject() each time a new instance is needed. A one-time prototype constructor injection into a singleton does not create per-call instances.

Source guide p. 24 • Beginner • References: F02, F03

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

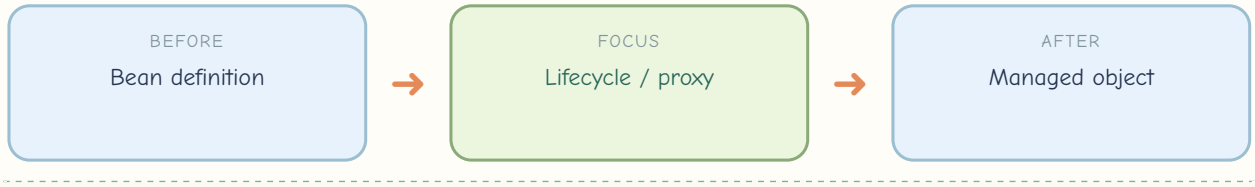
QUESTION 021 / 180

What do @PostConstruct and @PreDestroy do?

01 / THE SHORT INTERVIEW ANSWER

> **They supply managed initialization and destruction callbacks; Boot 3 uses the jakarta.annotation namespace.**

DRAW THE IDEA — @PostConstruct @PreDestroy



02 / THE ORIGINAL EXAMPLE

```
@PostConstruct
void init() { validateConfiguration(); }
@PreDestroy
void close() { releaseOwnedResources(); }
```

03 / WHY THIS WORKS

Initialization runs after dependency injection. Destruction runs when the context destroys a managed bean under normal lifecycle handling. Abrupt process termination can prevent cleanup. Keep heavyweight remote startup work deliberate.

04 / WATCH OUT

> Prototype destruction and forced termination need separate handling.

05 / INTERVIEWER FOLLOW-UP

Why can transaction advice be unreliable in @PostConstruct?

Answer: Lifecycle callbacks may execute before the final externally used proxy is active. Move transactional startup work to a separate proxied service invoked at a supported later lifecycle stage.

Source guide p. 25 • Beginner • References: F02, F03

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

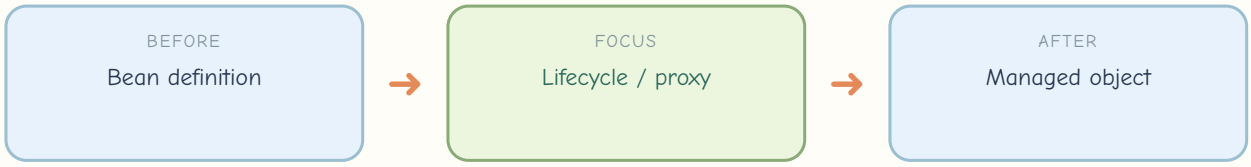
QUESTION 022 / 180

How does BeanPostProcessor differ from a factory post-processor?

01 / THE SHORT INTERVIEW ANSWER

A BeanFactoryPostProcessor changes definition metadata; a BeanPostProcessor works around created bean initialization and can wrap instances.

DRAW THE IDEA — BeanPostProcessor factory post-processo



02 / THE ORIGINAL EXAMPLE

```
Definitions -> BeanFactoryPostProcessor
Instances   -> BeanPostProcessor
AOP advice  -> often exposed through a proxy after processing
```

03 / WHY THIS WORKS

Changing a property definition belongs before ordinary object creation. Wrapping an object for advice operates later. Infrastructure post-processors should avoid triggering unrelated eager bean creation.

04 / WATCH OUT

Processing phases are not interchangeable.

05 / INTERVIEWER FOLLOW-UP

Why must some factory methods for post-processors be static?

Answer: A static @Bean factory for a post-processor can be instantiated without prematurely constructing its configuration instance, allowing infrastructure processing in the required phase.

Source guide p. 25 • Beginner • References: F02, F03

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

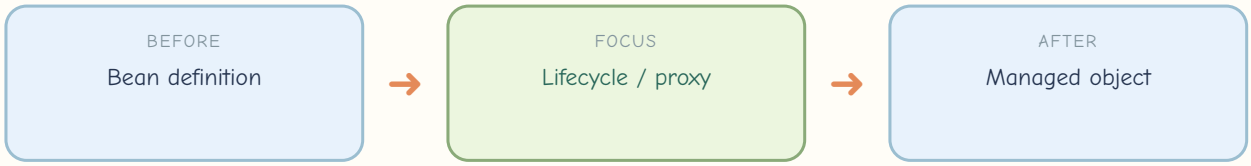
QUESTION 023 / 180

What is the difference between full and lite @Bean configuration?

01 / THE SHORT INTERVIEW ANSWER

Full proxy-enabled configuration intercepts inter-bean calls; lite or proxyBeanMethods=false calls use ordinary Java semantics.

DRAW THE IDEA — difference full lite @Bean



02 / THE ORIGINAL EXAMPLE

```
@Configuration(proxyBeanMethods = false)
class Config {
  @Bean Clock clock() { return Clock.systemUTC(); }
  @Bean Report report(Clock clock) { return new Report(clock); }
}
```

03 / WHY THIS WORKS

Method-parameter injection obtains the managed Clock without relying on intercepted calls. In this mode, directly calling clock() is an ordinary factory invocation and can create another instance.

04 / WATCH OUT

Do not assume every @Bean method call is a container lookup.

05 / INTERVIEWER FOLLOW-UP

When can disabling configuration proxies help?

Answer: Use proxyBeanMethods=false when @Bean methods do not call one another for managed instances. It avoids unnecessary CGLIB interception; inject dependencies as method parameters instead.

Source guide p. 26 • Beginner • References: F02, F03

CHAPTER 04 • SCOPES, LIFECYCLE AND PROXIES

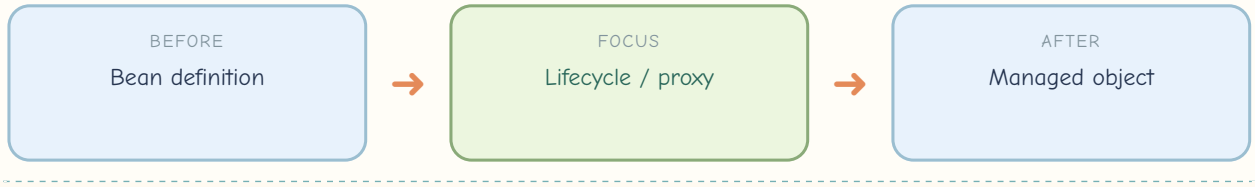
QUESTION 024 / 180

Why does self-invocation bypass common Spring advice?

01 / THE SHORT INTERVIEW ANSWER

> **In proxy-based AOP, a call through this does not pass through the proxy that applies transaction, cache, or async advice.**

DRAW THE IDEA — self-invocation bypass common advice



02 / THE ORIGINAL EXAMPLE

```
@Service
class Importer {
  void run() { save(); } // Direct self-call
  @Transactional
  public void save() { /* write */ }
}
```

03 / WHY THIS WORKS

Calling run through the proxy does not make its internal save call pass through that proxy again. Move the advised boundary to a collaborator or redesign the entry point. AspectJ weaving is a different execution model.

04 / WATCH OUT

> Adding another annotation does not change this call path.

05 / INTERVIEWER FOLLOW-UP

Which caller-to-proxy path would activate the transaction?

Answer: A caller must invoke the advised method on the Spring-managed proxy, for example another service injected with the transactional bean. A this.save() self-call bypasses that proxy.

Source guide p. 26 • Beginner • References: F02, F03

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

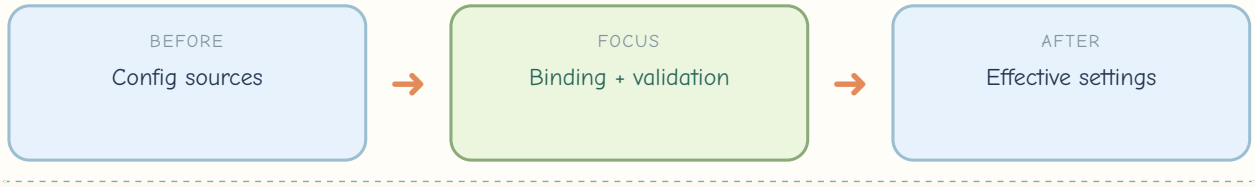
QUESTION 025 / 180

How do @Value and @ConfigurationProperties differ?

01 / THE SHORT INTERVIEW ANSWER

@Value suits individual expressions; @ConfigurationProperties binds a related, typed group of settings.

DRAW THE IDEA — @Value @ConfigurationProperties



02 / THE ORIGINAL EXAMPLE

```
@ConfigurationProperties(prefix = "payment")
public record PaymentProperties(Duration timeout, URI baseUrl) {}
// Register with @ConfigurationPropertiesScan or an enabling annotation.
```

03 / WHY THIS WORKS

payment.timeout=2s binds into Duration. Typed groups support validation, structured ownership, and metadata. @Value can evaluate SpEL but is less suitable for a large configuration contract.

04 / WATCH OUT

A properties class must be registered to participate.

05 / INTERVIEWER FOLLOW-UP

Where would you reject a zero timeout?

Answer: Put a positive constraint on a validated @ConfigurationProperties field, such as @Positive Duration-compatible custom validation. Reject zero during startup binding, before creating the client.

Source guide p. 27 • Beginner • References: B05

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

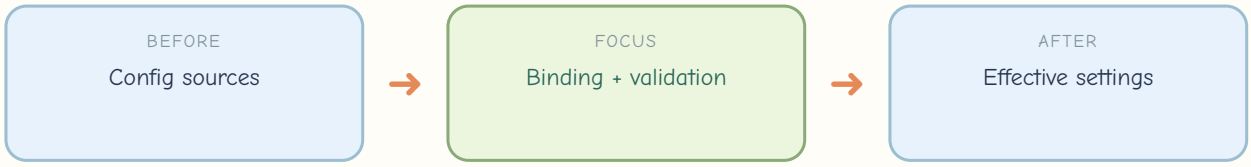
QUESTION 026 / 180

Which property wins when the same setting appears in several sources?

01 / THE SHORT INTERVIEW ANSWER

Boot has a documented source order; for normal runs, command-line arguments override environment variables and application files.

DRAW THE IDEA — property wins same setting



02 / THE ORIGINAL EXAMPLE

```
# application.properties:
server.port=8080
# Environment: SERVER_PORT=8081
java -jar app.jar --server.port=8082
# Effective port: 8082 under ordinary command-line handling
```

03 / WHY THIS WORKS

Later, higher-precedence sources replace earlier values. Tests and programmatic customization add more sources, so do not memorize this simplified subset as the entire order. Diagnose the effective value and origin.

04 / WATCH OUT

Editing a file may have no effect when an environment override exists.

05 / INTERVIEWER FOLLOW-UP

Does spring.config.location add to or replace default locations?

Answer: spring.config.location replaces the default search locations. spring.config.additional-location adds new locations while retaining defaults. Confirm import/order rules for the selected Boot version.

Source guide p. 27 • Beginner • References: B05

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

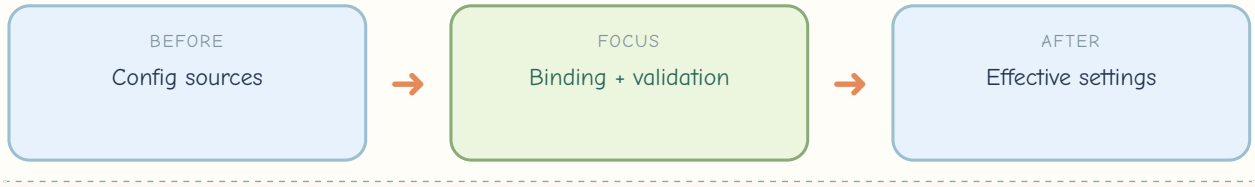
QUESTION 027 / 180

What are profiles and how are they activated?

01 / THE SHORT INTERVIEW ANSWER

Profiles select configuration or bean definitions for an environment or capability, using an explicitly active profile set.

DRAW THE IDEA — profiles they activated



02 / THE ORIGINAL EXAMPLE

```
@Profile("dev")
@Bean
MailSender fakeMail() { return new FakeMailSender(); }
# Launch: java -jar app.jar --spring.profiles.active=dev
```

03 / WHY THIS WORKS

The dev bean participates only when its condition matches. Profiles should express controlled variants, not conceal essential production dependencies. Profile-specific files can supply settings for the same codebase.

04 / WATCH OUT

A profile name is not a security boundary.

05 / INTERVIEWER FOLLOW-UP

Can several profiles be active together?

Answer: Yes. Activate a comma-separated set with spring.profiles.active, or via a supported profile group. Conflicts still follow property-source precedence.

Source guide p. 28 • Beginner • References: B05

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

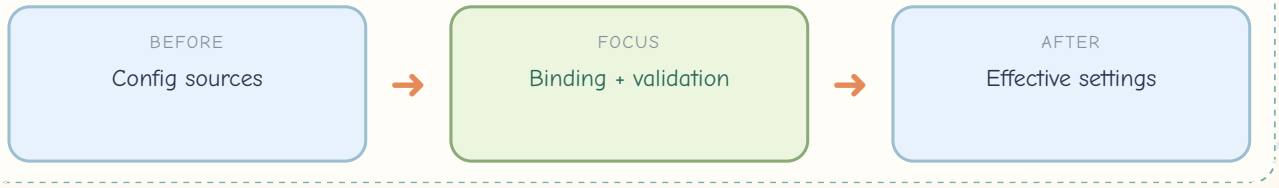
QUESTION 028 / 180

How do you validate configuration at startup?

01 / THE SHORT INTERVIEW ANSWER

Apply Bean Validation constraints to registered @ConfigurationProperties and enable validation through @Validated.

DRAW THE IDEA — validate configuration startup



02 / THE ORIGINAL EXAMPLE

```
@Validated
@ConfigurationProperties("client")
public record ClientProperties(@NotNull URI baseUrl,
                               @Min(1) int maxAttempts) {}
```

03 / WHY THIS WORKS

With the validation dependency and properties registration in place, invalid bound values fail startup. This detects a missing URL or zero retry attempts before requests arrive.

04 / WATCH OUT

Primitive default values can hide omitted settings without constraints.

05 / INTERVIEWER FOLLOW-UP

How would you validate a nested properties object?

Answer: Annotate the nested property with @Valid and add its own constraints; register the properties bean with validation enabled so invalid nested values fail startup.

Source guide p. 28 • Beginner • References: B05

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

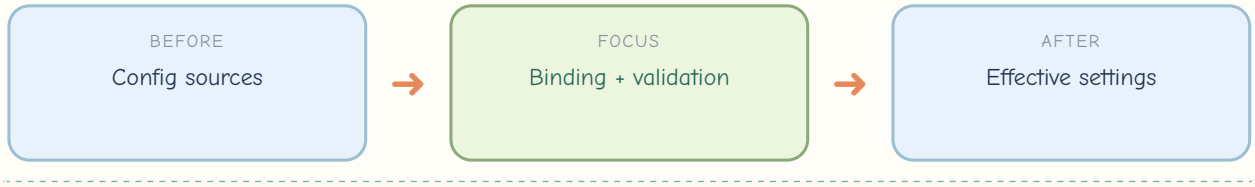
QUESTION 029 / 180

How should secrets be configured?

01 / THE SHORT INTERVIEW ANSWER

Inject secrets from an appropriate secret store or protected runtime source and avoid committing, logging, or exposing them.

DRAW THE IDEA — secrets configured



02 / THE ORIGINAL EXAMPLE

```
spring.datasource.password=${DB_PASSWORD}
# DB_PASSWORD is supplied at runtime.
# Restrict process, platform, and management access to this value.
```

03 / WHY THIS WORKS

A placeholder externalizes the secret; it does not encrypt the environment. Use a managed secret mechanism and rotation policy appropriate to deployment. Sanitize diagnostic outputs and avoid including secrets in exception messages.

04 / WATCH OUT

Environment variables alone are not an automatic secrecy guarantee.

05 / INTERVIEWER FOLLOW-UP

How will rotated credentials reach pooled connections?

Answer: Rotate secrets through the approved provider and explicitly refresh or recreate clients/connections as needed. Existing pooled connections may keep the old credentials until closed.

Source guide p. 29 • Beginner • References: B05

CHAPTER 05 • EXTERNAL CONFIGURATION AND PROFILES

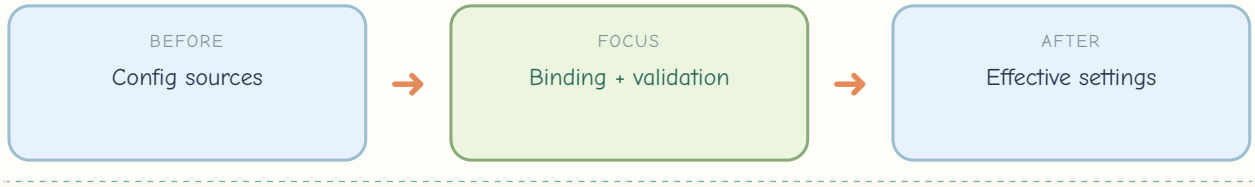
QUESTION 030 / 180

How do configuration imports and locations differ?

01 / THE SHORT INTERVIEW ANSWER

Imports load additional config data; spring.config.location replaces default search locations, while additional-location adds to them.

DRAW THE IDEA — configuration imports locations



02 / THE ORIGINAL EXAMPLE

```
spring.config.import=optional:file:./local-overrides.properties
# optional tolerates this file being absent.
# For required production config, absence should usually fail.
```

03 / WHY THIS WORKS

The optional prefix deliberately changes missing-resource behavior. Configuration order and profile rules determine the effective result. Use a clear required-versus-optional policy instead of silently accepting incomplete production settings.

04 / WATCH OUT

A config import is not automatic live refresh.

05 / INTERVIEWER FOLLOW-UP

What would make a missing secret mount fatal?

Answer: Use an appropriate non-optional config import/location or validate the secret as required at startup. Missing required configuration should fail fast and keep readiness false.

Source guide p. 29 • Beginner • References: B05

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

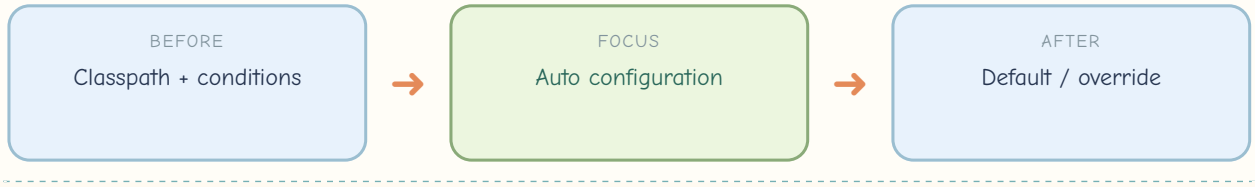
QUESTION 031 / 180

How does auto-configuration choose beans?

01 / THE SHORT INTERVIEW ANSWER

It evaluates conditions based on available classes, properties, existing bean definitions, and application type.

DRAW THE IDEA — auto-configuration choose beans



02 / THE ORIGINAL EXAMPLE

```
@AutoConfiguration
@ConditionalOnClass(PaymentClient.class)
class PaymentAutoConfiguration {
  @Bean @ConditionalOnMissingBean
  PaymentClient paymentClient() { return new PaymentClient(); }
}
```

03 / WHY THIS WORKS

The library class must be present. If the user already supplies a matching PaymentClient bean, this default backs off. This sketch assumes the client has an appropriate constructor.

04 / WATCH OUT

Conditions do not infer business intent beyond their tests.

05 / INTERVIEWER FOLLOW-UP

Why should an infrastructure default back off?

Answer: A starter default must back off under @ConditionalOnMissingBean so an application can supply its own implementation without duplicate candidates or an accidental override.

Source guide p. 30 • Beginner • References: B03, B04

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

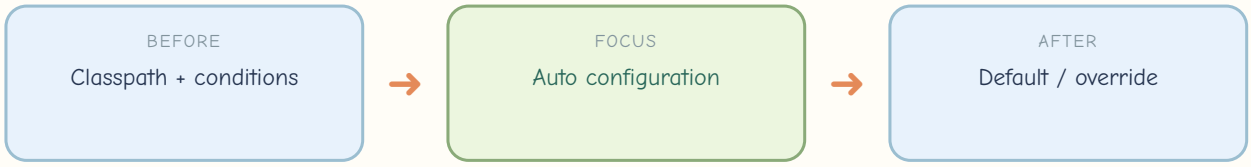
QUESTION 032 / 180

How do you register a Boot 3 auto-configuration in a library?

01 / THE SHORT INTERVIEW ANSWER

List its class in META-INF/spring/ org.springframework.boot.autoconfigure.AutoConfiguration.imports.

DRAW THE IDEA — register auto-configuration library



02 / THE ORIGINAL EXAMPLE

```
# File inside the published library jar:
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
# Contents, one class per line:
com.example.payment.PaymentAutoConfiguration
```

03 / WHY THIS WORKS

Boot discovers candidates through this imports resource. Keep auto-configuration outside consumer component scanning and use explicit imports for related components. The older spring.factories registration advice is version-sensitive.

04 / WATCH OUT

This resource is not application.properties.

05 / INTERVIEWER FOLLOW-UP

How would you verify registration in the built jar?

Answer: Inspect META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports inside the built jar and run a small context test proving the configuration loads.

Source guide p. 31 • Beginner • References: B03, B04

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

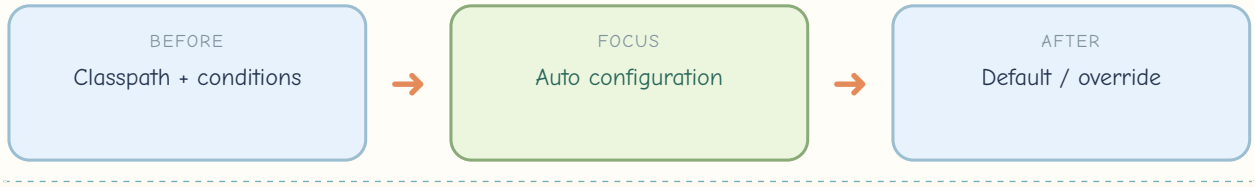
QUESTION 033 / 180

What does @ConditionalOnProperty actually check?

01 / THE SHORT INTERVIEW ANSWER

It tests selected property values using its prefix, name, havingValue, and matchIfMissing policy.

DRAW THE IDEA — @ConditionalOnProperty actually check



02 / THE ORIGINAL EXAMPLE

```
@Bean
@ConditionalOnProperty(prefix="audit", name="enabled", havingValue="true")
AuditSink auditSink() { return new AuditSink(); }
# audit.enabled=true -> candidate enabled; absent -> disabled
```

03 / WHY THIS WORKS

The explicit comparison and default make activation clear. Conditions choose definitions during startup; changing a file later does not automatically create or remove a running bean.

04 / WATCH OUT

Collection-indexed properties need careful handling.

05 / INTERVIEWER FOLLOW-UP

When would matchIfMissing=true be appropriate?

Answer: Use it only when absence should mean enabled by design; for sensitive or expensive features prefer explicit opt-in instead of matchIfMissing=true.

Source guide p. 31 • Beginner • References: B03, B04

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

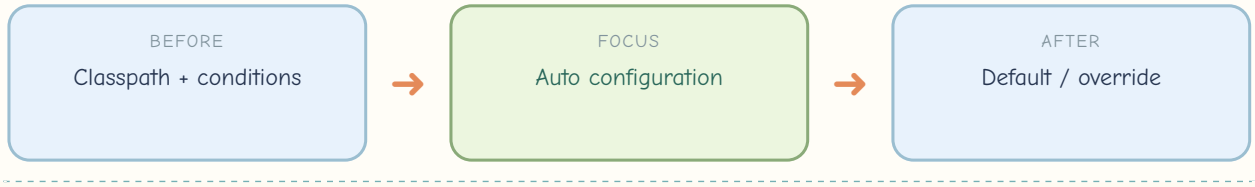
QUESTION 034 / 180

How can you exclude an unwanted auto-configuration?

01 / THE SHORT INTERVIEW ANSWER

> Use a targeted exclusion when the capability is intentionally absent, or provide a supported custom bean when replacing a default.

DRAW THE IDEA — exclude unwanted auto-configuration



02 / THE ORIGINAL EXAMPLE

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
class App { /* launch code omitted */ }
```

03 / WHY THIS WORKS

For the Boot 3.x baseline, this disables DataSource auto-configuration. It is appropriate for a service that does not use JDBC. If persistence is required, fixing its properties is usually better than excluding its setup.

04 / WATCH OUT

> Exclusion can hide a broken requirement.

05 / INTERVIEWER FOLLOW-UP

How would you exclude a class not on the compile classpath?

Answer: Use spring.autoconfigure.exclude with the auto-configuration's fully qualified name, or the configuration exclusion mechanism accepting class names; avoid requiring an absent class literal.

Source guide p. 32 • Beginner • References: B03, B04

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

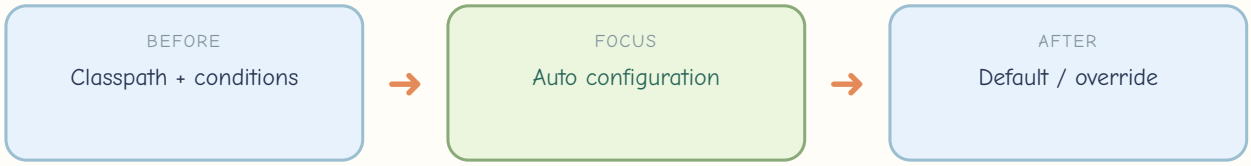
QUESTION 035 / 180

How should a company starter be designed?

01 / THE SHORT INTERVIEW ANSWER

Separate useful dependencies from conditional setup, expose typed properties, and permit consumer overrides.

DRAW THE IDEA — company starter designed



02 / THE ORIGINAL EXAMPLE

```
payment-spring-boot-starter -> dependencies
payment-spring-boot-autoconfigure -> conditional beans
payment-client -> core transport behavior
```

03 / WHY THIS WORKS

Keep business logic independent of Boot where useful. Test activation, absence, user overrides, and invalid settings. Pick a company namespace rather than pretending to be an official Spring starter.

04 / WATCH OUT

Broad component scanning can register unwanted consumer beans.

05 / INTERVIEWER FOLLOW-UP

Which defaults should consumers be able to replace?

Answer: Let consumers replace clients, timeouts, meters, authentication adapters and resource settings through properties or beans, while keeping secure defaults and validation.

Source guide p. 32 • Beginner • References: B03, B04

CHAPTER 06 • AUTO-CONFIGURATION AND CUSTOM STARTERS

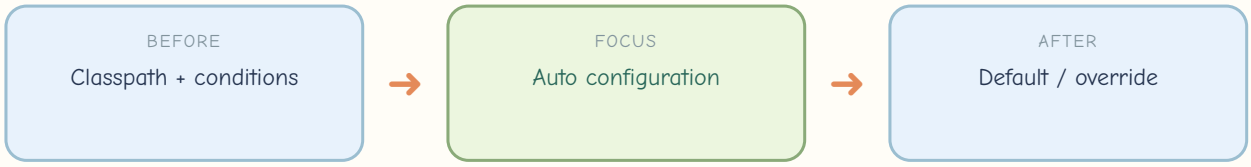
QUESTION 036 / 180

How do you test an auto-configuration without a full application?

01 / THE SHORT INTERVIEW ANSWER

Use ApplicationContextRunner to create small contexts with chosen properties and user beans.

DRAW THE IDEA — test auto-configuration without full



02 / THE ORIGINAL EXAMPLE

```
new ApplicationContextRunner()
  .withConfiguration(AutoConfigurations.of(PaymentAutoConfiguration.class))
  .withPropertyValues("payment.enabled=true")
  .run(ctx -> assertThat(ctx).hasSingleBean(PaymentClient.class));
```

03 / WHY THIS WORKS

This test isolates conditional wiring. Add a user-provided bean case and assert that the default backs off. Also test missing classes where applicable using a filtered class loader.

04 / WATCH OUT

One successful activation test misses negative conditions.

05 / INTERVIEWER FOLLOW-UP

How would you test binding failure?

Answer: Use ApplicationContextRunner with invalid properties and assert startup failure and its cause. A successful-context assertion alone cannot prove invalid configuration is rejected.

Source guide p. 33 • Beginner • References: B03, B04

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 037 / 180

What does DispatcherServlet do?

01 / THE SHORT INTERVIEW ANSWER

It coordinates MVC handler mapping, handler invocation, exception resolution, and response rendering or conversion.

DRAW THE IDEA — DispatcherServlet



02 / THE ORIGINAL EXAMPLE

```
GET /orders/7
 -> servlet filters
 -> DispatcherServlet
 -> matching controller method
 -> message converter -> HTTP response
```

03 / WHY THIS WORKS

The servlet is the MVC front controller. It delegates each responsibility to supporting strategies instead of containing every controller itself. Security filters can reject a request before MVC ever runs.

04 / WATCH OUT

A request need not reach the controller to produce an error response.

05 / INTERVIEWER FOLLOW-UP

Which component turns a DTO into JSON?

Answer: HttpMessageConverter performs response serialization, commonly MappingJackson2HttpMessageConverter for JSON in Boot 3 MVC. Handler mapping and adapters select/invoke the controller.

Source guide p. 34 • Intermediate • References: W01, W02

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 038 / 180

How do @Controller and @RestController differ?

01 / THE SHORT INTERVIEW ANSWER

@Controller normally participates in view handling; @RestController adds class-level @ResponseBody semantics.

DRAW THE IDEA — @Controller @RestController



02 / THE ORIGINAL EXAMPLE

```
@RestController
class GreetingController {
  @GetMapping("/hello") String hello() { return "Hello"; }
}
// Response body: Hello, rather than a view named Hello.
```

03 / WHY THIS WORKS

With @Controller alone, a String return commonly identifies a view unless @ResponseBody is applied. A REST controller can still use ResponseEntity for status and headers.

04 / WATCH OUT

A returned String is not automatically a JSON object.

05 / INTERVIEWER FOLLOW-UP

What controls response content type?

Answer: Produces/Accept negotiation, converter support and media-type configuration determine Content-Type. @RestController only changes how handler return values are treated.

Source guide p. 35 • Intermediate • References: W01, W02

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 039 / 180

How do path variables, query parameters, and request bodies differ?

01 / THE SHORT INTERVIEW ANSWER

Path variables identify route segments; query parameters refine a request; bodies carry a structured payload.

DRAW THE IDEA — path variables query parameters



02 / THE ORIGINAL EXAMPLE

```
@PostMapping("/customers/{id}/notes")
void add(@PathVariable long id, @RequestParam boolean notify,
         @RequestBody NoteRequest body) { /* use inputs */ }
// POST /customers/7/notes?notify=true with JSON body
```

03 / WHY THIS WORKS

Spring converts route and query strings to declared types and uses a message converter for the body. Missing or malformed inputs follow binding and error-handling rules.

04 / WATCH OUT

@RequestBody is not the same as form-data binding.

05 / INTERVIEWER FOLLOW-UP

How would an optional query parameter be declared?

Answer: Use @RequestParam(required=false), Optional<T>, or a defaultValue according to the contract. Distinguish omitted values from explicitly empty values when it matters.

Source guide p. 35 • Intermediate • References: W01, W02

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 040 / 180

How do filters, interceptors and AOP differ?

01 / THE SHORT INTERVIEW ANSWER

Filters wrap servlet traffic; MVC interceptors surround handler execution; AOP advises eligible bean method calls.

DRAW THE IDEA — filters interceptors AOP



02 / THE ORIGINAL EXAMPLE

```
Filter -> request/response and security boundary
Interceptor -> handler metadata and MVC lifecycle
AOP -> service method calls through an advice boundary
```

03 / WHY THIS WORKS

Choose the level that owns the concern. A servlet filter can affect unmapped requests; a handler interceptor can inspect a matched controller. Service authorization or transaction logic often belongs beyond HTTP routing.

04 / WATCH OUT

MVC async dispatch changes callback behavior.

05 / INTERVIEWER FOLLOW-UP

Where should a correlation id be created and cleaned up?

Answer: Create or accept a validated correlation ID near the request filter boundary; put it in scoped logging context and remove it in finally to avoid reuse leaks.

Source guide p. 36 • Intermediate • References: W01, W02

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 041 / 180

How does content negotiation work?

01 / THE SHORT INTERVIEW ANSWER

MVC selects response representation using request preferences and supported converters or declared media types.

DRAW THE IDEA — content negotiation work



02 / THE ORIGINAL EXAMPLE

```
@GetMapping(value="/status", produces="application/json")
Map<String, String> status() { return Map.of("state", "ready"); }
# Accept: application/json -> JSON representation
```

03 / WHY THIS WORKS

The produces condition constrains matching; message converters serialize the result. Unsupported media preferences can produce 406, and an unsupported request body media type can produce 415.

04 / WATCH OUT

Accept and Content-Type have different meanings.

05 / INTERVIEWER FOLLOW-UP

What happens if no converter can write the returned type?

Answer: Spring may return an error such as 406 Not Acceptable when no supported representation matches the client's Accept header; an unwritable declared return can yield converter-related failures.

Source guide p. 36 • Intermediate • References: W01, W02

CHAPTER 07 • SPRING MVC REQUEST PROCESSING

QUESTION 042 / 180

Should you add @EnableWebMvc in a Boot MVC application?

01 / THE SHORT INTERVIEW ANSWER

> **Usually use WebMvcConfigurer for targeted customization; @EnableWebMvc takes control of MVC configuration and affects Boot defaults.**

DRAW THE IDEA — add @EnableWebMvc MVC



02 / THE ORIGINAL EXAMPLE

```
@Configuration
class WebConfig implements WebMvcConfigurer {
  @Override public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(new TimingInterceptor());
  }
}
```

03 / WHY THIS WORKS

This extends MVC behavior without intentionally replacing its Boot configuration path. Use @EnableWebMvc only when you want that level of ownership and have checked the resulting differences.

04 / WATCH OUT

> A common tutorial annotation can change important defaults.

05 / INTERVIEWER FOLLOW-UP

How would you customize converters without replacing all defaults?

Answer: Implement WebMvcConfigurer or register selected converters/customizers so Boot's other MVC defaults remain. @EnableWebMvc changes the auto-configuration baseline significantly.

Source guide p. 37 • Intermediate • References: W01, W02

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 043 / 180

When should an API return 200, 201, or 204?

01 / THE SHORT INTERVIEW ANSWER

> **Use 200 for a successful representation, 201 for newly created resources, and 204 for success without a body.**

DRAW THE IDEA — API return 200 201



02 / THE ORIGINAL EXAMPLE

```
@PostMapping("/orders")
ResponseEntity<OrderDto> create(@RequestBody CreateOrder request) {
  OrderDto dto = service.create(request);
  return ResponseEntity.created(URI.create("/orders/" + dto.id())).body(dto);
}
```

03 / WHY THIS WORKS

The response sends 201 plus Location and the created representation. A 204 response must not include a response body. Choose codes from the operation's actual contract rather than returning 200 for every outcome.

04 / WATCH OUT

HTTP success does not explain every domain state.

05 / INTERVIEWER FOLLOW-UP

What status represents an asynchronous accepted operation?

Answer: 202 Accepted means processing has started or been queued but is not finished. Supply an operation ID or status URL; never imply downstream success.

Source guide p. 38 • Intermediate • References: W01, W03

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 044 / 180

How do PUT and PATCH differ?

01 / THE SHORT INTERVIEW ANSWER

PUT requests replacement semantics for the target representation; PATCH requests a defined partial modification format.

DRAW THE IDEA — PUT PATCH



02 / THE ORIGINAL EXAMPLE

```
PUT /customers/7  -> replacement representation
PATCH /customers/7 -> selected modifications
// Define omitted fields, explicit nulls, and version conflicts.
```

03 / WHY THIS WORKS

A partial DTO blindly copied over an entity can erase fields unintentionally. PATCH needs documented semantics, such as merge-patch or a domain-specific operation. Neither verb alone implements concurrency control.

04 / WATCH OUT

PATCH is not automatically idempotent.

05 / INTERVIEWER FOLLOW-UP

How would you prevent lost updates?

Answer: Use an ETag/If-Match condition or a version field checked atomically. A stale version should fail clearly, typically with 412 or a documented conflict.

Source guide p. 38 • Intermediate • References: W01, W03

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 045 / 180

What makes an HTTP operation idempotent?

01 / THE SHORT INTERVIEW ANSWER

> **Repeating the same intended operation has the same intended effect as one execution, even if response details differ.**

DRAW THE IDEA — makes HTTP operation idempotent



02 / THE ORIGINAL EXAMPLE

```
DELETE /orders/7 -> first request removes it
DELETE /orders/7 -> later request does not remove another resource
// The second status may differ without changing idempotent intent.
```

03 / WHY THIS WORKS

Idempotence concerns the intended server effect, not identical responses or zero audit activity. For POST payments, implement operation identity and deduplication when safe retries are required.

04 / WATCH OUT

Retrying a timed-out POST can duplicate business effects.

05 / INTERVIEWER FOLLOW-UP

How should an idempotency-key record be persisted?

Answer: Persist a scoped unique key, request fingerprint, processing state and final result in a durable transaction. Replay must return the same result or a clear in-progress state.

Source guide p. 39 • Intermediate • References: W01, W03

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 046 / 180

Why return DTOs instead of exposing JPA entities?

01 / THE SHORT INTERVIEW ANSWER

DTOs keep the API contract separate from persistence, control fields, and avoid accidental lazy loading and internal data exposure.

DRAW THE IDEA — return DTOs instead exposing



02 / THE ORIGINAL EXAMPLE

```
record CustomerDto(long id, String displayName) {}
CustomerDto toDto(Customer c) {
  return new CustomerDto(c.getId(), c.getDisplayName());
}
```

03 / WHY THIS WORKS

Only approved fields are serialized. Persistence associations and passwords do not become API output merely because they exist in an entity. Mapping work can also be performed through query projections.

04 / WATCH OUT

A DTO alone does not enforce row-level authorization.

05 / INTERVIEWER FOLLOW-UP

When is a projection better than loading a full entity?

Answer: A projection reads only required columns and avoids loading the full entity graph. Use it for read-only API views; use entities when domain changes need managed state.

Source guide p. 39 • Intermediate • References: W01, W03

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 047 / 180

How should pagination and sorting be designed?

01 / THE SHORT INTERVIEW ANSWER

Bound page size, whitelist sort fields, and provide a deterministic order with a tie breaker.

DRAW THE IDEA — pagination sorting designed



02 / THE ORIGINAL EXAMPLE

```
Pageable p = PageRequest.of(0, 20,
    Sort.by("createdAt").descending().and(Sort.by("id")));
Page<Order> result = repository.findAll(p);
```

03 / WHY THIS WORKS

The id tie breaker stabilizes rows sharing a timestamp. Offset pagination can become expensive or shift under concurrent writes; cursor-based designs may suit large changing datasets.

04 / WATCH OUT

Raw client sort expressions must not become arbitrary query text.

05 / INTERVIEWER FOLLOW-UP

What are the tradeoffs of Page versus Slice?

Answer: Page usually executes/needs total-count information; Slice determines only whether another segment exists, often avoiding an expensive count query. Define consistent ordering.

Source guide p. 40 • Intermediate • References: W01, W03

CHAPTER 08 • REST CONTRACTS AND API DESIGN

QUESTION 048 / 180

How should API versioning be chosen?

01 / THE SHORT INTERVIEW ANSWER

> **Version when compatibility changes, choosing a consistent path, header, or media-type policy backed by deprecation rules.**

DRAW THE IDEA — API versioning chosen



02 / THE ORIGINAL EXAMPLE

```
GET /api/v1/customers/7 -> legacy contract
GET /api/v2/customers/7 -> revised incompatible contract
// Keep both supported during a planned migration window.
```

03 / WHY THIS WORKS

A new optional field may not require a new version, but changed field meaning can. Document client compatibility, sunset dates, and observable usage. Version numbers do not replace a migration strategy.

04 / WATCH OUT

Creating a new version for every code change creates needless complexity.

05 / INTERVIEWER FOLLOW-UP

How would you know when v1 can be retired?

Answer: Use traffic telemetry, consumer inventory, contract tests and a communicated deprecation window to verify no supported client needs v1. Do not retire it just because v2 exists.

Source guide p. 40 • Intermediate • References: W01, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

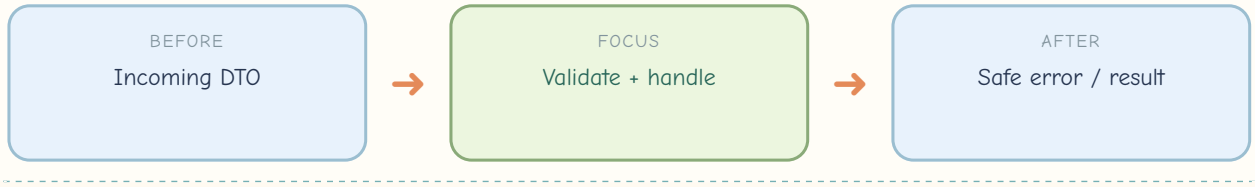
QUESTION 049 / 180

What is the purpose of @Valid on a request body?

01 / THE SHORT INTERVIEW ANSWER

> It asks the configured Bean Validation integration to validate the bound object before ordinary controller execution.

DRAW THE IDEA — purpose @Valid request body



02 / THE ORIGINAL EXAMPLE

```
record CreateUser(@NotBlank String name, @Email String email) {}
@PostMapping("/users")
UserDto create(@Valid @RequestBody CreateUser request) {
  return service.create(request);
}
```

03 / WHY THIS WORKS

With the validation starter, blank names fail before the method body runs. JSON syntax and type conversion failures are separate from constraint failures. @Email does not establish mailbox ownership.

04 / WATCH OUT

> @Valid is not a business uniqueness check.

05 / INTERVIEWER FOLLOW-UP

How do you validate nested DTOs?

Answer: Annotate the nested property with @Valid and give nested fields constraints. The outer DTO's @Valid triggers traversal into nested validated objects.

Source guide p. 41 • Intermediate • References: W02, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

QUESTION 050 / 180

How do @Valid and @Validated differ?

01 / THE SHORT INTERVIEW ANSWER

> **@Valid is the standard cascading marker; @Validated adds Spring integration and validation-group selection.**

DRAW THE IDEA — @Valid @Validated



02 / THE ORIGINAL EXAMPLE

```
interface Create {}
record Request(@NotNull(groups=Create.class) String reference) {}
// @Validated(Create.class) selects the Create validation group.
```

03 / WHY THIS WORKS

Groups allow different constraints for different operations, but can complicate a shared DTO. In MVC versions with built-in method validation, class-level @Validated changes the mechanism used; select it deliberately.

04 / WATCH OUT

> Not every method constraint follows the same exception path.

05 / INTERVIEWER FOLLOW-UP

When would separate create/update DTOs be clearer?

Answer: Create and update often have different required fields and semantics. Separate DTO types can make those contracts clearer than complex validation groups.

Source guide p. 41 • Intermediate • References: W02, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

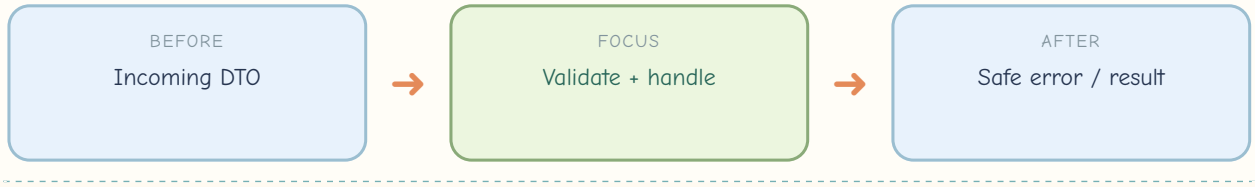
QUESTION 051 / 180

How does @RestControllerAdvice help?

01 / THE SHORT INTERVIEW ANSWER

> It centralizes REST exception handlers and shared controller concerns without duplicating catch logic in every endpoint.

DRAW THE IDEA — @RestControllerAdvice help



02 / THE ORIGINAL EXAMPLE

```
@RestControllerAdvice
class Errors {
  @ExceptionHandler(OrderNotFound.class)
  ResponseEntity<ProblemDetail> missing(OrderNotFound ex) {
    return ResponseEntity.of(ProblemDetail.forStatus(404)).build();
  }
}
```

03 / WHY THIS WORKS

This maps a specific domain absence to 404. Add a stable error code and safe details appropriate to the API. Infrastructure failures should follow a separate contract.

04 / WATCH OUT

> Servlet-filter security failures are not automatically handled here.

05 / INTERVIEWER FOLLOW-UP

Where should authentication failures be translated?

Answer: Authentication failures should usually be handled in Spring Security's AuthenticationEntryPoint/ filter-chain mechanisms, rather than relying only on @RestControllerAdvice.

Source guide p. 42 • Intermediate • References: W02, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

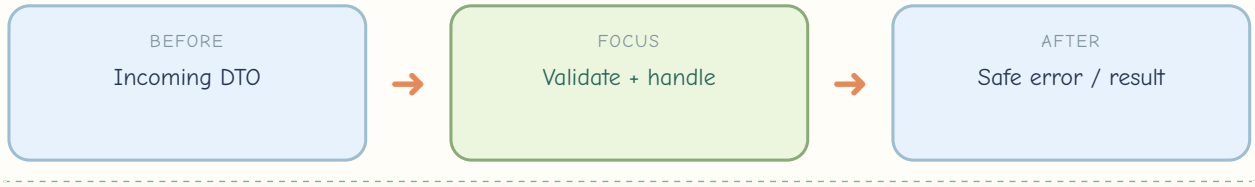
QUESTION 052 / 180

What is ProblemDetail useful for?

01 / THE SHORT INTERVIEW ANSWER

> **It represents a standard problem response shape with status, title, detail, and extensible fields.**

DRAW THE IDEA — ProblemDetail useful



02 / THE ORIGINAL EXAMPLE

```
ProblemDetail p = ProblemDetail.forStatusAndDetail(
    HttpStatus.CONFLICT, "Order version changed");
p.setProperty("code", "ORDER_VERSION_CONFLICT");
// Return with matching HTTP status and a safe response body.
```

03 / WHY THIS WORKS

A stable code lets clients branch without parsing human messages. Avoid including database traces, internal paths, or secret values. Consistency matters across validation and domain failures.

04 / WATCH OUT

A problem response needs a matching HTTP status.

05 / INTERVIEWER FOLLOW-UP

How will clients correlate it with server diagnostics?

Answer: Include a safe trace/correlation ID in the problem response and emit the same ID in protected logs or traces. Do not expose stack traces or secrets.

Source guide p. 42 • Intermediate • References: W02, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

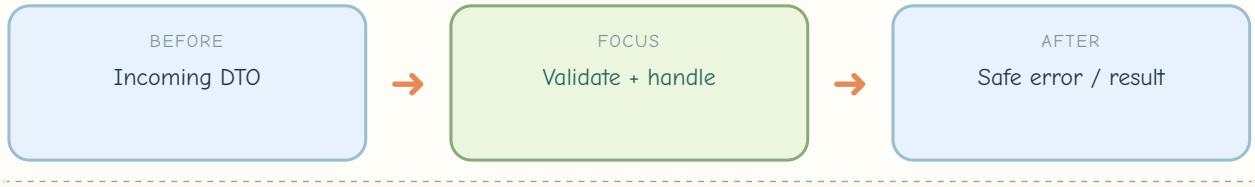
QUESTION 053 / 180

How should validation failures be reported clearly?

01 / THE SHORT INTERVIEW ANSWER

Return safe field-level codes and messages while distinguishing malformed syntax, constraint violations, and business conflicts.

DRAW THE IDEA — validation failures reported clearly



02 / THE ORIGINAL EXAMPLE

```
{
  "status":400,
  "code":"VALIDATION_FAILED",
  "errors":[{"field":"name","code":"NotBlank"}]
}
```

03 / WHY THIS WORKS

This illustrative contract tells the client which field to repair. Map the actual MVC exception types for your version and validation path. Do not echo confidential rejected values.

04 / WATCH OUT

All failures should not become one generic 500 or 200 response.

05 / INTERVIEWER FOLLOW-UP

How would object-level cross-field constraints be represented?

Answer: Use a class-level Bean Validation constraint for fields that must agree, such as start <= end. Report a stable field-group or object-level violation.

Source guide p. 43 • Intermediate • References: W02, W03

CHAPTER 09 • VALIDATION AND EXCEPTION RESPONSES

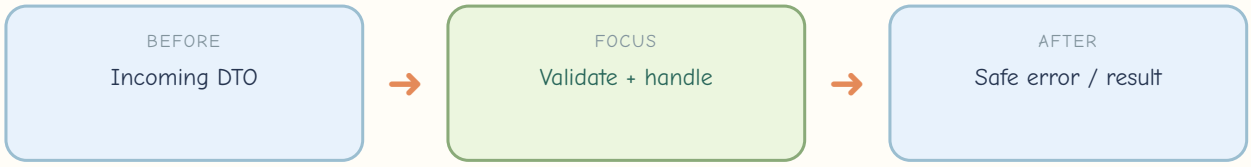
QUESTION 054 / 180

Where should uniqueness and cross-resource rules be enforced?

01 / THE SHORT INTERVIEW ANSWER

> **Enforce business rules in the service and authoritative data constraints in the database, not only in request validation.**

DRAW THE IDEA — uniqueness cross-resource rules enforce



02 / THE ORIGINAL EXAMPLE

```
Check email availability -> insert user
Two concurrent requests can both pass the check.
Unique database constraint -> final authority
Constraint conflict -> controlled application response
```

03 / WHY THIS WORKS

An early check improves usability but cannot eliminate the race. Handle the specific constraint failure and define normalization before uniqueness. Domain conflicts need a clear status and safe error code.

04 / WATCH OUT

> A pre-insert SELECT is not a concurrency guarantee.

05 / INTERVIEWER FOLLOW-UP

How would you distinguish duplicate email from another constraint failure?

Answer: Inspect the violated constraint name or trusted SQL error classification, translate known uniqueness failures into a domain-specific conflict, and leave unknown failures generic.

Source guide p. 43 • Intermediate • References: W02, W03

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

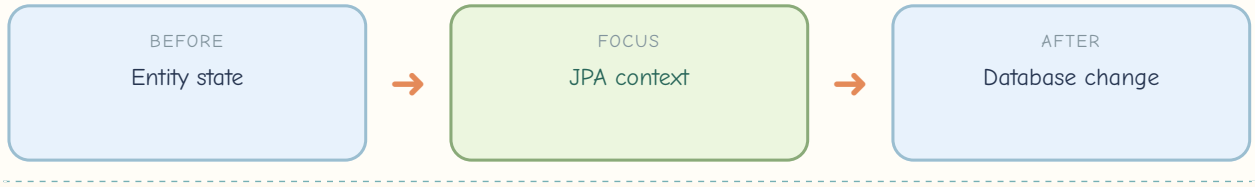
QUESTION 055 / 180

How do JPA, Hibernate and Spring Data JPA differ?

01 / THE SHORT INTERVIEW ANSWER

JPA defines persistence contracts, Hibernate is a provider, and Spring Data JPA supplies repository abstractions on top.

DRAW THE IDEA — JPA Hibernate Data JPA



02 / THE ORIGINAL EXAMPLE

```
interface CustomerRepository extends JpaRepository<Customer, Long> {
  Optional<Customer> findByEmail(String email);
}
```

03 / WHY THIS WORKS

Spring Data builds the repository implementation, which uses JPA infrastructure backed by the chosen provider. Hibernate is common but not the specification itself. A derived query still runs a database query.

04 / WATCH OUT

Repository abstractions do not remove the need to understand SQL.

05 / INTERVIEWER FOLLOW-UP

When would a custom repository implementation help?

Answer: Use a custom repository when an operation needs complex query construction, batch updates, vendor features or persistence behavior not expressible cleanly by derived methods.

Source guide p. 44 • Intermediate • References: D01, D02

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

QUESTION 056 / 180

What are transient, managed, detached and removed entities?

01 / THE SHORT INTERVIEW ANSWER

They describe whether an entity is new, tracked by a persistence context, no longer tracked, or scheduled for deletion.

DRAW THE IDEA — transient managed detached removed



02 / THE ORIGINAL EXAMPLE

```
Customer c = new Customer(); // Transient
entityManager.persist(c);     // Managed in this context
entityManager.detach(c);      // Detached
// remove requires an appropriately managed entity.
```

03 / WHY THIS WORKS

Managed changes can be detected and written at flush. Editing a detached object does not automatically update the database. State and transaction boundaries determine which operations are valid.

04 / WATCH OUT

A non-null id does not always prove an object is managed.

05 / INTERVIEWER FOLLOW-UP

How would contains(entity) help diagnosis?

Answer: EntityManager.contains(entity) tells whether that exact instance is currently managed in the persistence context; it does not establish that the database row still exists.

Source guide p. 44 • Intermediate • References: D01, D02

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

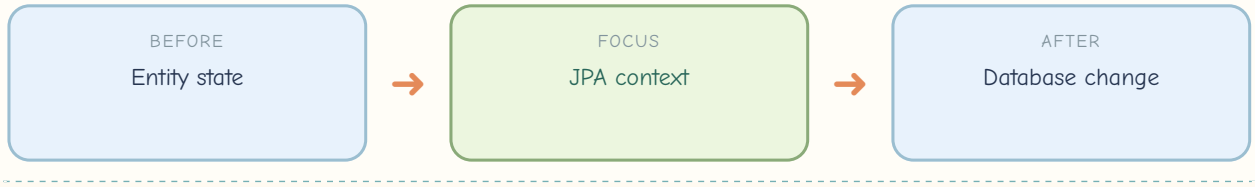
QUESTION 057 / 180

How do persist, merge and repository save differ?

01 / THE SHORT INTERVIEW ANSWER

> **persist makes a new entity managed; merge copies state into a managed instance; Spring Data save chooses persistence behavior from newness detection.**

DRAW THE IDEA — persist merge repository save



02 / THE ORIGINAL EXAMPLE

```
Customer detached = loadDetached();
Customer managed = entityManager.merge(detached);
// Continue with managed, not an assumption that detached changed state.
```

03 / WHY THIS WORKS

merge returns the managed object used by the persistence context. The supplied object need not become managed. Spring Data newness can depend on version, identifier, or Persistable configuration.

04 / WATCH OUT

> save is not an unconditional SQL INSERT.

05 / INTERVIEWER FOLLOW-UP

How would an assigned id affect newness detection?

Answer: With an assigned non-null ID and no version/newness policy, Spring Data may treat the entity as existing and call merge. Implement Persistable or appropriate new-state detection when needed.

Source guide p. 45 • Intermediate • References: D01, D02

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

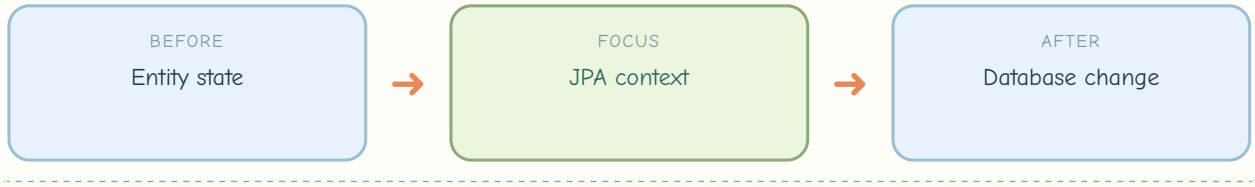
QUESTION 058 / 180

What is dirty checking?

01 / THE SHORT INTERVIEW ANSWER

> **The persistence provider detects relevant changes to managed entities and synchronizes them at flush within the transaction.**

DRAW THE IDEA — dirty checking



02 / THE ORIGINAL EXAMPLE

```
@Transactional
public void rename(long id, String name) {
  Customer c = repository.findById(id).orElseThrow();
  c.setName(name);
}
```

03 / WHY THIS WORKS

The entity is managed in the active transaction, so changes can be flushed without another save call. Flush timing and SQL depend on provider behavior. The update still commits only if the transaction succeeds.

04 / WATCH OUT

> Detached changes do not participate in dirty checking.

05 / INTERVIEWER FOLLOW-UP

What happens if the transaction rolls back after flush?

Answer: Flushing sends SQL but does not commit it. If the enclosing transaction rolls back successfully, its database changes are undone despite the earlier flush.

Source guide p. 45 • Intermediate • References: D01, D02

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

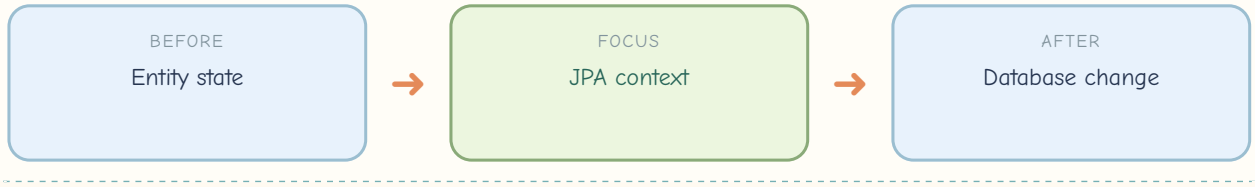
QUESTION 059 / 180

Who owns a bidirectional JPA relationship?

01 / THE SHORT INTERVIEW ANSWER

The owning side controls the relationship mapping update; maintain both Java sides for consistent in-memory behavior.

DRAW THE IDEA — Who owns bidirectional JPA



02 / THE ORIGINAL EXAMPLE

```
@OneToMany(mappedBy="order") List<Line> lines;
// In Line:
@ManyToOne Order order;
// Add helper: order.lines.add(line); line.order = order;
```

03 / WHY THIS WORKS

Here Line.order owns the relationship. Updating only the inverse collection can leave database linkage unchanged. Cascade and orphanRemoval govern lifecycle behavior separately from ownership.

04 / WATCH OUT

Cascade is not a fetch policy.

05 / INTERVIEWER FOLLOW-UP

When should orphanRemoval delete a child?

Answer: Use orphanRemoval when removing a child from its owning parent collection truly means deleting it. Do not use it for children shared or transferred to another aggregate.

Source guide p. 46 • Intermediate • References: D01, D02

CHAPTER 10 • SPRING DATA JPA AND ENTITY OWNERSHIP

QUESTION 060 / 180

Why should schema migrations replace ddl-auto=update in production?

01 / THE SHORT INTERVIEW ANSWER

Versioned migrations make changes reviewable and repeatable; automatic update is not a controlled compatibility and rollback plan.

DRAW THE IDEA — schema migrations replace ddl-auto



02 / THE ORIGINAL EXAMPLE

```
spring.jpa.hibernate.ddl-auto=validate
# Apply approved Flyway or Liquibase migrations separately.
# Expand schema, migrate consumers, then contract later.
```

03 / WHY THIS WORKS

Validation checks mapping compatibility; it does not perform a migration. A versioned change can be reviewed for locks, data backfill, compatibility, and recovery. Use one clear owner for schema initialization.

04 / WATCH OUT

Rolling application code back may not roll schema back safely.

05 / INTERVIEWER FOLLOW-UP

How would you add a required column without downtime?

Answer: Add the nullable column, deploy compatible readers/writers, backfill existing rows, then enforce NOT NULL only after every writer supplies the value and old versions are gone.

Source guide p. 46 • Intermediate • References: D01, D02

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

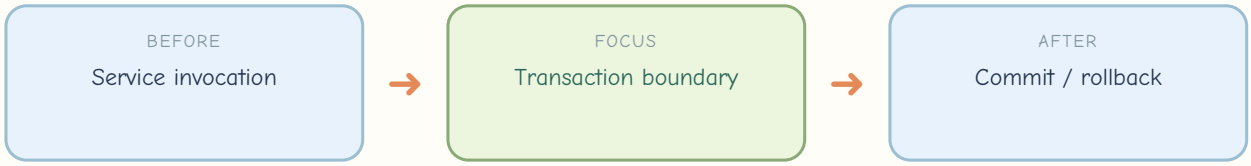
QUESTION 061 / 180

Where should @Transactional usually be placed?

01 / THE SHORT INTERVIEW ANSWER

> **Place it on a service boundary that owns one database unit of work, rather than scattering unrelated transactions across controller steps.**

DRAW THE IDEA — @Transactional usually placed



02 / THE ORIGINAL EXAMPLE

```
@Transactional
public void transfer(long from, long to, BigDecimal amount) {
  debit(from, amount);
  credit(to, amount);
  recordTransfer(from, to, amount);
}
```

03 / WHY THIS WORKS

The database operations participate in one local transaction when the method is invoked through its configured transaction boundary and uses the right manager. Internal helpers can share it.

04 / WATCH OUT

> This does not automatically make a remote HTTP call atomic with the database.

05 / INTERVIEWER FOLLOW-UP

How would you publish a durable transfer event?

Answer: Write the business change and an outbox event record in the same local transaction. A separate relay publishes the event with retry and deduplication.

Source guide p. 47 • Intermediate • References: D03, D04

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

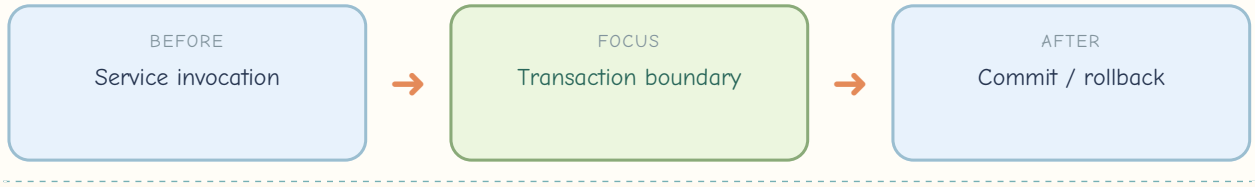
QUESTION 062 / 180

Which failures trigger default rollback?

01 / THE SHORT INTERVIEW ANSWER

By default, runtime exceptions and Errors trigger rollback; checked exceptions require an explicit rule when rollback is intended.

DRAW THE IDEA — failures trigger default rollback



02 / THE ORIGINAL EXAMPLE

```
@Transactional(rollbackFor = IOException.class)
public void importOrders() throws IOException {
  repository.save(order);
  throw new IOException("Invalid source");
}
```

03 / WHY THIS WORKS

The explicit rule makes this checked failure roll back the participating transaction. If the exception is caught and suppressed inside the boundary, the interceptor may not see it. Global rules can also be configured, so state your assumptions.

04 / WATCH OUT

catch-and-return can accidentally commit partial work.

05 / INTERVIEWER FOLLOW-UP

How would a rollback-only marker affect the outer caller?

Answer: When a participating inner transaction marks rollback-only, the outer call may complete its logic but fail on commit with UnexpectedRollbackException. Do not claim it committed.

Source guide p. 48 • Intermediate • References: D03, D04

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

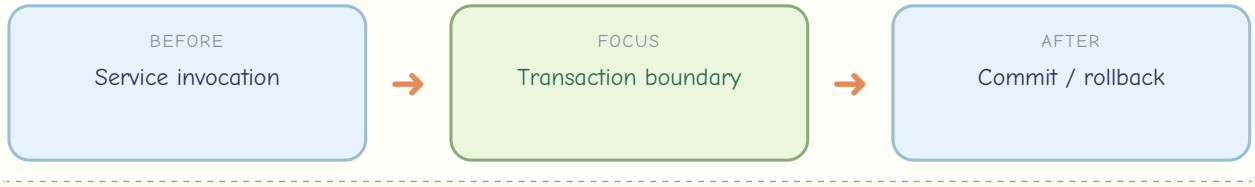
QUESTION 063 / 180

How do REQUIRED and REQUIRES_NEW differ?

01 / THE SHORT INTERVIEW ANSWER

> **REQUIRED joins an existing transaction or starts one; REQUIRES_NEW uses an independent transaction while suspending an existing one.**

DRAW THE IDEA — REQUIRED REQUIRES_NEW



02 / THE ORIGINAL EXAMPLE

```
Outer REQUIRED -> connection/transaction A
  Inner REQUIRES_NEW -> independent transaction B
  B can commit even if A later rolls back
```

03 / WHY THIS WORKS

This requires crossing an advised boundary. Independent propagation can need another database connection while the outer one is held. Use it for a deliberate consistency contract, not as a generic exception workaround.

04 / WATCH OUT

Pool exhaustion can arise under nested independent transactions.

05 / INTERVIEWER FOLLOW-UP

When is NESTED behavior different?

Answer: NESTED uses savepoints within one physical transaction when supported; REQUIRES_NEW uses an independent transaction and often another connection. Support varies by manager.

Source guide p. 48 • Intermediate • References: D03, D04

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

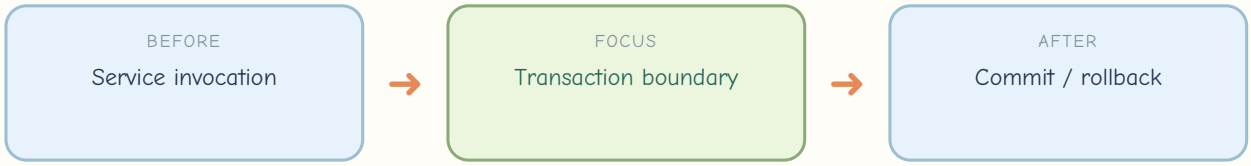
QUESTION 064 / 180

What does transaction isolation control?

01 / THE SHORT INTERVIEW ANSWER

> **It controls permitted interactions with concurrent database transactions, according to the database's supported semantics.**

DRAW THE IDEA — transaction isolation control



02 / THE ORIGINAL EXAMPLE

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public BalanceDto readBalance(long id) { return loadBalance(id); }
```

03 / WHY THIS WORKS

READ_COMMITTED generally prevents dirty reads but does not universally guarantee repeatable reads or eliminate every anomaly. Database implementation and query shape matter. Isolation on a joining method may not replace the already-running transaction's isolation.

04 / WATCH OUT

A label alone is not a proof of business correctness.

05 / INTERVIEWER FOLLOW-UP

Which anomaly threatens a check-then-update invariant?

Answer: A check-then-update can suffer non-repeatable reads, lost updates or write skew depending on constraints and isolation. Use a conditional write or locking to enforce the actual invariant.

Source guide p. 49 • Intermediate • References: D03, D04

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

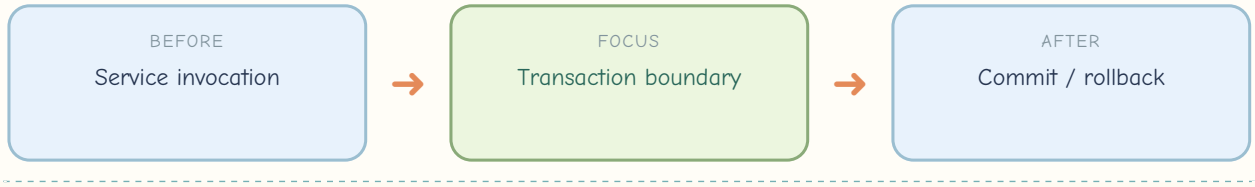
QUESTION 065 / 180

Does readOnly=true prevent every write?

01 / THE SHORT INTERVIEW ANSWER

> It is a transaction hint that may affect optimizations and connection behavior; it is not a universal write-prohibition guarantee.

DRAW THE IDEA — readOnly true prevent every



02 / THE ORIGINAL EXAMPLE

```
@Transactional(readOnly = true)
public List<CustomerDto> list() {
  return repository.findAll().stream().map(this::toDto).toList();
}
```

03 / WHY THIS WORKS

Use the hint for genuinely read-only work. Behavior depends on the manager, provider, and database. Do not rely on this annotation as a security or data-integrity constraint.

04 / WATCH OUT

An ordinary collection mutation is not blocked by a database read-only hint.

05 / INTERVIEWER FOLLOW-UP

How would you enforce database-level write restrictions?

Answer: Use a database read-only account, transaction-level read-only mode where supported, or explicit permissions. Spring's readOnly hint alone is not a security boundary.

Source guide p. 49 • Intermediate • References: D03, D04

CHAPTER 11 • TRANSACTIONS AND CONSISTENCY

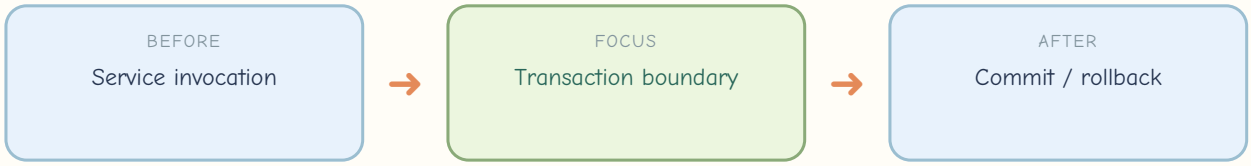
QUESTION 066 / 180

Do transactions propagate automatically to @Async tasks?

01 / THE SHORT INTERVIEW ANSWER

> **Ordinary imperative transactions are thread-bound, so a new async thread does not inherit the caller's transaction.**

DRAW THE IDEA — transactions propagate automatically @A



02 / THE ORIGINAL EXAMPLE

```
@Transactional
public void submit() {
  repository.save(order);
  asyncNotifier.send(order.getId()); // Another thread
}
```

03 / WHY THIS WORKS

The task can run before the transaction commits and may not see the row. Use after-commit dispatch when best-effort is acceptable or a durable outbox for recoverable delivery. Reactive transactions use Reactor context instead of this thread model.

04 / WATCH OUT

> Passing a managed entity between threads introduces another ownership problem.

05 / INTERVIEWER FOLLOW-UP

How would you handle task submission followed by rollback?

Answer: A new asynchronous task has a separate transaction context. Publishing a task before commit can expose rolled-back data; enqueue durable intent in an outbox instead.

Source guide p. 50 • Intermediate • References: D03, D04

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

QUESTION 067 / 180

What is the N+1 query problem?

01 / THE SHORT INTERVIEW ANSWER

> **One query loads parents and later access triggers additional child queries, often one per parent.**

DRAW THE IDEA — N+1 query problem



02 / THE ORIGINAL EXAMPLE

```
// Load 20 orders in one query.
orders.forEach(o -> o.getLines().size());
// With lazy associations, this can trigger many more queries.
```

03 / WHY THIS WORKS

The extra round trips can dominate latency. A suitable fetch join, entity graph, projection, or batch-fetch policy can help. Measure the generated SQL for the actual access pattern rather than assuming one repository call equals one query.

04 / WATCH OUT

EAGER does not guarantee one efficient query.

05 / INTERVIEWER FOLLOW-UP

Why is collection fetch-join pagination risky?

Answer: Joining a to-many collection multiplies rows and can make SQL limit/offset paginate joined rows rather than distinct root entities. Page IDs first or use targeted batch fetching.

Source guide p. 51 • Intermediate • References: D01, D02, D03

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

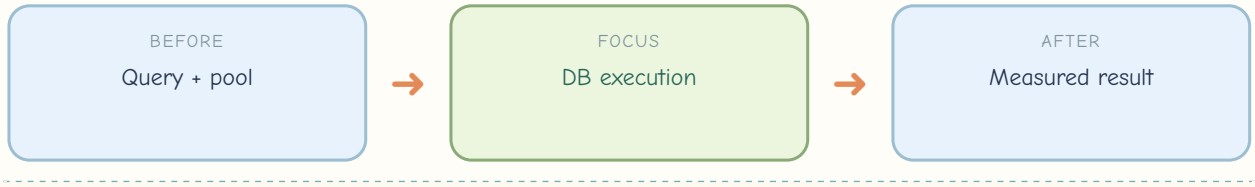
QUESTION 068 / 180

What is LazyInitializationException telling you?

01 / THE SHORT INTERVIEW ANSWER

Code tried to initialize lazy state without an appropriate open persistence context or session.

DRAW THE IDEA — LazyInitializationException telling



02 / THE ORIGINAL EXAMPLE

```
Order detached = service.loadOrder(id);
// Outside its persistence boundary:
detached.getLines().size(); // May fail when not already loaded
```

03 / WHY THIS WORKS

Choose the required result shape inside a deliberate boundary. DTO projection or explicit fetch planning is preferable to making every relationship eager. Open-in-view can mask the boundary while allowing queries during response serialization.

04 / WATCH OUT

Lazy failure is often an ownership design issue.

05 / INTERVIEWER FOLLOW-UP

How would you count lines without loading all of them?

Answer: Use a count query such as SELECT COUNT(line) at the database instead of loading every child entity or initializing its collection.

Source guide p. 51 • Intermediate • References: D01, D02, D03

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

QUESTION 069 / 180

How do Page, Slice and keyset pagination differ?

01 / THE SHORT INTERVIEW ANSWER

> **Page includes total-count information, Slice focuses on another-page existence, and keyset uses a stable last-seen key.**

DRAW THE IDEA — Page Slice keyset pagination



02 / THE ORIGINAL EXAMPLE

```
SELECT ... FROM orders
WHERE (created_at, id) < (:lastTime, :lastId)
ORDER BY created_at DESC, id DESC
LIMIT :pageSize
```

03 / WHY THIS WORKS

This SQL is a conceptual keyset pattern; syntax and index design depend on the database. It avoids scanning an ever-growing offset, but needs cursor semantics and matching deterministic order.

04 / WATCH OUT

A count query can be more expensive than fetching the page.

05 / INTERVIEWER FOLLOW-UP

How would filters affect cursor validity?

Answer: A cursor must bind to the same tenant, filters and sort definition. If any changes, reject or restart pagination; always add a unique tie-breaker.

Source guide p. 52 • Intermediate • References: D01, D02, D03

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

QUESTION 070 / 180

How should HikariCP pool size be chosen?

01 / THE SHORT INTERVIEW ANSWER

Align pool capacity with measured database capacity, application concurrency, and the number of service replicas.

DRAW THE IDEA — HikariCP pool size chosen



02 / THE ORIGINAL EXAMPLE

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=2000
# Ten replicas can allow up to 200 connections in aggregate.
```

03 / WHY THIS WORKS

These values illustrate units and capacity, not a recommendation. A larger pool may increase database contention. Separate connection acquisition wait from query execution and socket deadlines.

04 / WATCH OUT

Hikari connection-timeout is not a query timeout.

05 / INTERVIEWER FOLLOW-UP

How does a rolling deployment change peak connection count?

Answer: Rolling deployments may run old and new pods together, with maxSurge adding temporary replicas. Multiply each replica's max pool size by peak concurrent pods, plus other database clients.

Source guide p. 52 • Intermediate • References: D01, D02, D03

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

QUESTION 071 / 180

How do optimistic and pessimistic locking differ?

01 / THE SHORT INTERVIEW ANSWER

Optimistic locking detects conflicting updates through a version; pessimistic locking obtains database locks during the operation.

DRAW THE IDEA — optimistic pessimistic locking



02 / THE ORIGINAL EXAMPLE

```
@Entity
class Inventory {
  @Id Long id;
  @Version long version;
  int available;
}
```

03 / WHY THIS WORKS

A version-checked update fails if another writer changed the version. Retrying needs fresh state and business revalidation. Pessimistic locking can serialize updates but creates waits and deadlock risks.

04 / WATCH OUT

Versioning does not protect unrelated rows automatically.

05 / INTERVIEWER FOLLOW-UP

Which approach fits a highly contended stock counter?

Answer: For very high contention, an atomic conditional SQL update may outperform repeated optimistic retries. Pessimistic locking is useful when a short critical section must serialize conflicts.

Source guide p. 53 • Intermediate • References: D01, D02, D03

CHAPTER 12 • DATABASE PERFORMANCE AND CONCURRENCY

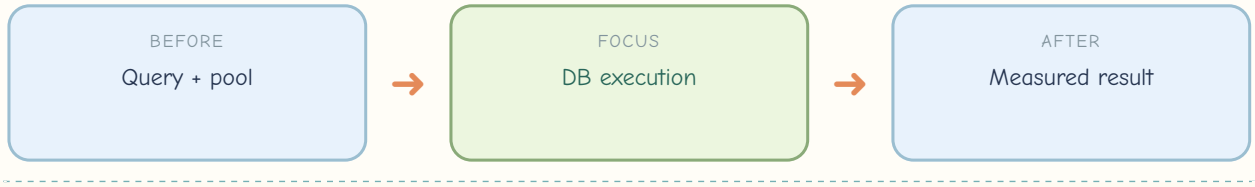
QUESTION 072 / 180

Why can bulk updates leave managed entities stale?

01 / THE SHORT INTERVIEW ANSWER

JPQL or SQL bulk updates operate on rows without applying ordinary per-entity managed-state synchronization.

DRAW THE IDEA — bulk updates leave managed



02 / THE ORIGINAL EXAMPLE

```
@Modifying(clearAutomatically=true, flushAutomatically=true)
@Query("update Customer c set c.active=false where c.lastSeen < :cutoff")
int deactivate(@Param("cutoff") Instant cutoff);
```

03 / WHY THIS WORKS

Run this in an appropriate transaction. Flushing and clearing can reduce stale-state problems, but clearing detaches managed objects and changes ownership. Check version handling and callbacks separately.

04 / WATCH OUT

Bulk update semantics differ from editing each entity.

05 / INTERVIEWER FOLLOW-UP

What happens to existing entity references after clear?

Answer: clear() detaches managed instances; existing references still exist in Java but no longer receive persistence-context updates. Re-read needed entities after a bulk operation.

Source guide p. 53 • Intermediate • References: D01, D02, D03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

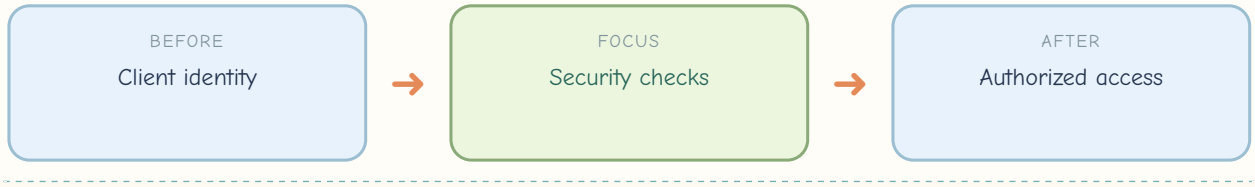
QUESTION 073 / 180

How do authentication and authorization differ?

01 / THE SHORT INTERVIEW ANSWER

Authentication verifies who the caller is; authorization decides which operation that caller may perform.

DRAW THE IDEA — authentication authorization



02 / THE ORIGINAL EXAMPLE

```
Unauthenticated request -> identity may need authentication
Authenticated user -> still needs permission for /admin
// Login success does not imply administrator rights.
```

03 / WHY THIS WORKS

A valid identity can still receive an access-denied response. URL rules are one layer; business-object ownership may need method or service checks. Keep failure contracts distinct.

04 / WATCH OUT

A valid token alone does not authorize every resource.

05 / INTERVIEWER FOLLOW-UP

How would you check tenant ownership?

Answer: Derive tenant identity from an authenticated trusted principal/claim and check both tenant and resource ownership at the service or data access boundary.

Source guide p. 54 • Intermediate • References: S01, S02, S03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

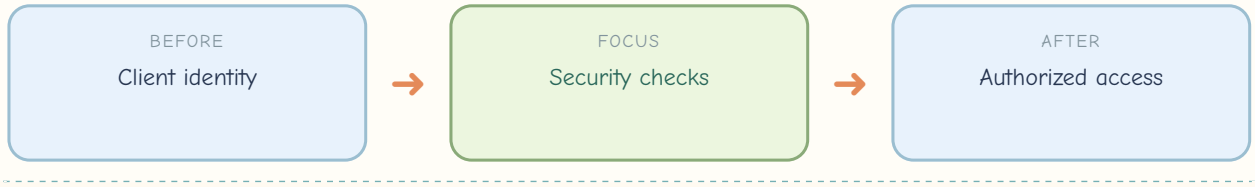
QUESTION 074 / 180

How is security configured in modern Boot applications?

01 / THE SHORT INTERVIEW ANSWER

Declare SecurityFilterChain beans using the supported HttpSecurity DSL; do not base new examples on the removed adapter.

DRAW THE IDEA — security configured modern applications



02 / THE ORIGINAL EXAMPLE

```
@Bean
SecurityFilterChain chain(HttpSecurity http) throws Exception {
  http.authorizeHttpRequests(a -> a.requestMatchers("/public/**").permitAll()
      .anyRequest().authenticated());
  return http.build();
}
```

03 / WHY THIS WORKS

This defines authorization but must be paired with the authentication mechanism you intend, such as resource-server JWT or login. Supplying custom chains means owning their rules, including management endpoints.

04 / WATCH OUT

Matcher order and which chain matches matter.

05 / INTERVIEWER FOLLOW-UP

How would you define a separate actuator chain?

Answer: Use distinct SecurityFilterChain beans with request matchers/order; give actuator routes a restrictive, separately tested rule instead of accidentally inheriting public API access.

Source guide p. 55 • Intermediate • References: S01, S02, S03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

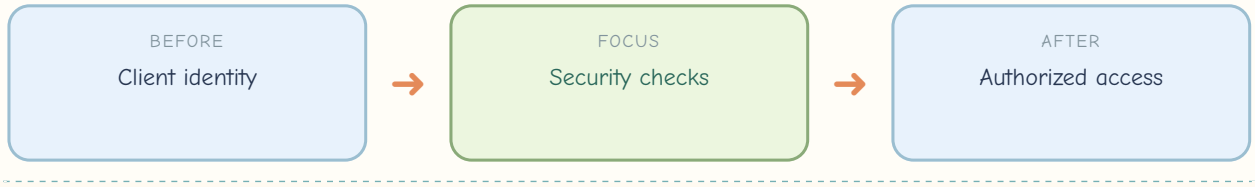
QUESTION 075 / 180

When should CSRF protection be retained?

01 / THE SHORT INTERVIEW ANSWER

Retain it for requests whose credentials are automatically attached by browsers, such as cookie sessions; choose a separate policy for truly bearer-only APIs.

DRAW THE IDEA — CSRF protection retained



02 / THE ORIGINAL EXAMPLE

```
Cookie/session application -> CSRF token required for unsafe methods
Bearer token explicitly sent in Authorization header ->
  assess credential behavior and client architecture
```

03 / WHY THIS WORKS

Stateless does not automatically mean CSRF-safe if browser credentials are still auto-sent. A blanket disable copied from a tutorial can weaken a cookie-based app. CORS addresses a different browser concern.

04 / WATCH OUT

JWT stored in a cookie can still create CSRF exposure.

05 / INTERVIEWER FOLLOW-UP

What does SameSite mitigate and what remains?

Answer: SameSite cookies reduce some cross-site sending, but do not replace CSRF tokens for every supported browser flow, and they do not solve XSS or incorrect authorization.

Source guide p. 55 • Intermediate • References: S01, S02, S03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

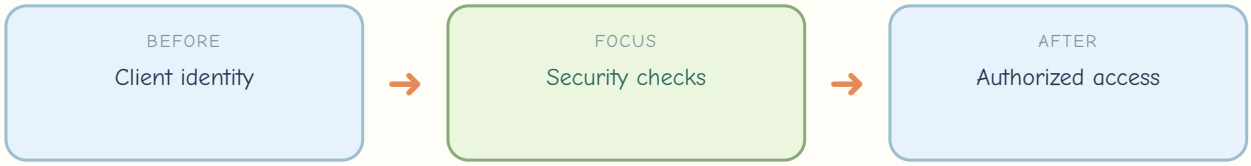
QUESTION 076 / 180

How does CORS differ from authentication?

01 / THE SHORT INTERVIEW ANSWER

> **CORS controls browser cross-origin access rules; it does not establish caller identity or authorize non-browser clients.**

DRAW THE IDEA — CORS authentication



02 / THE ORIGINAL EXAMPLE

```
Allowed origin: https://app.example
Allowed method: GET
Allowed header: Authorization
Preflight OPTIONS -> handled before ordinary bearer authentication
```

03 / WHY THIS WORKS

Configure a precise CORS policy and integrate it with security. Browser preflight generally carries no bearer access token. A curl request is not stopped merely because its Origin is outside a CORS allowlist.

04 / WATCH OUT

> Wildcard origins and credentialed responses have restrictions.

05 / INTERVIEWER FOLLOW-UP

Why can a preflight receive 401 while the endpoint works in Postman?

Answer: Browsers send an unauthenticated OPTIONS preflight. Permit the intended CORS handling before authorization; Postman does not enforce browser CORS rules.

Source guide p. 56 • Intermediate • References: S01, S02, S03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

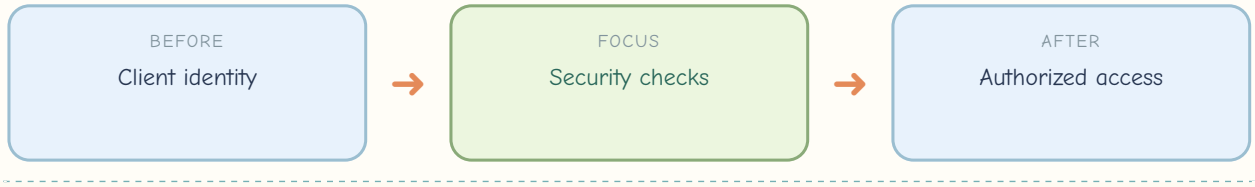
QUESTION 077 / 180

How should passwords be stored?

01 / THE SHORT INTERVIEW ANSWER

Store a salted adaptive password hash using a supported PasswordEncoder; never plaintext or reversible application encryption for login verification.

DRAW THE IDEA — passwords stored



02 / THE ORIGINAL EXAMPLE

```
PasswordEncoder encoder = PasswordEncoderFactories.createDelegatingPasswordEncoder();
String stored = encoder.encode("example-password");
System.out.println(encoder.matches("example-password", stored)); // true
```

03 / WHY THIS WORKS

The stored format identifies the encoder, enabling policy evolution. Compare by matches rather than hashing twice and comparing strings because salts vary. Set cost using deployment measurements.

04 / WATCH OUT

Logs and fixture files can still leak raw credentials.

05 / INTERVIEWER FOLLOW-UP

How would you upgrade stored hashes on successful login?

Answer: On successful login, detect an outdated password-hash encoding and rehash with the current PasswordEncoder policy; never convert to plaintext or skip verification.

Source guide p. 56 • Intermediate • References: S01, S02, S03

CHAPTER 13 • SPRING SECURITY FUNDAMENTALS

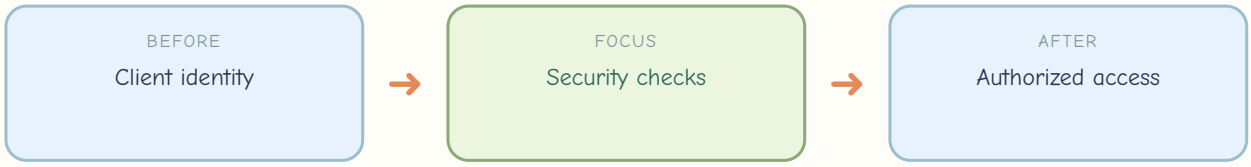
QUESTION 078 / 180

How do method authorization and URL rules complement each other?

01 / THE SHORT INTERVIEW ANSWER

URL rules protect routes; method checks enforce business permissions even through other entry points.

DRAW THE IDEA — method authorization URL rules



02 / THE ORIGINAL EXAMPLE

```
@EnableMethodSecurity
@Configuration
class MethodSecurityConfig {}
@PreAuthorize("hasAuthority('orders:approve')")
public void approve(long id) { /* also enforce ownership */ }
```

03 / WHY THIS WORKS

The method call must cross the advised boundary. A messaging consumer or another controller can reuse protected service logic. Authorities alone may not prove the caller owns this particular order.

04 / WATCH OUT

Self-invocation can bypass method advice.

05 / INTERVIEWER FOLLOW-UP

How would you test an object-level permission denial?

Answer: Use a test identity allowed at the URL but denied by service-level ownership checks. Assert 403 and that the protected record was not changed.

Source guide p. 57 • Intermediate • References: S01, S02, S03

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

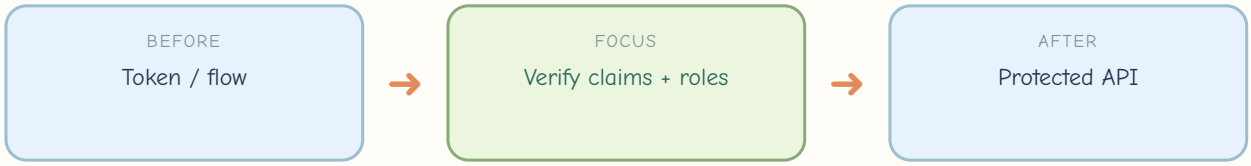
QUESTION 079 / 180

What is the difference between JWT, OAuth and OIDC?

01 / THE SHORT INTERVIEW ANSWER

> **JWT is a token format; OAuth 2.0 is an authorization framework; OIDC adds identity authentication on top of OAuth.**

DRAW THE IDEA — difference JWT OAuth OIDC



02 / THE ORIGINAL EXAMPLE

```
JWT: encoded claims, often signed
OAuth: obtain scoped access to resources
OIDC: identity protocol with an ID token
```

03 / WHY THIS WORKS

An OAuth access token can be opaque rather than JWT. A JWT can exist outside OAuth. OIDC ID tokens are meant for the client identity flow, not automatically for authorization at your API.

04 / WATCH OUT

> These terms do not name interchangeable login mechanisms.

05 / INTERVIEWER FOLLOW-UP

Which token should an API accept?

Answer: APIs accept access tokens intended for that resource server, not ID tokens intended for the client application. Validate issuer, audience, expiry and scopes.

Source guide p. 58 • Intermediate • References: S04, S05

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

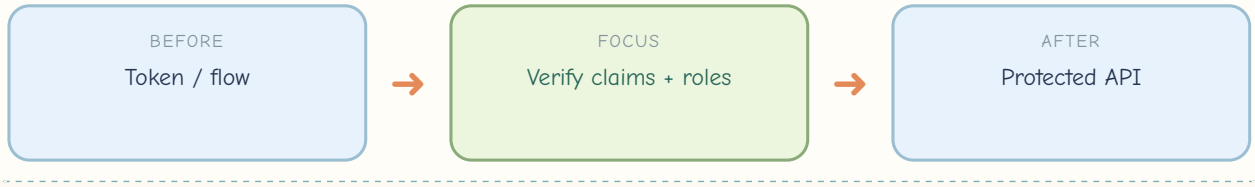
QUESTION 080 / 180

What must a resource server validate in a JWT?

01 / THE SHORT INTERVIEW ANSWER

Validate the signature and allowed algorithms, time claims, trusted issuer, intended audience, and required application claims.

DRAW THE IDEA — must resource server validate



02 / THE ORIGINAL EXAMPLE

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://id.example/issuer
# Add an audience validator for this API as required.
# A configured JWT decoder and resource-server chain validate tokens.
```

03 / WHY THIS WORKS

issuer-uri supports trusted discovery and issuer validation. Audience policy must match the API and configuration; do not assume every setup checks your custom audience automatically. Never trust claims just because Base64 decoding succeeded.

04 / WATCH OUT

Signature validity alone is insufficient.

05 / INTERVIEWER FOLLOW-UP

How do you handle clock skew safely?

Answer: Allow only a small policy-defined clock-skew tolerance for nbf/exp checks. Large leeway makes expired tokens usable for longer and should not mask clock drift.

Source guide p. 59 • Intermediate • References: S04, S05

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

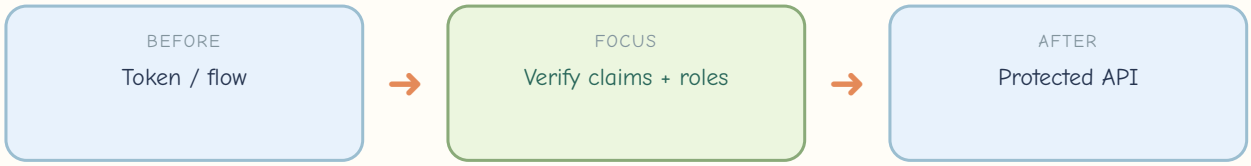
QUESTION 081 / 180

What does a JWT resource-server SecurityFilterChain look like?

01 / THE SHORT INTERVIEW ANSWER

Enable bearer-token authentication with the resource-server JWT DSL and explicit authorization rules.

DRAW THE IDEA — JWT resource-server SecurityFilterChain



02 / THE ORIGINAL EXAMPLE

```
http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(a -> a.anyRequest().authenticated())
    .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
// Apply inside a SecurityFilterChain @Bean and configure a decoder.
```

03 / WHY THIS WORKS

Bearer authentication delegates validation to JwtDecoder. This sketch keeps CSRF policy separate so cookie-based authentication is not accidentally weakened. Issuer, audience, and authority mapping must match your identity provider.

04 / WATCH OUT

The resource server does not mint tokens merely by enabling this DSL.

05 / INTERVIEWER FOLLOW-UP

How are scopes mapped to authorities by default?

Answer: Scopes commonly become SCOPE_<name> GrantedAuthority values. Customize a JwtAuthenticationConverter only when your token claim and prefix contract requires it.

Source guide p. 59 • Intermediate • References: S04, S05

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

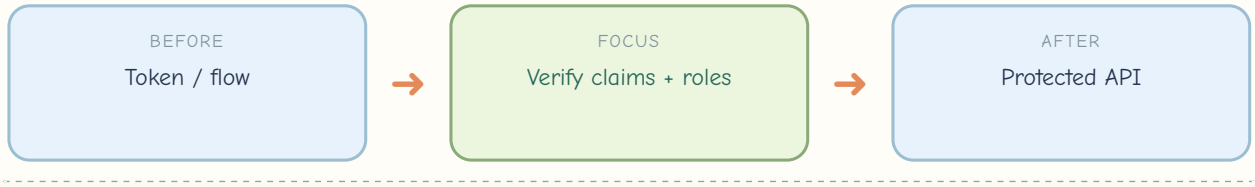
QUESTION 082 / 180

What is authorization code with PKCE used for?

01 / THE SHORT INTERVIEW ANSWER

> It lets a client exchange a redirected authorization code with proof of possession of its original verifier, reducing intercepted-code misuse.

DRAW THE IDEA — authorization code PKCE used



02 / THE ORIGINAL EXAMPLE

```
Client -> authorization request with code challenge + state
Browser -> authorization server -> callback with code
Client -> token endpoint with code + verifier
```

03 / WHY THIS WORKS

Verify redirect URIs and state, and use OIDC nonce validation where required. PKCE is useful for public clients and modern authorization-code flows; it does not replace all client or redirect controls.

04 / WATCH OUT

Do not recommend the password grant for new client designs.

05 / INTERVIEWER FOLLOW-UP

How is a public client different from a confidential client?

Answer: A public client cannot keep a client secret confidential; PKCE protects its authorization-code exchange. A confidential client can authenticate itself securely to the token endpoint.

Source guide p. 60 • Intermediate • References: S04, S05

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

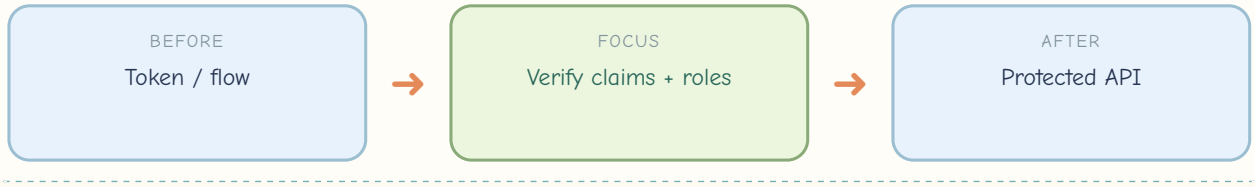
QUESTION 083 / 180

How do access tokens and refresh tokens differ?

01 / THE SHORT INTERVIEW ANSWER

Access tokens authorize resource requests; refresh tokens obtain new access tokens from the authorization server.

DRAW THE IDEA — access tokens refresh tokens



02 / THE ORIGINAL EXAMPLE

```
API request -> access token
Access expires -> client uses refresh token at token endpoint
API should not accept a refresh token as its bearer credential.
```

03 / WHY THIS WORKS

Refresh tokens generally need stronger storage and rotation/reuse policy. Not every flow issues one. Resource servers normally validate the access token; token issuance and refresh belong to the authorization server.

04 / WATCH OUT

Longer access-token lifetime delays revocation effects.

05 / INTERVIEWER FOLLOW-UP

How would rotation detect stolen refresh-token reuse?

Answer: Store refresh-token family state; each rotation invalidates its predecessor. Reuse of an old token signals theft or concurrency misuse and should revoke or quarantine the family.

Source guide p. 60 • Intermediate • References: S04, S05

CHAPTER 14 • JWT, OAUTH 2.0 AND OPENID CONNECT

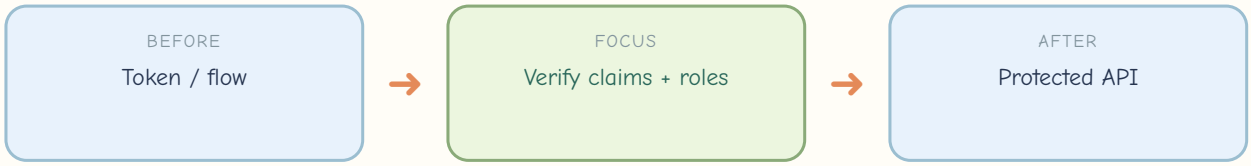
QUESTION 084 / 180

Can a signed JWT be revoked immediately without server state?

01 / THE SHORT INTERVIEW ANSWER

Local signature checks alone cannot know every early revocation; immediate revocation needs state, introspection, or another coordinated policy.

DRAW THE IDEA — signed JWT revoked immediately



02 / THE ORIGINAL EXAMPLE

```
JWT exp in 10 minutes + local verification
Logout now -> token may still validate until exp
Options -> short lifetime, denylist, introspection, key policy
```

03 / WHY THIS WORKS

Choose a revocation requirement explicitly. Key rotation affects many tokens and is not a per-user logout mechanism. JWT payloads are ordinarily readable unless encrypted, so do not store secrets in claims.

04 / WATCH OUT

Deleting a browser token does not invalidate a stolen copy.

05 / INTERVIEWER FOLLOW-UP

When is opaque-token introspection preferable?

Answer: Introspection lets the resource server ask an authorization server whether a token is active, supporting central revocation at the cost of a runtime dependency and caching tradeoffs.

Source guide p. 61 • Intermediate • References: S04, S05

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

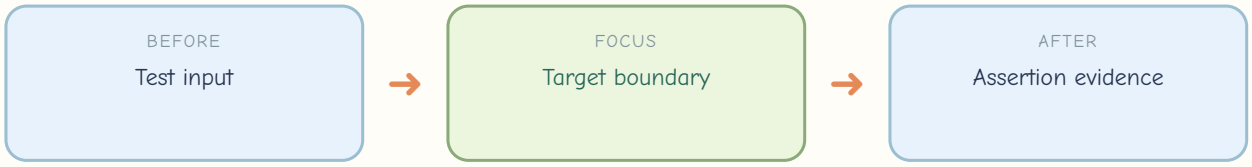
QUESTION 085 / 180

When should you use a plain unit test instead of @SpringBootTest?

01 / THE SHORT INTERVIEW ANSWER

Use plain tests for business behavior that does not require container wiring; reserve context tests for integration questions.

DRAW THE IDEA — plain unit test instead



02 / THE ORIGINAL EXAMPLE

```
@Test
void rejectsNegativeAmount() {
  Checkout checkout = new Checkout(fakeGateway);
  assertThrows(IllegalArgumentException.class,
      () -> checkout.pay(new BigDecimal("-1")));
}
```

03 / WHY THIS WORKS

A direct test is fast and verifies a business rule without a server or database. It cannot verify component discovery, proxy advice, or configuration binding. Pick context tests for those boundaries.

04 / WATCH OUT

Booting Spring for every arithmetic rule wastes time.

05 / INTERVIEWER FOLLOW-UP

Which test would expose a missing bean?

Answer: A context-loading @SpringBootTest or focused ApplicationContextRunner test detects missing beans. Plain unit tests are best when only the algorithm and collaborators need verification.

Source guide p. 62 • Intermediate • References: T01, T02, T03

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

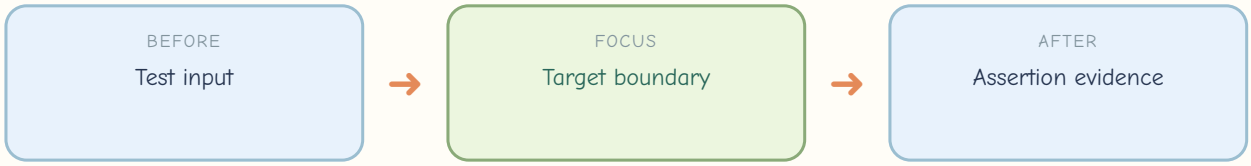
QUESTION 086 / 180

How do @WebMvcTest and @SpringBootTest differ?

01 / THE SHORT INTERVIEW ANSWER

@WebMvcTest loads an MVC-focused slice; @SpringBootTest loads the application's broader Boot context.

DRAW THE IDEA — @WebMvcTest @SpringBootTest



02 / THE ORIGINAL EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderWebTest {
  @Autowired MockMvc mvc;
  @MockitoBean OrderService service;
}
```

03 / WHY THIS WORKS

The mocked service isolates routing, binding, serialization, and selected security behavior. @MockitoBean is available in the modern Spring Framework used by the chosen Boot baseline. A full test is needed when real service wiring is the subject.

04 / WATCH OUT

MVC slices can include security filters.

05 / INTERVIEWER FOLLOW-UP

What would adding addFilters=false remove from coverage?

Answer: It bypasses security filters and therefore misses authentication, authorization and CORS behavior in that test. Do not use it for endpoints whose security contract matters.

Source guide p. 63 • Intermediate • References: T01, T02, T03

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

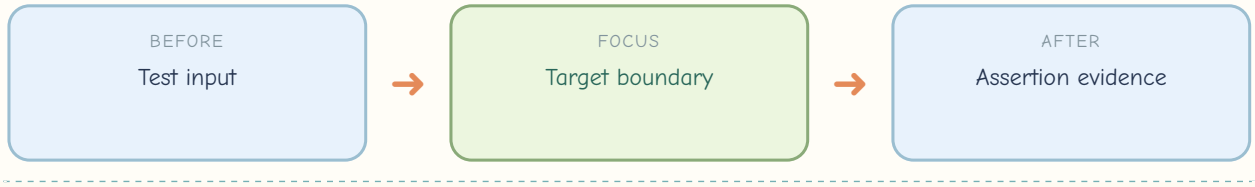
QUESTION 087 / 180

What should a MockMvc endpoint test assert?

01 / THE SHORT INTERVIEW ANSWER

Assert status, headers, body contract, binding/validation behavior, and security behavior appropriate to the endpoint.

DRAW THE IDEA — MockMvc endpoint test assert



02 / THE ORIGINAL EXAMPLE

```
mvc.perform(get("/orders/7").accept(MediaType.APPLICATION_JSON))
   .andExpect(status().isOk())
   .andExpect(jsonPath("$.id").value(7));
```

03 / WHY THIS WORKS

The test assumes the service fixture returns order 7 and request authorization is satisfied. Add malformed input, not-found, and forbidden cases rather than asserting only success. MockMvc does not start an external HTTP server.

04 / WATCH OUT

Mocking the controller itself would bypass the endpoint behavior.

05 / INTERVIEWER FOLLOW-UP

When should a real-port test be added?

Answer: Use a real-port test to verify actual server, servlet container, proxy headers, sockets, TLS, or integration behavior that mocked servlet tests do not exercise.

Source guide p. 63 • Intermediate • References: T01, T02, T03

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

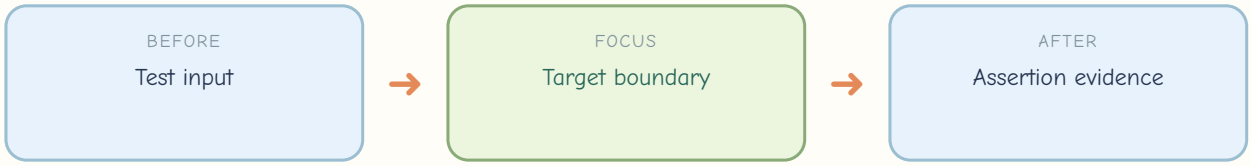
QUESTION 088 / 180

What does @DataJpaTest cover and what can it miss?

01 / THE SHORT INTERVIEW ANSWER

It focuses on JPA infrastructure and repositories, often with transactional rollback; database substitution can hide production differences.

DRAW THE IDEA — @DataJpaTest cover miss



02 / THE ORIGINAL EXAMPLE

```
@DataJpaTest
class CustomerRepositoryTest {
  @Autowired CustomerRepository repository;
  // Persist fixtures, execute query, assert rows and ordering.
}
```

03 / WHY THIS WORKS

A slice verifies mappings and query semantics in its selected database. Test dialect-specific SQL, constraints, locking, and migrations against the actual database engine when important. A transactionally rolled-back test may not exercise commit behavior.

04 / WATCH OUT

An in-memory database is not proof of MySQL or PostgreSQL behavior.

05 / INTERVIEWER FOLLOW-UP

Why explicitly flush before asserting a constraint failure?

Answer: A database constraint may be enforced only when SQL is flushed. Explicit flush makes that failure occur inside the test, instead of after assertions.

Source guide p. 64 • Intermediate • References: T01, T02, T03

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

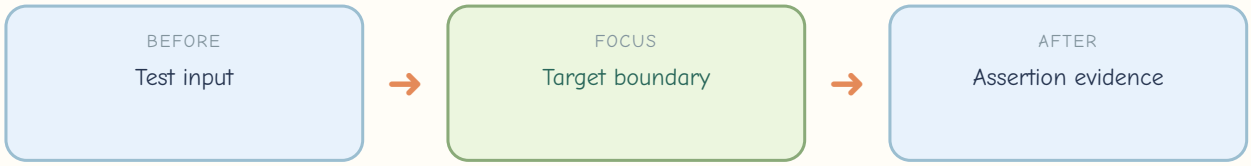
QUESTION 089 / 180

How does Testcontainers improve persistence integration tests?

01 / THE SHORT INTERVIEW ANSWER

It runs disposable instances of the target service, exposing real protocol and engine behavior instead of substituting a fake dialect.

DRAW THE IDEA — Testcontainers improve persistence inte



02 / THE ORIGINAL EXAMPLE

```
@Testcontainers
@SpringBootTest
class DatabaseIT {
  @Container @ServiceConnection
  static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");
}
```

03 / WHY THIS WORKS

This requires the corresponding test dependencies and an available container runtime. @ServiceConnection supplies supported connection details. Choose a database version aligned with deployment and verify migrations.

04 / WATCH OUT

Tests still need stable data isolation and resource cleanup.

05 / INTERVIEWER FOLLOW-UP

How would you test a schema change against an older application?

Answer: Run old-version application tests against the new migrated schema and new-version tests against the old compatible stage, ideally with Testcontainers and actual migration scripts.

Source guide p. 64 • Intermediate • References: T01, T02, T03

CHAPTER 15 • TESTING FROM UNITS TO REAL BOUNDARIES

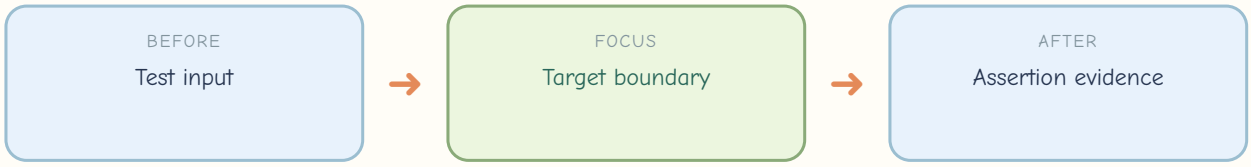
QUESTION 090 / 180

How should asynchronous and security tests be made meaningful?

01 / THE SHORT INTERVIEW ANSWER

Wait for observable completion with a bound and exercise the actual authorization path, including denied outcomes.

DRAW THE IDEA — asynchronous security tests made



02 / THE ORIGINAL EXAMPLE

```
// Avoid an arbitrary Thread.sleep.
Future<Result> result = service.processAsync();
assertEquals("done", result.get(2, TimeUnit.SECONDS).status());
// Security cases: absent token, invalid token, wrong scope, valid scope.
```

03 / WHY THIS WORKS

A deadline prevents hanging while completion proves the event occurred. Fake authentication can test authorization policy but does not verify real signature decoding. Add token-validation integration cases where needed.

04 / WATCH OUT

A passing happy path does not prove tenant isolation.

05 / INTERVIEWER FOLLOW-UP

Which tests should use a real JwtDecoder?

Answer: At least some security integration tests should validate actual signed tokens and decoder configuration, instead of mocking every Authentication or JwtDecoder.

Source guide p. 65 • Intermediate • References: T01, T02, T03

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

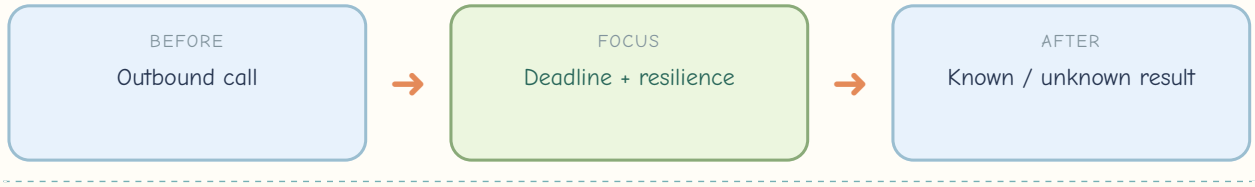
QUESTION 091 / 180

How do RestClient, WebClient and RestTemplate differ?

01 / THE SHORT INTERVIEW ANSWER

RestClient is a synchronous fluent client; WebClient provides reactive composition; RestTemplate is a legacy synchronous template style.

DRAW THE IDEA — RestClient WebClient RestTemplate



02 / THE ORIGINAL EXAMPLE

```
String body = RestClient.create().get()
    .uri("https://service.example/status")
    .retrieve().body(String.class);
```

03 / WHY THIS WORKS

This is a synchronous network call, not a production timeout configuration. Choose the underlying request factory or connector with explicit deadlines and pooling. API return style alone does not prove transport behavior.

04 / WATCH OUT

WebClient.block still blocks the caller.

05 / INTERVIEWER FOLLOW-UP

Which client fits a reactive chain?

Answer: WebClient fits a reactive chain where the caller can keep work non-blocking end to end. Blocking with .block() inside an event loop defeats that benefit.

Source guide p. 66 • Intermediate • References: I01, R02

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

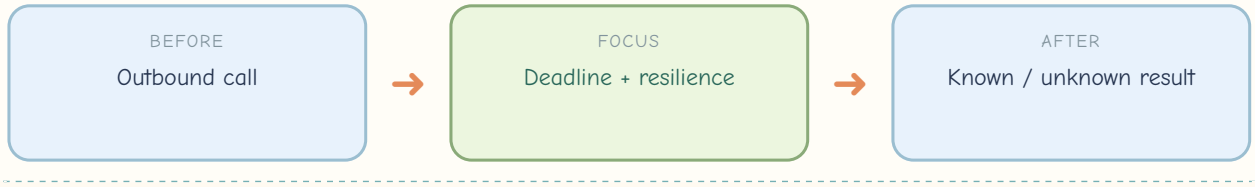
QUESTION 092 / 180

Which timeouts should a service-to-service call consider?

01 / THE SHORT INTERVIEW ANSWER

Consider connection acquisition, connect, TLS, read/response, and overall operation deadlines according to the selected client.

DRAW THE IDEA — timeouts service-to-service call consid



02 / THE ORIGINAL EXAMPLE

```
Total request budget: 2 seconds
Pool/connect budget: bounded subset
Remote response + retries: must fit remaining budget
Cancellation: close or stop owned work where supported
```

03 / WHY THIS WORKS

A single socket read timeout may not bound every stage or retry. Carry the caller's deadline across the chain and measure pool waiting separately. Connector option names and semantics are implementation-specific.

04 / WATCH OUT

A retry budget cannot exceed the endpoint's useful lifetime.

05 / INTERVIEWER FOLLOW-UP

How would DNS delay be observed?

Answer: DNS resolution may happen before connection establishment and have its own timeout/caching behavior. Measure DNS timing or instrument the resolver rather than assuming connectTimeout covers it.

Source guide p. 66 • Intermediate • References: I01, R02

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

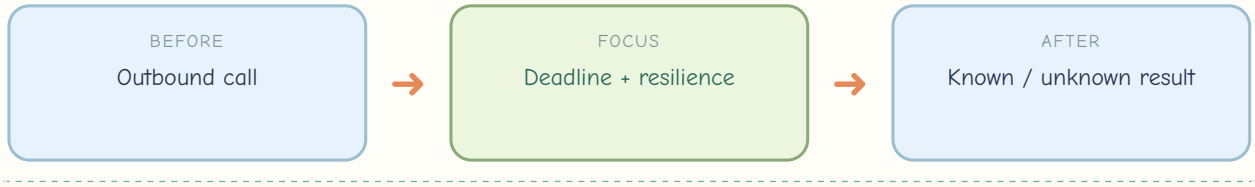
QUESTION 093 / 180

When is retry safe and when is it harmful?

01 / THE SHORT INTERVIEW ANSWER

> **Retry transient failures within a deadline only when repeated effects are safe or deduplicated; avoid amplifying overload.**

DRAW THE IDEA — retry safe harmful



02 / THE ORIGINAL EXAMPLE

```
GET catalog -> limited transient retry may fit
POST charge -> stable idempotency identity required
Retry 401 or validation error -> usually repeats a permanent failure
```

03 / WHY THIS WORKS

Classify the failure, use bounded attempts with backoff and jitter, and honor the downstream contract. A timeout is an unknown outcome for side-effecting calls. Cascaded retries can multiply traffic across services.

04 / WATCH OUT

> Retrying every exception is not resilience.

05 / INTERVIEWER FOLLOW-UP

Where should the one owning retry layer live?

Answer: Choose one retry-owning boundary, usually the caller or designated transport policy, with a request-wide attempt budget. Disable repeated retries elsewhere to avoid multiplication.

Source guide p. 67 • Intermediate • References: I01, R02

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

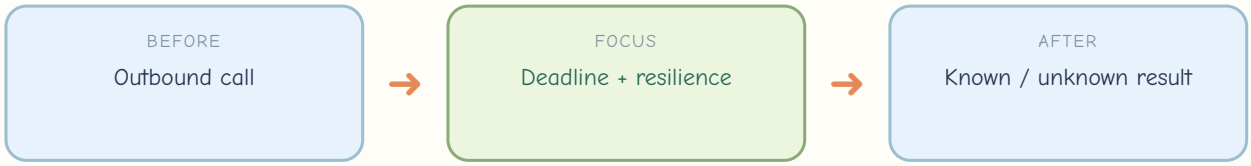
QUESTION 094 / 180

How do circuit breakers, bulkheads and rate limiters differ?

01 / THE SHORT INTERVIEW ANSWER

A circuit breaker stops repeated calls after failure patterns; a bulkhead limits concurrent resource use; a rate limiter controls admission over time.

DRAW THE IDEA — circuit breakers bulkheads rate



02 / THE ORIGINAL EXAMPLE

```
Breaker OPEN -> fast rejection according to policy
Bulkhead full -> reject or wait within a bound
Rate budget exceeded -> controlled throttle response
```

03 / WHY THIS WORKS

These solve different problems and can be combined with timeouts. None makes a slow call finish sooner by itself. Choose a meaningful fallback or explicit failure rather than returning misleading data.

04 / WATCH OUT

A circuit breaker is not a timeout.

05 / INTERVIEWER FOLLOW-UP

How do half-open probes prevent a recovery surge?

Answer: Limit concurrent half-open requests to a small probe budget, then reopen on failures or close only after sufficient successful probes. Otherwise recovery can be flooded.

Source guide p. 67 • Intermediate • References: I01, R02

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

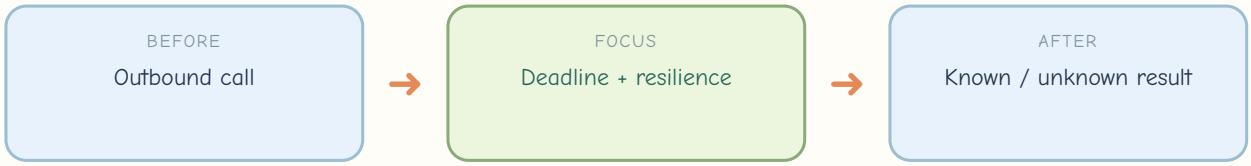
QUESTION 095 / 180

What does an HTTP client connection pool need beyond a maximum size?

01 / THE SHORT INTERVIEW ANSWER

It needs bounded pending acquisition, idle/lifetime policy, deadlines, cleanup, and visibility into saturation.

DRAW THE IDEA — HTTP client connection pool



02 / THE ORIGINAL EXAMPLE

```
Active connections: bounded
Pending acquisition queue: bounded
Acquire wait: bounded
Idle or expired connections: retired by policy
```

03 / WHY THIS WORKS

An unbounded wait queue can shift the outage from sockets to memory and latency. Observe pool wait and downstream duration, and ensure body consumption or closing releases resources.

04 / WATCH OUT

Unlimited client concurrency does not imply unlimited useful throughput.

05 / INTERVIEWER FOLLOW-UP

What can leak a connection after an error response?

Answer: Failing to consume or close a response body can keep a leased connection unavailable. Verify response closing on success, error, timeout and cancellation paths.

Source guide p. 68 • Intermediate • References: I01, R02

CHAPTER 16 • HTTP CLIENTS, DEADLINES AND RESILIENCE

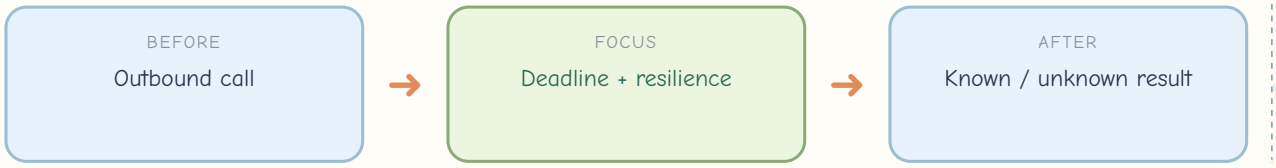
QUESTION 096 / 180

How should external client failures become API responses?

01 / THE SHORT INTERVIEW ANSWER

Translate them at a deliberate boundary while preserving cause and distinguishing unavailable, timed-out, absent, and invalid outcomes.

DRAW THE IDEA — external client failures become



02 / THE ORIGINAL EXAMPLE

```
Remote 404 -> domain absence if contract says so
Remote timeout -> unavailable/unknown outcome policy
Remote 500 -> dependency failure, not empty success
```

03 / WHY THIS WORKS

One external status can mean different things in different integrations. Keep internal transport details out of public error bodies while retaining diagnostic context in safe logs and traces.

04 / WATCH OUT

A fallback must not invent successful payment or authorization.

05 / INTERVIEWER FOLLOW-UP

How would you expose degraded cached data honestly?

Answer: Return explicit degraded status or staleness metadata when cached information is used; never treat stale stock or payment state as authoritative success.

Source guide p. 68 • Intermediate • References: I01, R02

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

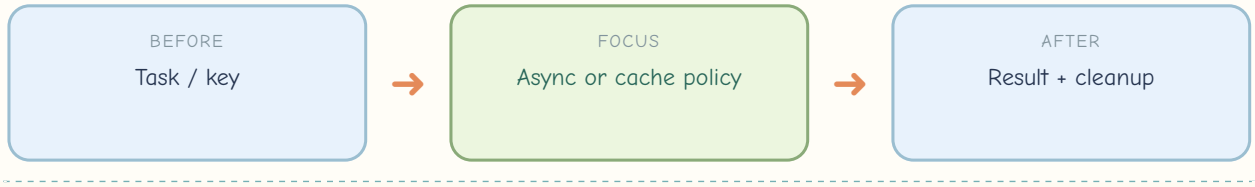
QUESTION 097 / 180

What does @Async do and what does it require?

01 / THE SHORT INTERVIEW ANSWER

> It submits advised method execution to an executor when async support is enabled and the call crosses the configured proxy boundary.

DRAW THE IDEA — @Async require



02 / THE ORIGINAL EXAMPLE

```
@EnableAsync
@Configuration
class AsyncConfig {}
@Async("notifications")
public CompletableFuture<Void> send(long orderId) {
  performSend(orderId); return CompletableFuture.completedFuture(null);
}
```

03 / WHY THIS WORKS

Define the named executor with appropriate bounds and shutdown policy. Self-calls bypass ordinary proxy advice. A CompletableFuture return exposes failure better than ignoring a void task's outcome.

04 / WATCH OUT

> Async is not durable job delivery.

05 / INTERVIEWER FOLLOW-UP

Who observes a failed notification task?

Answer: The task owner must observe Future/CompletableFuture outcomes, return status, or route errors to monitoring. A void @Async exception handler logs failures but does not provide durable retry.

Source guide p. 69 • Intermediate • References: I02, I03

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

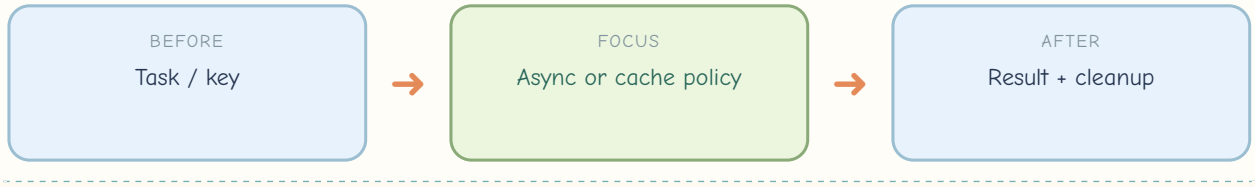
QUESTION 098 / 180

How should an async executor be bounded?

01 / THE SHORT INTERVIEW ANSWER

Use a deliberate worker count, finite queue, rejection behavior, and lifecycle ownership based on the workload.

DRAW THE IDEA — async executor bounded



02 / THE ORIGINAL EXAMPLE

```
ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
e.setCorePoolSize(4); e.setMaxPoolSize(8);
e.setQueueCapacity(100);
// Configure rejection, thread names, shutdown; register as a bean.
```

03 / WHY THIS WORKS

These numbers are illustrative. A full queue triggers the chosen rejection behavior, not magical elasticity. Managed initialization and destruction must occur. Avoid silently dropping important tasks.

04 / WATCH OUT

Extra workers do not solve a constrained database.

05 / INTERVIEWER FOLLOW-UP

How would an event-loop caller react to CallerRunsPolicy?

Answer: CallerRunsPolicy may execute the blocking task on the submitting event-loop thread, stalling unrelated requests. Use explicit rejection or asynchronous bounded admission instead.

Source guide p. 70 • Intermediate • References: I02, I03

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

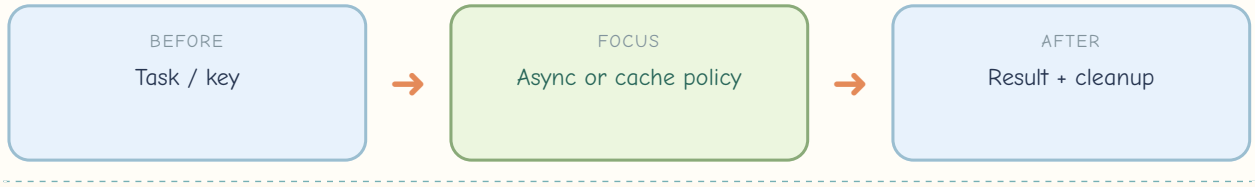
QUESTION 099 / 180

How do fixedDelay, fixedRate and cron scheduling differ?

01 / THE SHORT INTERVIEW ANSWER

fixedDelay waits after completion; fixedRate follows a time-based cadence; cron matches a calendar schedule.

DRAW THE IDEA — fixedDelay fixedRate cron scheduling



02 / THE ORIGINAL EXAMPLE

```
@Scheduled(fixedDelay = 5000)
void refresh() { loadSnapshot(); }
@Scheduled(cron = "0 0 9 * * *", zone = "Asia/Kolkata")
void daily() { runDailyReport(); }
```

03 / WHY THIS WORKS

With scheduling enabled, refresh waits five seconds after a run completes before the next start. Cron uses Spring's field format and the explicit zone. Scheduler capacity and overlaps need deliberate configuration.

04 / WATCH OUT

Every application replica may run the same schedule.

05 / INTERVIEWER FOLLOW-UP

How would you ensure only one durable job instance executes?

Answer: Use a shared durable job authority, such as a database claim with unique key or scheduler clustering, so pods cannot run the same logical job without coordination.

Source guide p. 70 • Intermediate • References: I02, I03

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

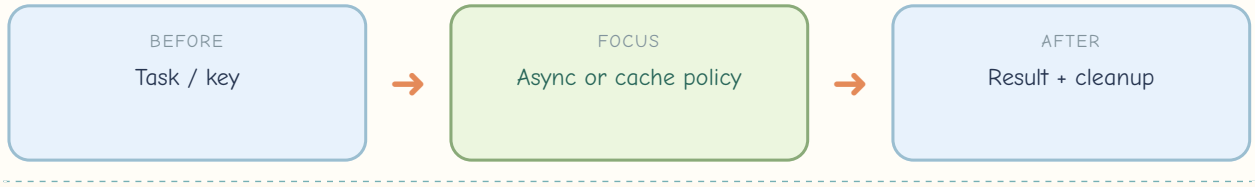
QUESTION 100 / 180

How do @Cacheable, @CachePut and @CacheEvict differ?

01 / THE SHORT INTERVIEW ANSWER

@Cacheable can skip execution on a hit; @CachePut executes and updates the cache; @CacheEvict removes entries.

DRAW THE IDEA — @Cacheable @CachePut @CacheEvict



02 / THE ORIGINAL EXAMPLE

```
@Cacheable(cacheNames="products", key="#id")
public ProductDto load(long id) { return repository.loadDto(id); }
@CacheEvict(cacheNames="products", key="#id")
public void update(long id, UpdateProduct request) { /* persist */ }
```

03 / WHY THIS WORKS

Caching support and a suitable manager must be enabled. Use stable keys and immutable results. Eviction timing, transactions, and failures determine whether readers can see stale data.

04 / WATCH OUT

Annotation-based caching also has proxy self-call limits.

05 / INTERVIEWER FOLLOW-UP

What if an update commits but eviction fails?

Answer: A committed update followed by failed eviction leaves stale cache data. Use versioned keys, durable invalidation events, bounded TTL or an appropriate write-through strategy and repair.

Source guide p. 71 • Intermediate • References: I02, I03

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

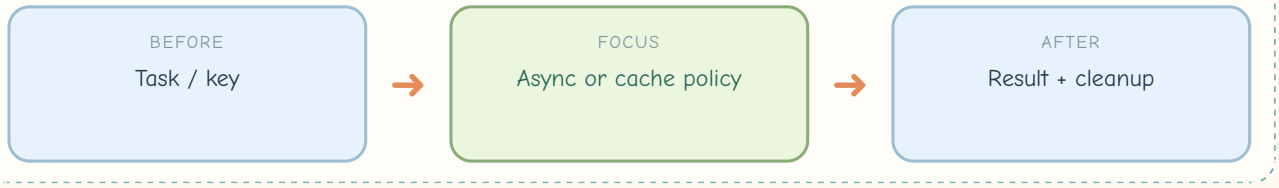
QUESTION 101 / 180

Does @Cacheable(sync=true) solve every cache stampede?

01 / THE SHORT INTERVIEW ANSWER

It requests synchronized loading supported by the cache provider, but does not universally coordinate all replicas or all failure paths.

DRAW THE IDEA — @Cacheable sync true solve



02 / THE ORIGINAL EXAMPLE

```
@Cacheable(cacheNames="catalog", key="#id", sync=true)
public CatalogDto load(long id) { return remote.load(id); }
```

03 / WHY THIS WORKS

Check provider semantics and annotation restrictions. Distributed stampede handling can require leases, shared in-flight coordination, expiration jitter, or stale-while-refresh policy. Define failed-load cleanup.

04 / WATCH OUT

Local synchronization is not automatically a distributed lock.

05 / INTERVIEWER FOLLOW-UP

How would synchronized loading behave across ten pods?

Answer: The sync behavior is normally local to a cache entry/provider, not a ten-pod coordination guarantee. Each pod may load the same item unless using suitable distributed coordination.

Source guide p. 71 • Intermediate • References: I02, I03

CHAPTER 17 • ASYNC EXECUTION, SCHEDULING AND CACHING

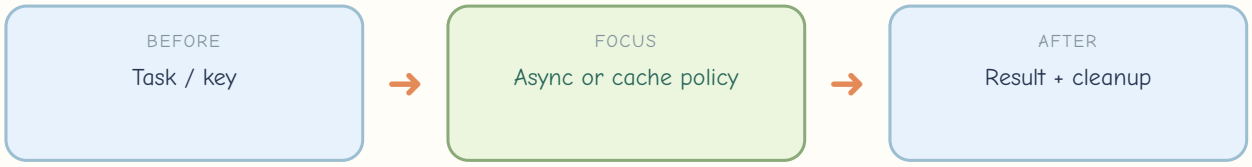
QUESTION 102 / 180

How should a cache key include tenant and authorization context?

01 / THE SHORT INTERVIEW ANSWER

Include all input dimensions affecting the result, and enforce permission independently rather than treating a hit as authorization.

DRAW THE IDEA — cache key include tenant



02 / THE ORIGINAL EXAMPLE

```
CacheKey(tenantId, productId, locale)
// If visibility changes by user role, address that dimension
// or cache only an authorization-independent representation.
```

03 / WHY THIS WORKS

A productId-only key can serve another tenant's data. Cache identity, freshness, and permission are separate contracts. Prefer low-cardinality stable dimensions and bounded retention.

04 / WATCH OUT

A cached DTO can leak data despite valid authentication.

05 / INTERVIEWER FOLLOW-UP

How would permission revocation invalidate cached views?

Answer: A permission change must invalidate or version relevant cached views, or force a fresh authorization check. Never trust a previously cached result as permanent authorization.

Source guide p. 72 • Intermediate • References: I02, I03

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 103 / 180

How does a @KafkaListener process records?

01 / THE SHORT INTERVIEW ANSWER

> **Spring creates listener containers that poll Kafka and invoke the configured handler with conversion and error behavior.**

DRAW THE IDEA — @KafkaListener process records



02 / THE ORIGINAL EXAMPLE

```
@KafkaListener(topics="orders", groupId="billing")
void onOrder(OrderCreated event) { billing.handle(event); }
```

03 / WHY THIS WORKS

The example assumes matching serialization and container configuration. Partition count limits concurrent ownership within a group. Define acknowledgment, retries, recovery, and idempotency rather than assuming the annotation supplies them all.

04 / WATCH OUT

> A listener exception needs an explicit recovery policy.

05 / INTERVIEWER FOLLOW-UP

What happens during a rebalance?

Answer: A rebalance revokes/assigns partitions and may replay uncommitted records. Make effects idempotent and handle inflight work and offset commits carefully.

Source guide p. 73 • Intermediate • References: I04, I05

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 104 / 180

How do producer acknowledgments differ from consumer offsets?

01 / THE SHORT INTERVIEW ANSWER

Producer acknowledgments concern broker acceptance; consumer offsets track progress through a partition.

DRAW THE IDEA — producer acknowledgments consumer offse



02 / THE ORIGINAL EXAMPLE

```
Producer acks=all -> broker replication acknowledgment policy
Consumer commits offset -> recorded consumption position
Business transaction committed -> application side effect
```

03 / WHY THIS WORKS

These are separate milestones. Broker acknowledgment does not prove a billing database update occurred. Committing an offset before durable processing can lose the effect; committing afterward can lead to redelivery.

04 / WATCH OUT

Offset numbers are partition-specific, not global record ids.

05 / INTERVIEWER FOLLOW-UP

How do min.insync.replicas and acks interact?

Answer: acks=all waits for acknowledgments from the in-sync replica set under broker configuration; min.insync.replicas enforces a minimum ISR size for such writes or rejects them.

Source guide p. 74 • Intermediate • References: I04, I05

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 105 / 180

Why does a consumer need idempotency under at-least-once delivery?

01 / THE SHORT INTERVIEW ANSWER

> **A record can be processed again after failure or rebalance, so repeated business effects must be prevented when required.**

DRAW THE IDEA — consumer idempotency under at-least-onc



02 / THE ORIGINAL EXAMPLE

```
@Transactional
public void handle(Event e) {
  if (processed.exists(e.id())) return;
  applyEffect(e);
  processed.insert(e.id()); // Unique constraint required
}
```

03 / WHY THIS WORKS

This is a conceptual handler: the duplicate marker and business effect must share the right atomic boundary. A check alone races; the unique marker and conflict handling matter. External effects need their own idempotency protocol.

04 / WATCH OUT

> In-memory dedup does not survive restart.

05 / INTERVIEWER FOLLOW-UP

How would you scope an event id by tenant or producer?

Answer: Use a stable event identity combined with trusted source/tenant scope and a unique processed-event key in the same transaction as the effect. Cross-tenant collisions must not suppress valid events.

Source guide p. 74 • Intermediate • References: I04, I05

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 106 / 180

What is the transactional outbox pattern?

01 / THE SHORT INTERVIEW ANSWER

Write the business update and event intent in one database transaction, then publish the durable intent asynchronously.

DRAW THE IDEA — transactional outbox pattern



02 / THE ORIGINAL EXAMPLE

```
@Transactional
public void place(CreateOrder command) {
  orders.insert(command);
  outbox.insert(new OrderCreated(command.operationId()));
}
```

03 / WHY THIS WORKS

A publisher later sends pending outbox rows and tracks progress. Publishing can repeat if acknowledgment and marking are interrupted, so consumers still need idempotency. Monitor backlog age and define retention.

04 / WATCH OUT

Outbox is not automatic end-to-end exactly-once effect.

05 / INTERVIEWER FOLLOW-UP

How would you prevent losing an event after commit?

Answer: Persist an outbox row in the same transaction as the business update. A relay retries publication after restart, so intent is not lost when the broker is unavailable.

Source guide p. 75 • Intermediate • References: I04, I05

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 107 / 180

How should poison records and retry topics be handled?

01 / THE SHORT INTERVIEW ANSWER

Classify transient versus permanent failures, bound attempts, and route exhausted records with diagnostic context to a controlled recovery path.

DRAW THE IDEA — poison records retry topics



02 / THE ORIGINAL EXAMPLE

```
Temporary dependency outage -> bounded delayed retry
Invalid schema/business poison -> quarantine or DLT policy
Recovered message -> replay with idempotency protection
```

03 / WHY THIS WORKS

Choose the correct container/error-handler mechanism for the project's Spring Kafka version. Non-blocking retries can change ordering, and partition pausing affects throughput. Operators need ownership and replay tooling.

04 / WATCH OUT

A dead-letter topic without monitoring is hidden data loss.

05 / INTERVIEWER FOLLOW-UP

Which errors must never be retried indefinitely?

Answer: Permanent schema failures, bad authentication, unknown tenant or impossible business input need quarantine/repair rather than infinite retries. Transient dependency errors can be retried with a budget.

Source guide p. 75 • Intermediate • References: I04, I05

CHAPTER 18 • MESSAGING, KAFKA AND RELIABLE EVENTS

QUESTION 108 / 180

What does Kafka exactly-once not automatically guarantee?

01 / THE SHORT INTERVIEW ANSWER

Kafka transaction guarantees apply to supported Kafka operations; they do not automatically include arbitrary HTTP or database side effects.

DRAW THE IDEA — Kafka exactly-once not automatically



02 / THE ORIGINAL EXAMPLE

```
Read Kafka -> write Kafka in supported transaction
Read Kafka -> charge external provider -> different atomic boundary
```

03 / WHY THIS WORKS

State the exact scope of the guarantee. Coordinating a database with offsets can still leave a failure window unless a supported protocol and recovery policy close it. Design idempotency for the external effect.

04 / WATCH OUT

'Exactly once' must name the observed business operation.

05 / INTERVIEWER FOLLOW-UP

When would outbox plus dedup be easier to reason about?

Answer: When database updates and Kafka transactions are separate, outbox plus consumer dedup often gives an easier end-to-end proof of business correctness.

Source guide p. 76 • Intermediate • References: I04, I05

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

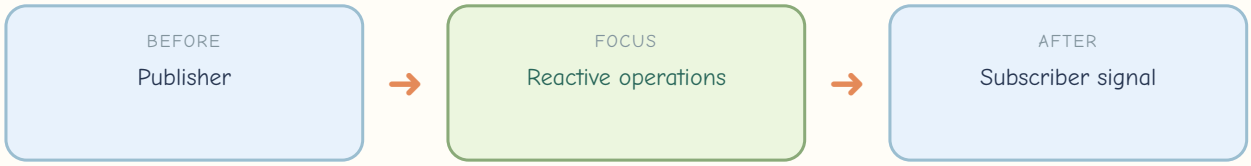
QUESTION 109 / 180

How do Spring MVC and WebFlux differ?

01 / THE SHORT INTERVIEW ANSWER

> **MVC uses the servlet request model; WebFlux provides reactive request handling and composition through publishers.**

DRAW THE IDEA — MVC WebFlux



02 / THE ORIGINAL EXAMPLE

```
@GetMapping("/products/{id}")
Mono<ProductDto> get(@PathVariable long id) {
  return reactiveRepository.findById(id).map(this::toDto);
}
```

03 / WHY THIS WORKS

The method returns a publisher describing the result. Benefits require suitable downstream and execution behavior; returning Mono does not make a JDBC query non-blocking. MVC can also support some async return types.

04 / WATCH OUT

Framework choice alone does not prove higher throughput.

05 / INTERVIEWER FOLLOW-UP

When is MVC simpler for a blocking stack?

Answer: MVC is often simpler when the service already uses blocking JPA/JDBC and servlet infrastructure. Adding reactive interfaces will not make those calls non-blocking.

Source guide p. 77 • Intermediate • References: R01, R02, B02

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

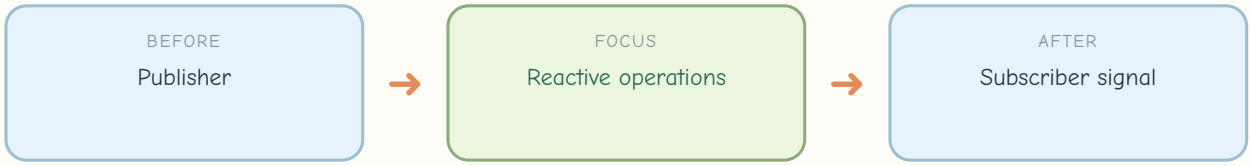
QUESTION 110 / 180

What is the difference between Mono and Flux?

01 / THE SHORT INTERVIEW ANSWER

Mono represents zero or one item; Flux represents zero to many items, both with completion or error signals.

DRAW THE IDEA — difference Mono Flux



02 / THE ORIGINAL EXAMPLE

```
Mono<String> one = Mono.just("A");
Flux<String> many = Flux.just("A", "B");
Mono<String> none = Mono.empty();
```

03 / WHY THIS WORKS

An empty publisher completes without a value; it does not emit null. A sequence can also fail. The API response must distinguish an absent domain result from an unavailable dependency.

04 / WATCH OUT

Reactor does not permit ordinary null elements.

05 / INTERVIEWER FOLLOW-UP

How would switchIfEmpty implement not-found behavior?

Answer: Use switchIfEmpty(Mono.error(new NotFoundException(...))) or build an explicit 404 response. An empty Mono is completion without a value, not automatically a not-found status.

Source guide p. 77 • Intermediate • References: R01, R02, B02

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

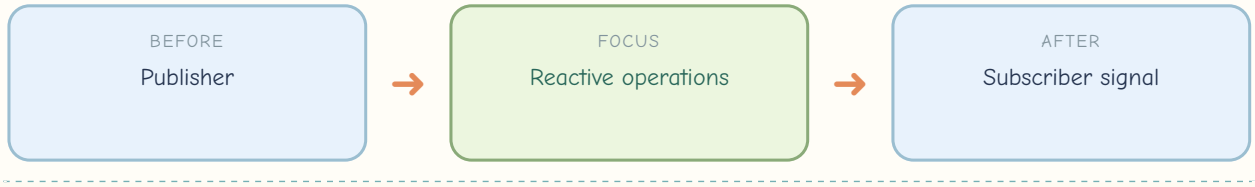
QUESTION 111 / 180

How do map and flatMap differ in Reactor?

01 / THE SHORT INTERVIEW ANSWER

map transforms a value synchronously; flatMap composes an asynchronous publisher and flattens its signals.

DRAW THE IDEA — map flatMap Reactor



02 / THE ORIGINAL EXAMPLE

```
Mono<OrderDto> result = findCustomer(id)
    .flatMap(c -> findOrder(c.orderId()))
    .map(this::toDto);
```

03 / WHY THIS WORKS

findOrder returns another Mono, so flatMap combines the asynchronous step. Flux.flatMap can interleave concurrent inner results; concatMap can preserve sequential processing when needed.

04 / WATCH OUT

Operator choice affects ordering and concurrency.

05 / INTERVIEWER FOLLOW-UP

When would flatMapSequential fit?

Answer: flatMapSequential can run inner publishers concurrently while emitting results in source order; use it when ordering matters but some overlap is useful.

Source guide p. 78 • Intermediate • References: R01, R02, B02

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

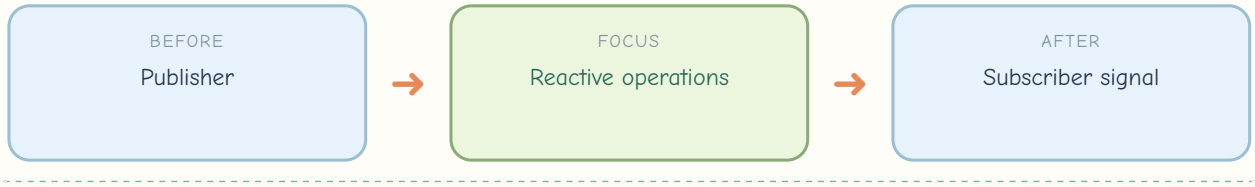
QUESTION 112 / 180

Why is blocking the event loop dangerous?

01 / THE SHORT INTERVIEW ANSWER

> **A blocked event-loop thread cannot promptly process other connections and signals assigned to it.**

DRAW THE IDEA — blocking event loop dangerous



02 / THE ORIGINAL EXAMPLE

```
Mono<Result> result = Mono.fromCallable(() -> blockingClient.call())
    .subscribeOn(Schedulers.boundedElastic());
```

03 / WHY THIS WORKS

This isolates a known blocking call from the event loop; it does not make the call non-blocking. Bound downstream concurrency and deadlines too. Avoid block or manual subscribe inside a request pipeline unless the ownership design explicitly calls for it.

04 / WATCH OUT

Offloading without admission control can move the bottleneck.

05 / INTERVIEWER FOLLOW-UP

How would you detect blocking in a load test?

Answer: Observe event-loop thread stacks, blocked-thread warnings and load-test p95/p99 latency. BlockHound can flag some blocking calls in supported test configurations.

Source guide p. 78 • Intermediate • References: R01, R02, B02

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

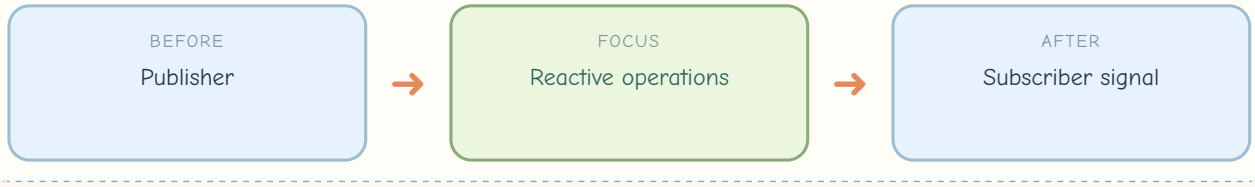
QUESTION 113 / 180

What are backpressure and cancellation?

01 / THE SHORT INTERVIEW ANSWER

Backpressure coordinates demand for signals; cancellation tells a publisher downstream no longer needs the sequence.

DRAW THE IDEA — backpressure cancellation



02 / THE ORIGINAL EXAMPLE

```
Flux<Record> records = source.take(10);
// After enough items, take can cancel upstream.
// The source and transport must support meaningful cancellation.
```

03 / WHY THIS WORKS

Demand handling does not automatically limit every queue or external database query. Some hot sources cannot slow their producer and need buffer, drop, or error policies. Cancellation also does not undo already-completed side effects.

04 / WATCH OUT

An unbounded collectList can still exhaust memory.

05 / INTERVIEWER FOLLOW-UP

How would you bound a streaming export?

Answer: Stream in bounded chunks with backpressure and cancellation checks, limit simultaneous exports, and release database/file resources on cancellation and timeout.

Source guide p. 79 • Intermediate • References: R01, R02, B02

CHAPTER 19 • WEBFLUX, REACTOR AND VIRTUAL THREADS

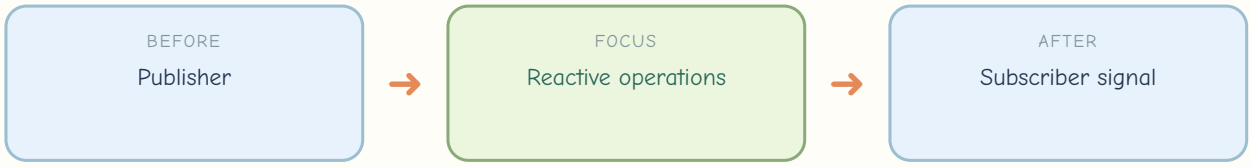
QUESTION 114 / 180

When might virtual threads be preferable to a reactive rewrite?

01 / THE SHORT INTERVIEW ANSWER

They can support large numbers of blocking tasks with straightforward code when the framework and dependencies support the chosen JDK configuration.

DRAW THE IDEA — might virtual threads preferable



02 / THE ORIGINAL EXAMPLE

```
spring.threads.virtual.enabled=true
# Boot baseline requires Java 21+ for virtual-thread support.
# Still bound database, remote-call, and admission capacity.
```

03 / WHY THIS WORKS

Check which Boot-managed execution components are affected rather than assuming every library thread is virtual. CPU-bound work does not gain processors. Java-version-specific pinning and carrier behavior need separate verification.

04 / WATCH OUT

A property flag is not an end-to-end capacity plan.

05 / INTERVIEWER FOLLOW-UP

Why can the database stay saturated after enabling it?

Answer: Virtual threads reduce platform-thread cost for waiting but do not add database connections. The same pool, SQL and database limits still apply; bound requests to the dependency.

Source guide p. 79 • Intermediate • References: R01, R02, B02

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

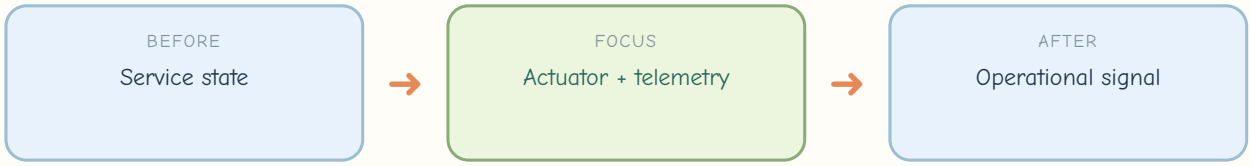
QUESTION 115 / 180

What does Spring Boot Actuator provide?

01 / THE SHORT INTERVIEW ANSWER

It integrates operational endpoints and infrastructure for health, metrics, diagnostics, and management.

DRAW THE IDEA — Actuator provide



02 / THE ORIGINAL EXAMPLE

```
<artifactId>spring-boot-starter-actuator</artifactId>
# Example routes after supported exposure:
# /actuator/health, /actuator/metrics
```

03 / WHY THIS WORKS

Adding the dependency supplies capability, but endpoint availability depends on enablement or access, exposure, security, and version. Select a minimal operational surface and restrict sensitive diagnostics.

04 / WATCH OUT

Presence in the context does not mean public HTTP access.

05 / INTERVIEWER FOLLOW-UP

Why might an enabled endpoint return 404?

Answer: An endpoint can be enabled yet not exposed on HTTP, blocked by security, moved to another management port/base path, or unavailable due to a missing dependency.

Source guide p. 80 • Senior • References: 001, 002

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

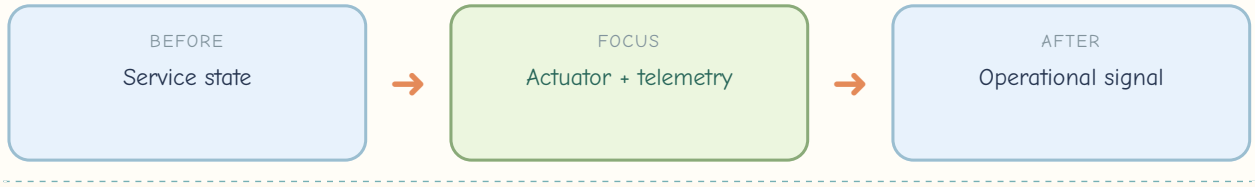
QUESTION 116 / 180

How do liveness and readiness differ?

01 / THE SHORT INTERVIEW ANSWER

Liveness asks whether the process should restart; readiness asks whether it should receive traffic.

DRAW THE IDEA — liveness readiness



02 / THE ORIGINAL EXAMPLE

```
/actuator/health/liveness  -> restart decision
/actuator/health/readiness -> routing decision
# Configure probe exposure and groups for the deployment.
```

03 / WHY THIS WORKS

A shared database outage should not normally make every instance fail liveness and restart together. Readiness can consider the service's ability to serve useful requests, with a deliberate dependency policy.

04 / WATCH OUT

A healthy management port alone can hide a broken main listener.

05 / INTERVIEWER FOLLOW-UP

Why expose probes on the application port too?

Answer: An independent management port may report healthy while the application port is broken. Probing the main port can verify the actual listener used by traffic.

Source guide p. 81 • Senior • References: 001, 002

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

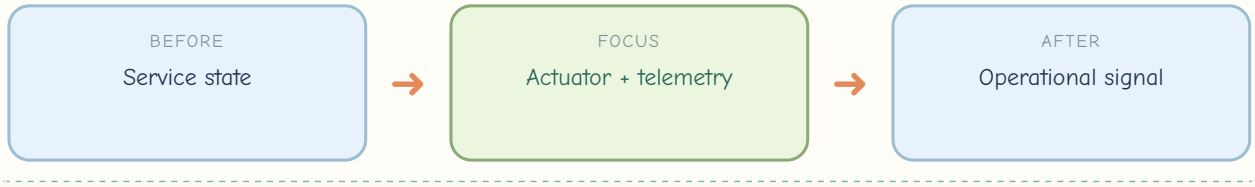
QUESTION 117 / 180

How should management endpoints be secured?

01 / THE SHORT INTERVIEW ANSWER

Expose only needed endpoints and enforce explicit network and authentication/ authorization policies.

DRAW THE IDEA — management endpoints secured



02 / THE ORIGINAL EXAMPLE

```
management.endpoints.web.exposure.include=health,info,prometheus
management.endpoint.health.show-details=when_authorized
# Prometheus capability also needs its registry dependency.
```

03 / WHY THIS WORKS

HTTP exposure does not grant permission, and permission does not necessarily create exposure. When custom security chains exist, own the management rules explicitly. Heap dumps, environment data, and configuration diagnostics can be sensitive.

04 / WATCH OUT

include=* is rarely a good production starting point.

05 / INTERVIEWER FOLLOW-UP

Which monitoring identity can access metrics?

Answer: Use a dedicated least-privilege monitoring principal or service identity permitted only to required endpoints, usually metrics/health; restrict all sensitive diagnostic endpoints.

Source guide p. 81 • Senior • References: 001, 002

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

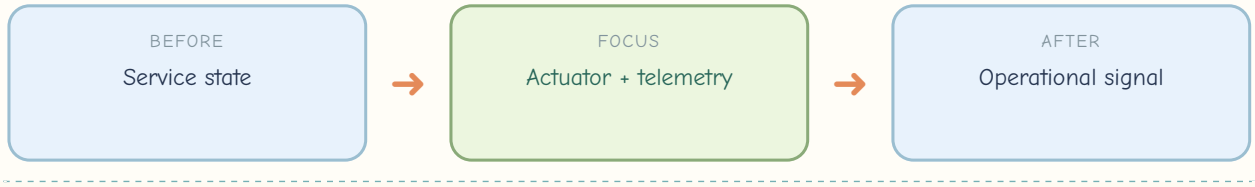
QUESTION 118 / 180

How do metrics, logs and traces differ?

01 / THE SHORT INTERVIEW ANSWER

> **Metrics summarize populations over time, logs record events, and traces connect spans across one operation.**

DRAW THE IDEA — metrics logs traces



02 / THE ORIGINAL EXAMPLE

```
Metric: p99 HTTP duration rose
Trace: database span consumed most of request time
Log: safe query failure context with matching trace id
```

03 / WHY THIS WORKS

Use metrics to locate the symptom and traces or logs to explain examples. None replaces the others. Keep propagation and sampling consistent across async and service boundaries.

04 / WATCH OUT

> A log line without correlation can be hard to join to a failing request.

05 / INTERVIEWER FOLLOW-UP

What can a low trace sampling rate miss?

Answer: Rare error traces can be unsampled at low rates. Combine traces with error counters, logs, exemplars and focused higher-sampling rules when investigating incidents.

Source guide p. 82 • Senior • References: 001, 002

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

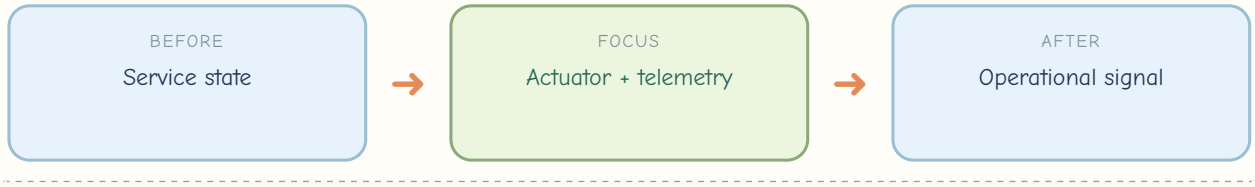
QUESTION 119 / 180

Why is metric-tag cardinality dangerous?

01 / THE SHORT INTERVIEW ANSWER

> **Every distinct tag combination can create another time series, increasing memory, storage, and query cost.**

DRAW THE IDEA — metric-tag cardinality dangerous



02 / THE ORIGINAL EXAMPLE

```
Good label: route=/orders/{id}, outcome=success
Bad label:  raw path=/orders/739421 or customerId=739421
```

03 / WHY THIS WORKS

Use normalized routes and bounded categories. Put per-request ids in appropriate logs or trace fields rather than meter tags. Even individually small label sets multiply when combined.

04 / WATCH OUT

> Adding a unique request id to a metric can explode series count.

05 / INTERVIEWER FOLLOW-UP

How would you detect runaway cardinality?

Answer: Inspect meter registry series counts and scrape sizes by metric/tag, alert on growth and forbid unbounded IDs like userId or requestId as metric labels.

Source guide p. 82 • Senior • References: 001, 002

CHAPTER 20 • ACTUATOR, OBSERVABILITY AND OPERATIONAL HEALTH

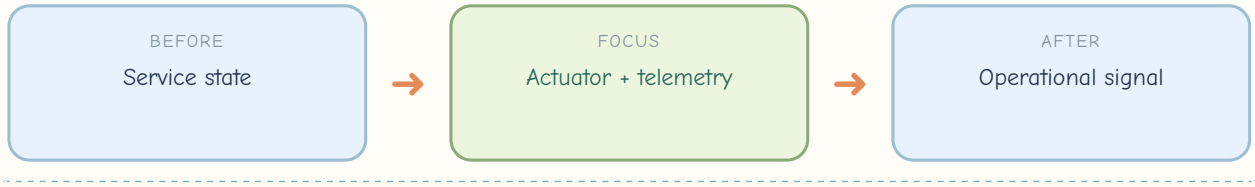
QUESTION 120 / 180

Which signals indicate database or executor saturation?

01 / THE SHORT INTERVIEW ANSWER

Observe active capacity, pending waits, wait duration, rejection, queue age, and useful completion rate together.

DRAW THE IDEA — signals indicate database executor



02 / THE ORIGINAL EXAMPLE

```
HTTP latency rises + Hikari pending waits rise
 -> inspect connection ownership and query latency
Queue age rises + CPU low
 -> inspect blocked workers and dependency capacity
```

03 / WHY THIS WORKS

A high request count alone cannot identify the bottleneck. Correlate resource waits with traces and thread stacks under the same time window. Alert on actionable capacity and service outcomes.

04 / WATCH OUT

Averages can hide tail latency.

05 / INTERVIEWER FOLLOW-UP

Which dashboard would separate slow SQL from pool acquisition?

Answer: Graph SQL execution time, Hikari active/pending/idle connections and acquisition wait beside request latency. Slow SQL and waiting for a connection appear differently.

Source guide p. 83 • Senior • References: 001, 002

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

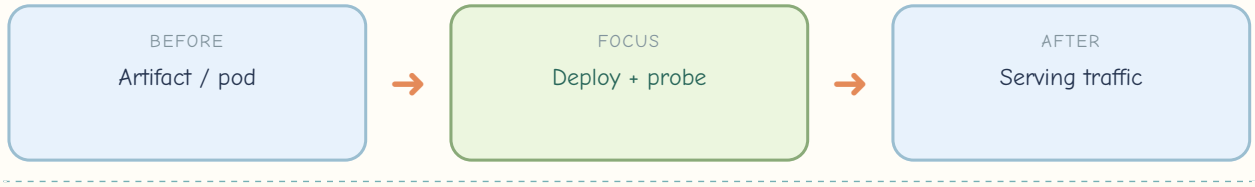
QUESTION 121 / 180

How does an executable Boot jar run?

01 / THE SHORT INTERVIEW ANSWER

> **The packaged jar includes a launcher layout and application dependencies so it can be started with java -jar.**

DRAW THE IDEA — executable jar run



02 / THE ORIGINAL EXAMPLE

```
mvn package
java -jar target/app.jar
# Use the Boot Maven/Gradle packaging integration.
```

03 / WHY THIS WORKS

The build plugin creates the executable layout; dependency management alone is not the packaging operation. Check the produced artifact rather than assuming every ordinary jar includes its dependencies and Boot launcher.

04 / WATCH OUT

A plain jar and an executable repackaged jar differ.

05 / INTERVIEWER FOLLOW-UP

When would a WAR deployment be required?

Answer: A WAR may be needed when an organization requires deployment to an external compatible servlet container. For most Boot services, executable jars simplify operations.

Source guide p. 84 • Senior • References: 001, 003, 004

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

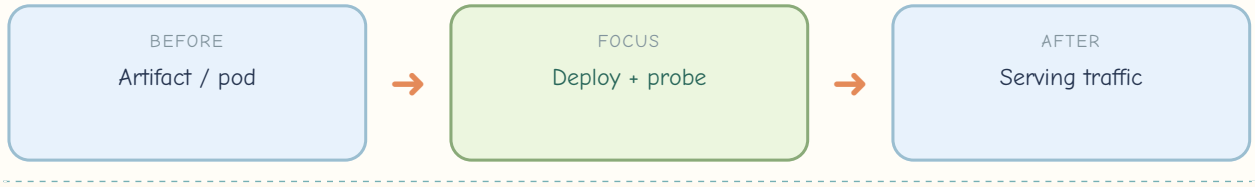
QUESTION 122 / 180

What should a production container include?

01 / THE SHORT INTERVIEW ANSWER

Include only the required runtime and artifact, use suitable permissions, and define environment and resource behavior deliberately.

DRAW THE IDEA — production container include



02 / THE ORIGINAL EXAMPLE

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY app.jar /app/app.jar
USER 10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

03 / WHY THIS WORKS

This is a minimal illustrative Dockerfile, not a complete hardened image policy. Pin and maintain an appropriate image version, ensure file permissions, and test native-memory headroom and signal handling.

04 / WATCH OUT

Container memory includes more than Java heap.

05 / INTERVIEWER FOLLOW-UP

How would you use image layers or buildpacks?

Answer: Use layered executable jars or buildpacks to improve caching and reproducible images. Run as non-root with a compatible JRE, health checks, limits and no embedded secrets.

Source guide p. 85 • Senior • References: 001, 003, 004

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

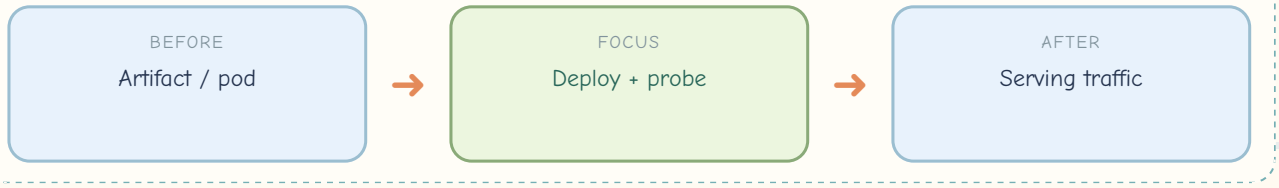
QUESTION 123 / 180

How should graceful shutdown work?

01 / THE SHORT INTERVIEW ANSWER

Stop new useful work, allow accepted requests to drain within a bound, then close managed resources with a recoverable policy for unfinished work.

DRAW THE IDEA — graceful shutdown work



02 / THE ORIGINAL EXAMPLE

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
# Platform termination grace must accommodate draining and cleanup.
```

03 / WHY THIS WORKS

Server behavior and defaults depend on version and implementation; the explicit setting documents intent. The platform should remove readiness and route traffic appropriately. Long jobs need durable recovery rather than relying solely on process life.

04 / WATCH OUT

SIGKILL prevents normal shutdown callbacks.

05 / INTERVIEWER FOLLOW-UP

What happens to in-flight external payments at the deadline?

Answer: After the deadline, the local request may end but the remote payment can still commit. Keep a stable operation ID and reconcile the final outcome rather than assuming failure.

Source guide p. 85 • Senior • References: 001, 003, 004

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

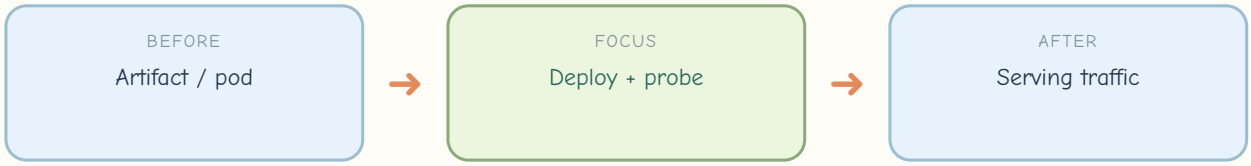
QUESTION 124 / 180

How should Kubernetes probes be selected?

01 / THE SHORT INTERVIEW ANSWER

Use startup for slow initialization, readiness for traffic eligibility, and liveness for genuine restart-worthy process failure.

DRAW THE IDEA — Kubernetes probes selected



02 / THE ORIGINAL EXAMPLE

```
startupProbe -> allow expected startup budget
readinessProbe -> application readiness endpoint
livenessProbe -> application liveness endpoint
terminationGracePeriodSeconds -> include shutdown budget
```

03 / WHY THIS WORKS

Coordinate thresholds and request timeouts. A probe against a separate management port can succeed while the business port is unavailable; test the same relevant serving path.

04 / WATCH OUT

Dependency outages should not cause cluster-wide restart storms.

05 / INTERVIEWER FOLLOW-UP

How would you verify readiness removal during rollout?

Answer: During a rollout, deliberately make one pod unready and confirm endpoint removal and traffic drain before termination. Inspect Service endpoints and ingress behavior.

Source guide p. 86 • Senior • References: 001, 003, 004

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

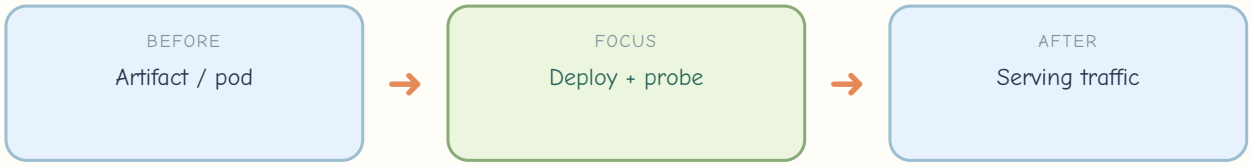
QUESTION 125 / 180

How should database capacity be budgeted across replicas?

01 / THE SHORT INTERVIEW ANSWER

Account for maximum replicas, rollout overlap, migrations, other clients, and per-instance connection limits.

DRAW THE IDEA — database capacity budgeted across



02 / THE ORIGINAL EXAMPLE

```
Normal: 8 replicas x 15 connections = up to 120
Rollout overlap: 10 x 15 = up to 150
Database budget must also cover non-service clients.
```

03 / WHY THIS WORKS

Autoscaling can increase waiting database concurrency rather than throughput. Align admission, pool settings, and database capacity before scaling. Observe the actual active connections rather than only configured maxima.

04 / WATCH OUT

A per-pod setting is not a cluster-wide limit.

05 / INTERVIEWER FOLLOW-UP

What does maxSurge do to peak resource use?

Answer: maxSurge permits temporary extra pods during deployment. Count their database pools, memory and CPU in the peak budget, not only the desired steady-state replica count.

Source guide p. 86 • Senior • References: 001, 003, 004

CHAPTER 21 • PACKAGING, DOCKER AND KUBERNETES DEPLOYMENT

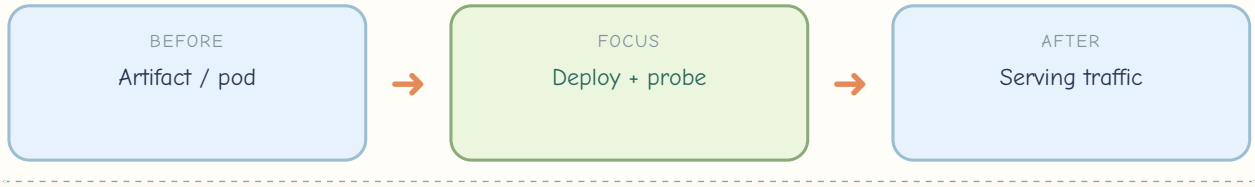
QUESTION 126 / 180

What makes a rollout safely reversible?

01 / THE SHORT INTERVIEW ANSWER

Backward-compatible contracts and schema transitions, a validated artifact, visible health, and a concrete rollback procedure.

DRAW THE IDEA — makes rollout safely reversible



02 / THE ORIGINAL EXAMPLE

```
Expand schema -> deploy compatible code -> verify
Migrate consumers/data -> later remove old schema
Rollback code -> previous version still understands current schema
```

03 / WHY THIS WORKS

Destructive schema changes can make code rollback impossible. Validate readiness and key business paths after deployment, not just pod count. Keep configuration and secret versions part of the release record.

04 / WATCH OUT

A ready pod is not proof that transfers or authorization work.

05 / INTERVIEWER FOLLOW-UP

What evidence would block a release?

Answer: Block rollout when critical authorization, payment idempotency, schema compatibility, readiness, data recovery or rollback evidence is missing for the release risk profile.

Source guide p. 87 • Senior • References: 001, 003, 004

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

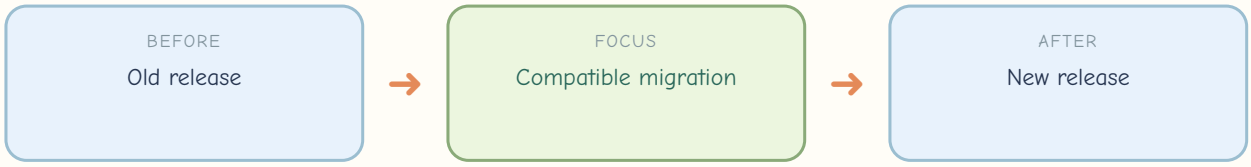
QUESTION 127 / 180

Why does Boot 3 require changes beyond a version number?

01 / THE SHORT INTERVIEW ANSWER

Boot 3 moves to the newer Spring ecosystem, Java 17+ baseline, and Jakarta namespaces for relevant enterprise APIs.

DRAW THE IDEA — require changes beyond version



02 / THE ORIGINAL EXAMPLE

```
// Boot 3 examples:
import jakarta.persistence.Entity;
import jakarta.validation.Valid;
import jakarta.annotation.PostConstruct;
```

03 / WHY THIS WORKS

Replacing imports is only one step. Review removed APIs, dependency compatibility, security configuration, behavior changes, and tests. Not every javax package moves; Java SE packages have their own ownership.

04 / WATCH OUT

Blanket javax-to-jakarta replacement can damage valid Java SE imports.

05 / INTERVIEWER FOLLOW-UP

How would an incompatible old library fail?

Answer: An old binary may throw NoClassDefFoundError, NoSuchMethodError or namespace-related failures at startup/invocation. Inspect transitive dependencies for javax/jakarta and API mismatches.

Source guide p. 88 • Senior • References: B02, M01, M02, T02

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

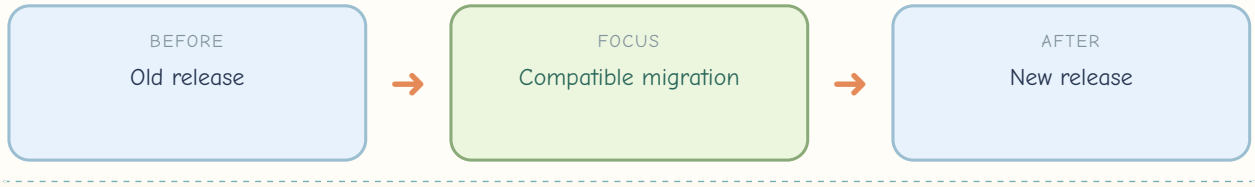
QUESTION 128 / 180

What should you know before moving Boot 3 to Boot 4?

01 / THE SHORT INTERVIEW ANSWER

> **Read the exact migration guide and compatibility matrix; Boot 4 changes the underlying Spring generation and modular organization.**

DRAW THE IDEA — know before moving



02 / THE ORIGINAL EXAMPLE

```
Inventory -> Boot, Framework, Security, Data, Cloud, third-party libraries
Upgrade dependencies -> repair imports/modules/test setup
Run behavior, serialization, security, and deployment checks
```

03 / WHY THIS WORKS

This guide's worked baseline remains Boot 3.x to avoid mixing examples. Boot 4 needs targeted verification of new modules, package locations and defaults. Use the selected release documentation instead of assuming all Boot 3 snippets copy unchanged.

04 / WATCH OUT

> The Java baseline alone does not prove library compatibility.

05 / INTERVIEWER FOLLOW-UP

How would Spring Cloud version alignment be verified?

Answer: Check the supported Spring Cloud release-train-to-Boot matrix and managed BOM versions, then test actual cloud starters in a smoke/integration application.

Source guide p. 88 • Senior • References: B02, M01, M02, T02

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

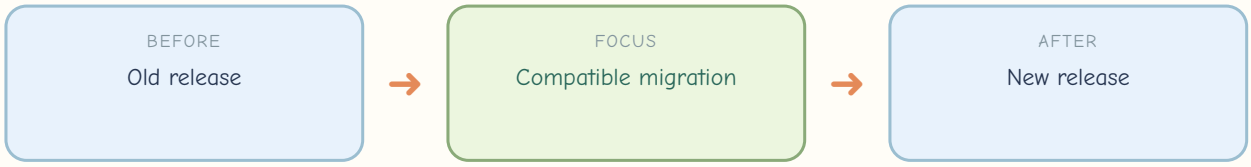
QUESTION 129 / 180

How should deprecated @MockBean examples be modernized?

01 / THE SHORT INTERVIEW ANSWER

For the supported Boot 3.5/Framework baseline, prefer Framework's @MockitoBean where its behavior fits.

DRAW THE IDEA — deprecated @MockBean examples modernize



02 / THE ORIGINAL EXAMPLE

```
import org.springframework.test.context.bean.override.mockito.MockitoBean;
@MockitoBean OrderService orderService;
// Check override semantics, qualifiers and scope in the test.
```

03 / WHY THIS WORKS

The replacement is more than a spelling exercise in complex test contexts. Review bean selection, hierarchy behavior, and exact version availability. Legacy interview questions may still use @MockBean.

04 / WATCH OUT

Do not use a modern annotation on an older unsupported framework version.

05 / INTERVIEWER FOLLOW-UP

How would you require an existing bean rather than create one?

Answer: Use @MockitoBean(enforceOverride=true) where supported to require replacement of an existing bean. Review selected framework version; the feature is version-specific.

Source guide p. 89 • Senior • References: B02, M01, M02, T02

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

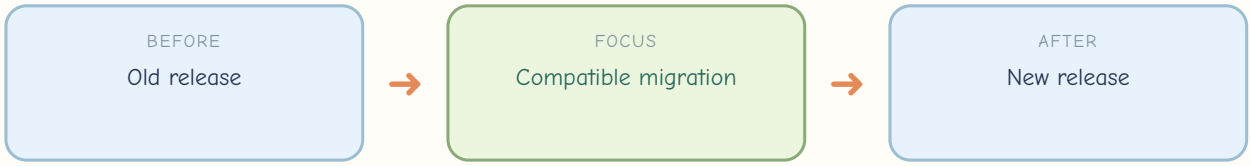
QUESTION 130 / 180

What are AOT processing and native images?

01 / THE SHORT INTERVIEW ANSWER

AOT prepares supported application infrastructure at build time; a native image compiles an executable with a constrained dynamic model.

DRAW THE IDEA — AOT processing native images



02 / THE ORIGINAL EXAMPLE

```
Build-time analysis -> generated infrastructure and hints
Native build -> platform-specific executable
Runtime -> validate reflection, resources, proxies, serialization
```

03 / WHY THIS WORKS

AOT and native packaging are related but distinct. Dynamic classpath behavior and reflection may need runtime hints. Measure startup, memory, throughput, build time, and operational tooling rather than assuming a free improvement.

04 / WATCH OUT

A native binary is not portable Java bytecode.

05 / INTERVIEWER FOLLOW-UP

Which plugin-loading behavior might be constrained?

Answer: Native-image closed-world analysis can restrict classes or plugins discovered dynamically after build time. Declare reflection/resource hints or choose JVM deployment when dynamic loading is essential.

Source guide p. 89 • Senior • References: B02, M01, M02, T02

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

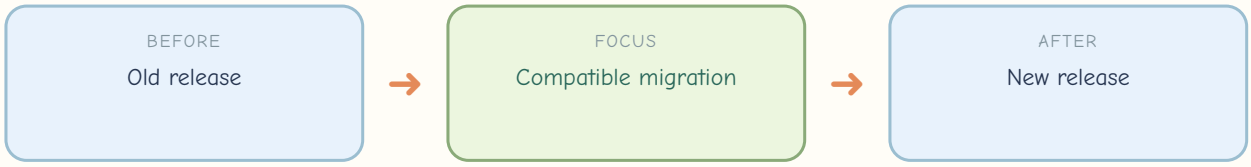
QUESTION 131 / 180

How should Java 21 features be introduced into Boot services?

01 / THE SHORT INTERVIEW ANSWER

Choose a compatible Boot/runtime baseline and evaluate one behavior change with representative correctness and capacity tests.

DRAW THE IDEA — Java features introduced into



02 / THE ORIGINAL EXAMPLE

```
Use records for DTOs -> validate serialization contract
Enable virtual threads -> measure downstream saturation
Use modern language features -> compile and deploy same target
```

03 / WHY THIS WORKS

Language syntax, library APIs, and execution configuration have different compatibility requirements. Virtual threads cannot eliminate connection limits. Keep build --release and deployed runtime aligned.

04 / WATCH OUT

Developers' local JDKs can hide production version incompatibility.

05 / INTERVIEWER FOLLOW-UP

How would CI enforce the supported bytecode target?

Answer: Compile CI with javac --release <target> or Maven compiler release, run bytecode checks, and test on the oldest supported production JVM.

Source guide p. 90 • Senior • References: B02, M01, M02, T02

CHAPTER 22 • MODERN SPRING AND MIGRATION CHOICES

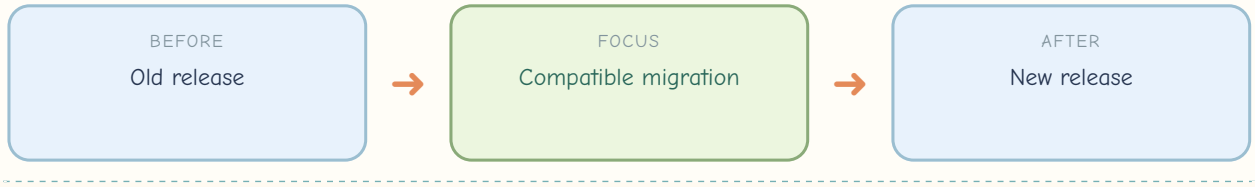
QUESTION 132 / 180

When should a modular monolith precede microservices?

01 / THE SHORT INTERVIEW ANSWER

Prefer clear module boundaries within one deployment when independent distribution does not yet justify its coordination cost.

DRAW THE IDEA — modular monolith precede microservices



02 / THE ORIGINAL EXAMPLE

```
Orders module -> published application interface
Billing module -> independent ownership and tests
One deployment -> simpler local transactions
```

03 / WHY THIS WORKS

A Boot application can be a well-structured monolith. Microservices add network failures, versioned contracts, deployment complexity, and distributed consistency choices. Split when ownership, scaling, release cadence, or fault isolation supports it.

04 / WATCH OUT

Multiple controllers do not constitute microservices.

05 / INTERVIEWER FOLLOW-UP

What boundary could be extracted without sharing its tables?

Answer: Extract a bounded business capability whose APIs, data, migration and team ownership can be separate. Stop other modules from reading its tables directly before remote extraction.

Source guide p. 90 • Senior • References: B02, M01, M02, T02

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

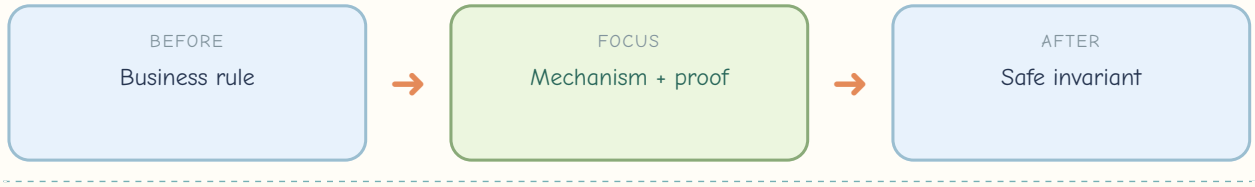
QUESTION 133 / 180

How do JDK proxies and class-based proxies differ?

01 / THE SHORT INTERVIEW ANSWER

JDK proxies implement interfaces; class-based proxies subclass eligible classes, so final/private method constraints matter.

DRAW THE IDEA — JDK proxies class-based proxies



02 / THE ORIGINAL EXAMPLE

```
Interface proxy -> expose interface methods
Class proxy -> subclass target type
final method -> cannot be overridden for subclass advice
```

03 / WHY THIS WORKS

The actual proxy strategy depends on configuration and integration. Do not assume a method is advised merely because it is public on an arbitrary object. Distinguish proxy type, method eligibility, and call path.

04 / WATCH OUT

A final class can constrain subclass proxy creation.

05 / INTERVIEWER FOLLOW-UP

How would injecting the concrete class affect an interface proxy?

Answer: An interface-based JDK proxy may not be assignable to the implementation concrete type. Inject the interface, or deliberately choose compatible class-based proxying where required.

Source guide p. 91 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

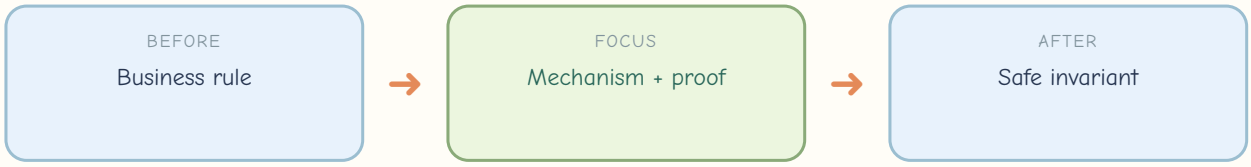
QUESTION 134 / 180

How does @Around advice differ from before and after advice?

01 / THE SHORT INTERVIEW ANSWER

Around advice controls whether and how the invocation proceeds and can observe or replace its result or failure.

DRAW THE IDEA — @Around advice before after



02 / THE ORIGINAL EXAMPLE

```
@Around("execution(* com.example.service.*.*(..))")
Object time(ProceedingJoinPoint p) throws Throwable {
  long start = System.nanoTime();
  try { return p.proceed(); }
  finally { recordElapsed(System.nanoTime() - start); }
}
```

03 / WHY THIS WORKS

Calling proceed performs the target invocation. The finally observes success or failure without masking the original outcome. This example assumes aspect registration and a safe recordElapsed implementation.

04 / WATCH OUT

Throwing from diagnostic advice can replace business failure.

05 / INTERVIEWER FOLLOW-UP

Why can repeated proceed duplicate a side effect?

Answer: Calling proceed more than once can rerun a payment or write method and duplicate effects. Retry advice must enforce idempotency and bounded attempts.

Source guide p. 92 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

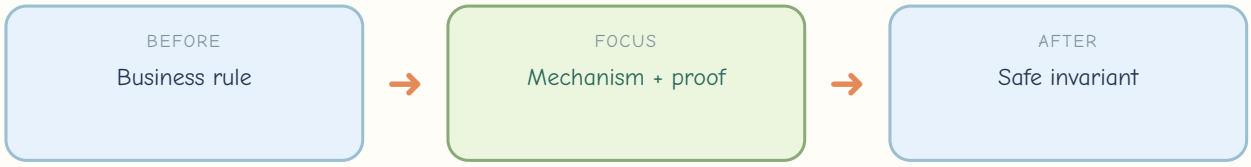
QUESTION 135 / 180

What is the difference between bean ordering and bean creation dependencies?

01 / THE SHORT INTERVIEW ANSWER

Ordering controls selected collections, callbacks, or advisors; construction order follows actual dependencies and explicit depends-on rules.

DRAW THE IDEA — difference bean ordering bean



02 / THE ORIGINAL EXAMPLE

```
@Order(10) class FirstHandler implements Handler {}
@Order(20) class SecondHandler implements Handler {}
// Injected ordered handler list can reflect these priorities.
```

03 / WHY THIS WORKS

This does not universally force FirstHandler to be constructed first. Auto-configuration ordering controls definition application, not every runtime bean lifecycle. State which ordering API owns the case.

04 / WATCH OUT

@Order is not a generic startup dependency declaration.

05 / INTERVIEWER FOLLOW-UP

When would @DependsOn be appropriate?

Answer: @DependsOn fits required initialization ordering between beans with side-effectful lifecycle steps. Prefer explicit constructor dependencies when one bean actually requires another.

Source guide p. 92 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

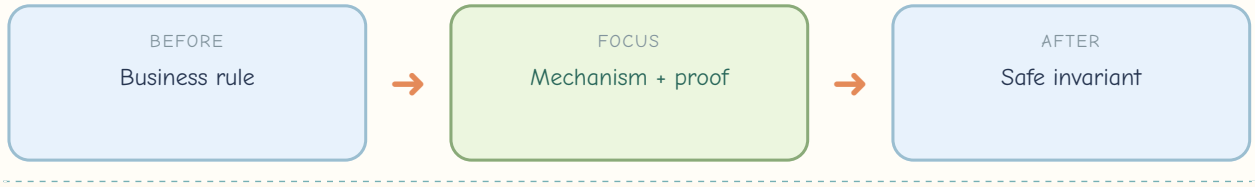
QUESTION 136 / 180

What does @TransactionalEventListener provide?

01 / THE SHORT INTERVIEW ANSWER

> It binds handling to a transaction phase such as after commit, when an event is published in an applicable transaction.

DRAW THE IDEA — @TransactionalEventListener provide



02 / THE ORIGINAL EXAMPLE

```
@TransactionalEventListener(phase=TransactionPhase.AFTER_COMMIT)
void committed(OrderPlaced e) { notifyBestEffort(e.id()); }
```

03 / WHY THIS WORKS

After-commit timing avoids sending a success signal before rollback, but an in-process event is not durable delivery. New database writes at this phase need a deliberate fresh transaction policy. Without a transaction, fallback behavior must be configured intentionally.

04 / WATCH OUT

A crash after commit can lose a non-durable notification.

05 / INTERVIEWER FOLLOW-UP

When should outbox replace an in-process event?

Answer: Use outbox when the event must survive process crashes, reach another service, or be replayed. @TransactionalEventListener is an in-process transaction-phase callback, not durable messaging.

Source guide p. 93 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

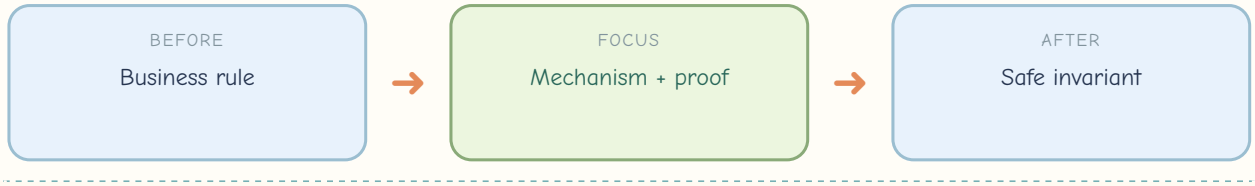
QUESTION 137 / 180

How do connection acquisition, transaction timeout, and query timeout differ?

01 / THE SHORT INTERVIEW ANSWER

They bound different waiting or work scopes and are enforced by different components.

DRAW THE IDEA — connection acquisition transaction time



02 / THE ORIGINAL EXAMPLE

```
Hikari acquisition timeout -> wait for available connection
Transaction timeout -> manager's applicable transaction limit
Query timeout -> database statement/provider execution policy
```

03 / WHY THIS WORKS

Do not expect a connection-wait setting to cancel a running SQL statement. Driver, manager, and database behavior determine enforcement. Add an overall request deadline and test how failure releases resources.

04 / WATCH OUT

Timeout property names are not interchangeable across clients.

05 / INTERVIEWER FOLLOW-UP

How would a network stall differ from a database lock wait?

Answer: Database lock waits show blocked statements and locking sessions; a network stall shows socket/connection wait behavior and transport timing. Correlate driver, DB and pool metrics.

Source guide p. 93 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

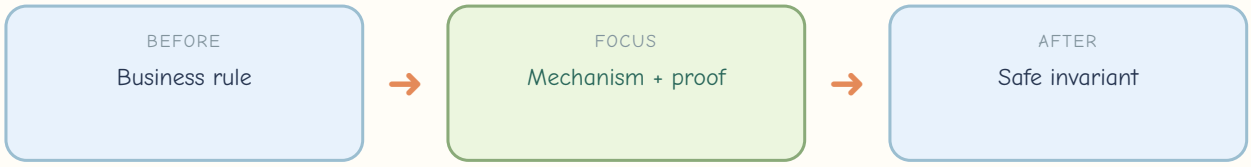
QUESTION 138 / 180

How should multiple DataSources and transaction managers be configured?

01 / THE SHORT INTERVIEW ANSWER

Define and qualify each infrastructure chain and select the intended manager for each transaction boundary.

DRAW THE IDEA — multiple DataSources transaction manage



02 / THE ORIGINAL EXAMPLE

```
@Transactional("billingTransactionManager")
public void recordCharge() { billingRepository.save(charge); }
// Configure matching DataSource, EntityManagerFactory,
// repository package and transaction manager.
```

03 / WHY THIS WORKS

One local manager does not automatically atomically commit both databases. Decide whether distributed coordination, outbox/saga, or a single authoritative store fits. Avoid ambiguous default bean selection.

04 / WATCH OUT

@Primary selects a default but does not unify consistency boundaries.

05 / INTERVIEWER FOLLOW-UP

How would a second database failure be recovered?

Answer: Two separate local transactions cannot automatically undo each other. Use durable coordination, idempotency and compensation or a properly supported distributed transaction protocol.

Source guide p. 94 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

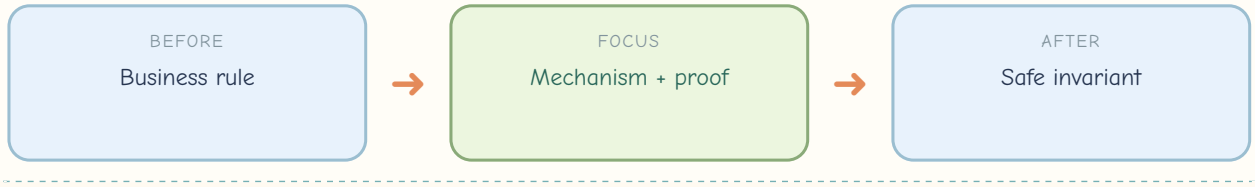
QUESTION 139 / 180

When are query specifications useful?

01 / THE SHORT INTERVIEW ANSWER

> **They compose typed optional predicates without building untrusted query strings, while query and index behavior still need inspection.**

DRAW THE IDEA — query specifications useful



02 / THE ORIGINAL EXAMPLE

```
Specification<Customer> active = (root, query, cb) ->
    cb.isTrue(root.get("active"));
Specification<Customer> tenant = (root, query, cb) ->
    cb.equal(root.get("tenantId"), tenantId);
repository.findAll(active.and(tenant), pageable);
```

03 / WHY THIS WORKS

The repository must support JpaSpecificationExecutor. Keep mandatory tenant predicates authoritative rather than optional client filters. Inspect joins, count queries, indexes, and returned row shape.

04 / WATCH OUT

> Type-safe construction is not automatic row authorization.

05 / INTERVIEWER FOLLOW-UP

When would a projection-specific query be clearer?

Answer: Use a targeted projection query when the endpoint needs a fixed field set or joins with predictable SQL. Specifications help flexible query predicates, not necessarily optimized read shapes.

Source guide p. 94 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

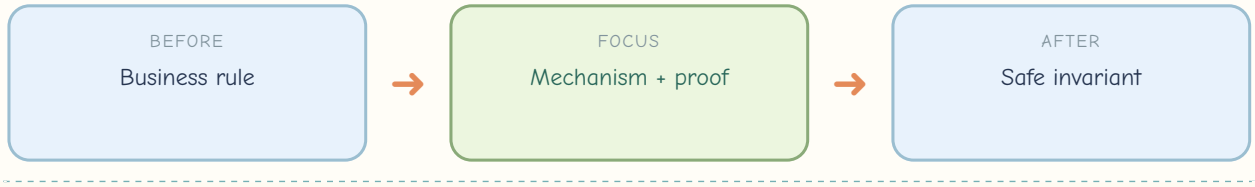
QUESTION 140 / 180

How can identifier generation affect batching?

01 / THE SHORT INTERVIEW ANSWER

> **The database/provider identifier strategy can affect when INSERTs execute and whether efficient batching is possible.**

DRAW THE IDEA — identifier generation affect batching



02 / THE ORIGINAL EXAMPLE

```
@GeneratedValue(strategy=GenerationType.IDENTITY) Long id;
// Compare with an appropriate sequence strategy where supported.
// Measure actual statement batches on the chosen engine.
```

03 / WHY THIS WORKS

Some identity-generation paths require obtaining ids from executed inserts, constraining insert batching. Sequence support and allocation policies vary by database. MySQL and PostgreSQL should not be assumed identical.

04 / WATCH OUT

> Configuring batch_size alone does not prove batching occurred.

05 / INTERVIEWER FOLLOW-UP

How would generated-key requirements change an import design?

Answer: Generated identity keys may force insert round-trips and restrict batching. Consider supported sequences, allocation strategies or staging/import operations after measuring.

Source guide p. 95 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

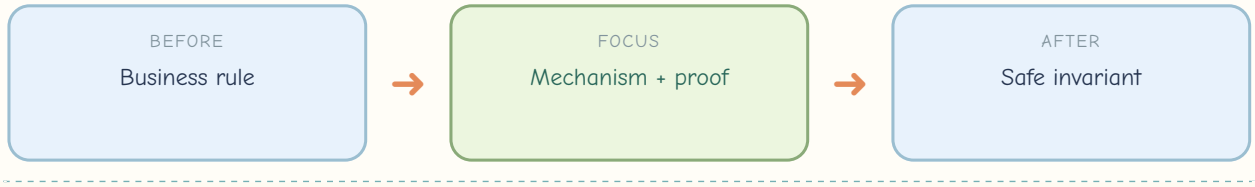
QUESTION 141 / 180

Why should indexes be chosen from workload evidence?

01 / THE SHORT INTERVIEW ANSWER

An index accelerates suitable lookup/order patterns but costs storage and write maintenance, and may not match a composite predicate.

DRAW THE IDEA — indexes chosen workload evidence



02 / THE ORIGINAL EXAMPLE

```
WHERE tenant_id = ? AND created_at < ?
ORDER BY created_at DESC, id DESC
Candidate index -> aligned tenant, time, id columns
```

03 / WHY THIS WORKS

Use the database execution plan with representative data and selectivity. Verify ordering, range conditions, rows examined, and write overhead. Exact index direction and plan behavior are engine-specific.

04 / WATCH OUT

An index on each individual column is not equivalent to a fitting composite index.

05 / INTERVIEWER FOLLOW-UP

How would low selectivity change the plan?

Answer: Low selectivity means an index may read too many rows and lose to a scan. Check actual query plans, row counts, filter order and covering possibilities.

Source guide p. 95 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

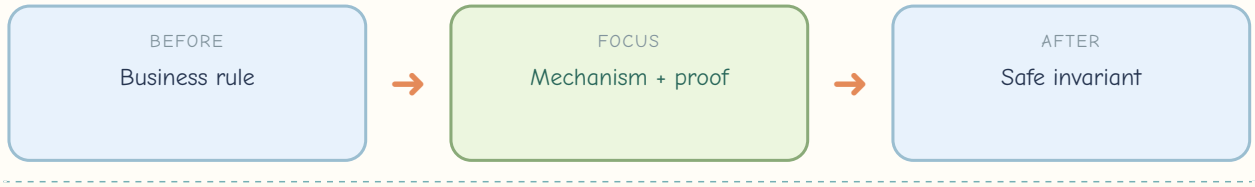
QUESTION 142 / 180

Does changing a configuration file refresh running beans automatically?

01 / THE SHORT INTERVIEW ANSWER

Ordinary Boot startup binding is not a general live-refresh mechanism; dynamic changes need an explicit supported design.

DRAW THE IDEA — changing configuration file refresh



02 / THE ORIGINAL EXAMPLE

```
Startup -> bind timeout=2s into bean
Edit file -> existing bean normally remains unchanged
Refresh policy -> supported rebinding or new immutable snapshot
```

03 / WHY THIS WORKS

Choose which settings may change, validate replacement atomically, and define resource reconfiguration. A new password value does not automatically replace every connection in an existing pool.

04 / WATCH OUT

Restart-required and live settings should not be mixed implicitly.

05 / INTERVIEWER FOLLOW-UP

How would readers observe a complete new policy snapshot?

Answer: Publish one validated immutable configuration object through a volatile/atomic reference. Each request reads the reference once so fields come from one version.

Source guide p. 96 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

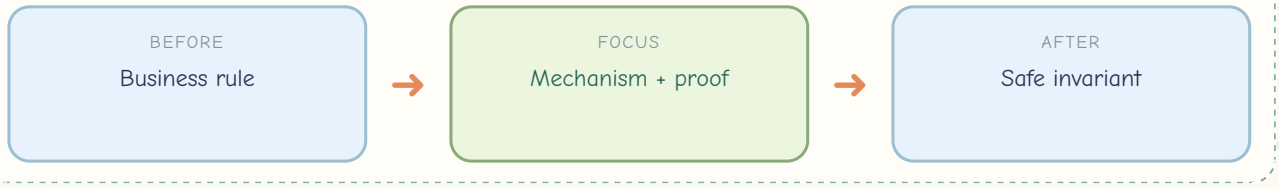
QUESTION 143 / 180

How do local and distributed caches differ?

01 / THE SHORT INTERVIEW ANSWER

> **Local caches offer per-instance low-latency storage; distributed caches share data across instances with network and consistency costs.**

DRAW THE IDEA — local distributed caches



02 / THE ORIGINAL EXAMPLE

```
Caffeine -> one process owns local entries
Redis -> service instances share remote cache state
Two-level cache -> additional invalidation and freshness policy
```

03 / WHY THIS WORKS

A local cache hit can diverge from another pod's result. A shared store still needs serialization, TTL, key versioning, availability, and invalidation semantics. Choose from workload and correctness requirements.

04 / WATCH OUT

> Redis being shared does not guarantee immediately coherent local near-caches.

05 / INTERVIEWER FOLLOW-UP

What is the stale-data tolerance?

Answer: Document maximum acceptable staleness by use case. Prices, authorization and stock may need stronger reads than marketing descriptions or optional recommendations.

Source guide p. 96 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

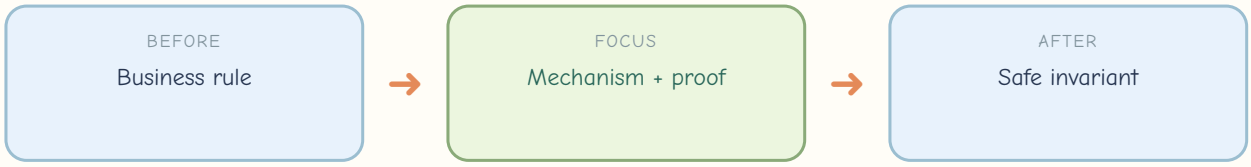
QUESTION 144 / 180

Why is a Redis lock not automatically safe for irreversible work?

01 / THE SHORT INTERVIEW ANSWER

Lease expiry, pauses and ownership transfer can let an old holder continue after a new holder acquires the lock.

DRAW THE IDEA — Redis lock not automatically



02 / THE ORIGINAL EXAMPLE

```
Owner A lease expires during a long pause
Owner B obtains new lease and proceeds
A resumes -> two actors may perform the protected effect
```

03 / WHY THIS WORKS

Use appropriate fencing or an authoritative idempotent/transactional protocol when the effect requires it. Unlock only the token you own and define renewal failure. The design must match the datastore and failure model.

04 / WATCH OUT

A lease is not proof that the old process stopped.

05 / INTERVIEWER FOLLOW-UP

Can the downstream service reject stale fencing tokens?

Answer: Only if the downstream storage/API checks monotonically advancing tokens and rejects obsolete ones. A lease alone cannot prevent a paused owner from resuming a harmful write.

Source guide p. 97 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

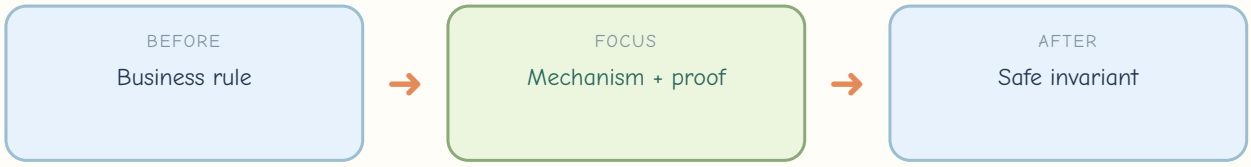
QUESTION 145 / 180

How should multipart upload limits and file ownership be designed?

01 / THE SHORT INTERVIEW ANSWER

Bound file and request size, validate accepted types, constrain storage paths, and clean up temporary resources under failure.

DRAW THE IDEA — multipart upload limits file



02 / THE ORIGINAL EXAMPLE

```
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=12MB
# Also configure upstream/proxy limits and processing admission.
```

03 / WHY THIS WORKS

The values are illustrative. Content-Type and extension are untrusted hints; inspect required content and avoid loading huge files into memory. Virus scanning or quarantine depends on the product's needs.

04 / WATCH OUT

Multipart limits do not replace path traversal protection.

05 / INTERVIEWER FOLLOW-UP

What happens if processing fails after a temporary file is created?

Answer: Own the temporary file lifecycle and delete or quarantine it in a failure path, preferably with scheduled orphan cleanup. Make processing retryable by upload identity.

Source guide p. 97 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

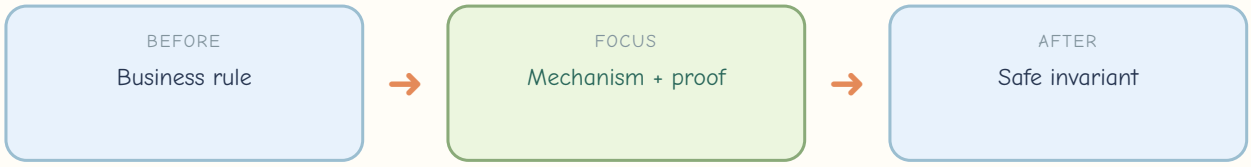
QUESTION 146 / 180

What is the distinction between synchronous MVC and MVC async timeout?

01 / THE SHORT INTERVIEW ANSWER

> **MVC async request timeout governs supported asynchronous request processing; it is not a universal timeout for ordinary blocking controllers.**

DRAW THE IDEA — distinction synchronous MVC MVC



02 / THE ORIGINAL EXAMPLE

```
spring.mvc.async.request-timeout=2s
// Applies to MVC asynchronous handling, not every synchronous method.
// Downstream clients and synchronous work need their own limits.
```

03 / WHY THIS WORKS

Map the specific servlet/async lifecycle before assuming cancellation. Expiring an HTTP response does not guarantee a background task stopped. Propagate deadlines and handle unknown business outcomes.

04 / WATCH OUT

A proxy timeout may close the client while server work continues.

05 / INTERVIEWER FOLLOW-UP

Which layer owns stopping an expensive export?

Answer: The job or request coordinator must own cancellation and limits; an MVC async timeout does not guarantee expensive work or an external call stopped.

Source guide p. 98 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

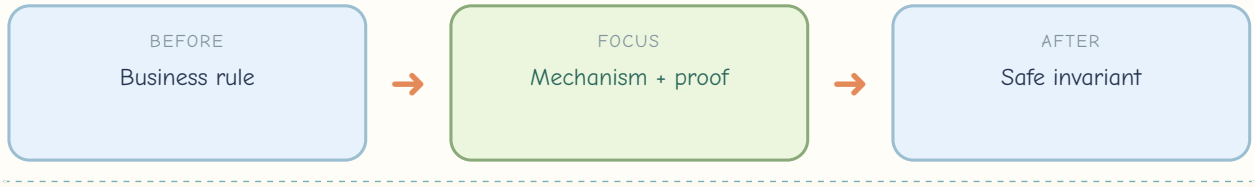
QUESTION 147 / 180

How do client-side load balancing and discovery differ?

01 / THE SHORT INTERVIEW ANSWER

Discovery identifies service instances; load balancing chooses where a call should go, with health and connection behavior part of the contract.

DRAW THE IDEA — client-side load balancing discovery



02 / THE ORIGINAL EXAMPLE

```
Logical service name -> discover candidates
Load-balancer policy -> choose an instance
Client call -> deadline, retry, pool, outcome
```

03 / WHY THIS WORKS

A Kubernetes Service may already provide routing, while another stack may use client-side selection. Avoid duplicating mechanisms without evidence. Use compatible Spring Cloud release alignment where that ecosystem is selected.

04 / WATCH OUT

Discovery success does not prove the chosen instance is ready.

05 / INTERVIEWER FOLLOW-UP

How would retry change routing and idempotency?

Answer: Service discovery locates instances; load balancing chooses one. Retrying to another instance must preserve an idempotency key when the earlier request's outcome is unknown.

Source guide p. 98 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

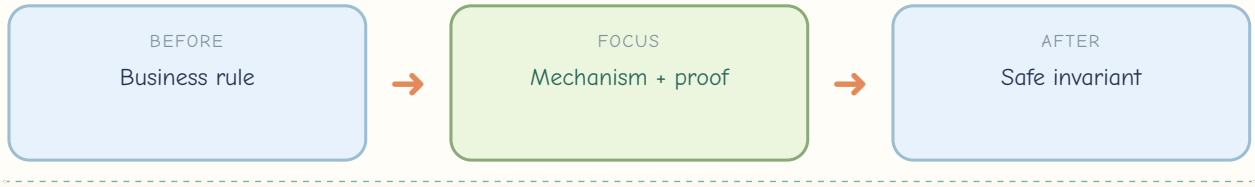
QUESTION 148 / 180

What should you know when using OpenFeign with Boot?

01 / THE SHORT INTERVIEW ANSWER

Feign supplies declarative client interfaces; timeouts, encoding, errors, authentication, pools and version alignment remain configuration responsibilities.

DRAW THE IDEA — know OpenFeign



02 / THE ORIGINAL EXAMPLE

```
@FeignClient(name="catalog")
interface CatalogClient {
  @GetMapping("/products/{id}") ProductDto get(@PathVariable("id") long id);
}
```

03 / WHY THIS WORKS

This requires compatible Spring Cloud OpenFeign and client registration. The standard use is synchronous, so do not treat it as a reactive client. Verify retry defaults for the integration rather than assuming raw Feign defaults.

04 / WATCH OUT

Annotation convenience does not remove remote-failure handling.

05 / INTERVIEWER FOLLOW-UP

How will tracing and token propagation be controlled?

Answer: Explicitly propagate trusted trace headers and authorized tokens using interceptors/configuration; never copy arbitrary inbound Authorization headers to unrelated downstream services.

Source guide p. 99 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

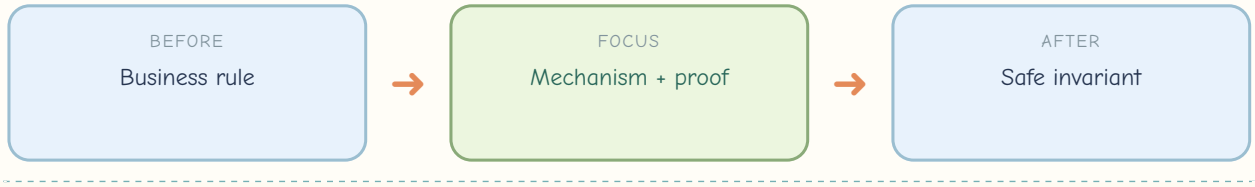
QUESTION 149 / 180

How do rate limiting and business quotas differ?

01 / THE SHORT INTERVIEW ANSWER

> **Rate limiting controls request admission over time; a business quota tracks a durable domain entitlement or consumption rule.**

DRAW THE IDEA — rate limiting business quotas



02 / THE ORIGINAL EXAMPLE

```
Rate: up to N calls per time window
Quota: purchased allowance remaining for this account
// Choose authoritative identity, storage, and concurrency policy.
```

03 / WHY THIS WORKS

A process-local limiter may multiply capacity across replicas. A durable quota needs consistent accounting, idempotency and failure handling. Return a meaningful throttle contract with retry information where appropriate.

04 / WATCH OUT

IP address alone may be a poor account identity.

05 / INTERVIEWER FOLLOW-UP

What must happen atomically when a paid quota is consumed?

Answer: Consume a paid quota with an atomic authoritative reservation and durable usage record. A read-then-increment across replicas can overspend the quota.

Source guide p. 99 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

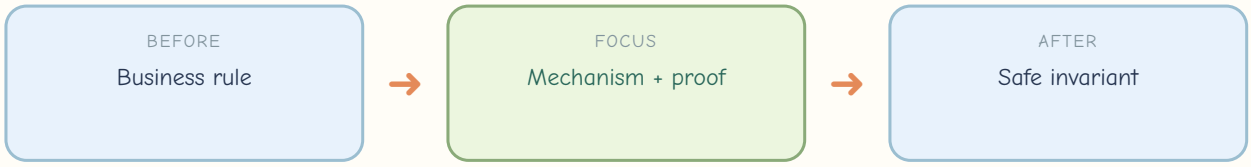
QUESTION 150 / 180

How should Spring Batch be distinguished from a scheduled method?

01 / THE SHORT INTERVIEW ANSWER

Spring Batch supplies job/step metadata, chunk processing and restart semantics; scheduling only determines when work is triggered.

DRAW THE IDEA — Batch distinguished scheduled method



02 / THE ORIGINAL EXAMPLE

```
Schedule -> launch Job
Job -> Step -> read/process/write chunks
Repository -> execution and restart metadata
```

03 / WHY THIS WORKS

Choose a job identity and parameters carefully, then test restart after a failed chunk. The batch version and transaction boundaries matter. A for-loop inside @Scheduled does not acquire durable recovery semantics automatically.

04 / WATCH OUT

Re-running a job needs duplicate-effect policy.

05 / INTERVIEWER FOLLOW-UP

What state should a restart skip versus repeat?

Answer: On restart, skip committed/recorded work and repeat only idempotent uncommitted items based on durable job/step state and checkpoint semantics.

Source guide p. 100 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

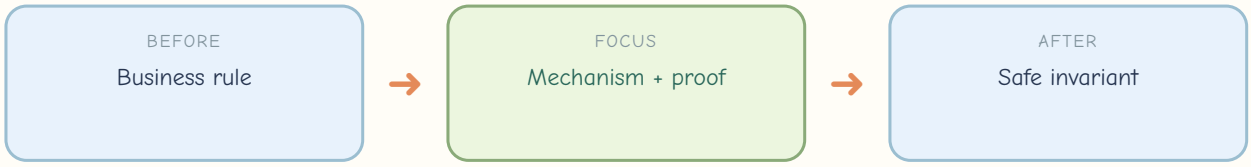
QUESTION 151 / 180

How should a saga differ from a local @Transactional method?

01 / THE SHORT INTERVIEW ANSWER

> **A saga coordinates multiple independently committed steps and compensations; a local transaction rolls back participating resources atomically.**

DRAW THE IDEA — saga local @Transactional method



02 / THE ORIGINAL EXAMPLE

```
Reserve stock -> authorize payment -> create shipment
Failure later -> release stock or void authorization as defined
```

03 / WHY THIS WORKS

Compensation is another business action, not time travel. It can fail or require manual resolution. Persist workflow state, deadlines and operation ids, and make both forward and compensating actions idempotent.

04 / WATCH OUT

> @Transactional does not turn remote services into one database.

05 / INTERVIEWER FOLLOW-UP

Which step is irreversible and needs special handling?

Answer: Mark irreversible steps such as external settlement or shipped goods explicitly; handle them with manual/reconciliation paths rather than pretending compensation is a database rollback.

Source guide p. 100 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

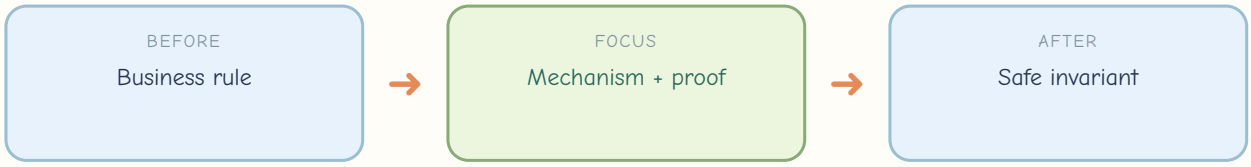
QUESTION 152 / 180

What is the difference between business audit and application logging?

01 / THE SHORT INTERVIEW ANSWER

Audit records prove defined business events and often require durability and controlled access; diagnostic logs help operate and debug the service.

DRAW THE IDEA — difference business audit logging



02 / THE ORIGINAL EXAMPLE

```
Audit: who approved order, when, under which operation identity
Log: dependency timed out, trace id, safe error context
```

03 / WHY THIS WORKS

Do not assume sampled, rotating debug logs satisfy an audit requirement. Define retention, tamper resistance, consistency, and sensitive-field policy. Conversely, avoid storing every diagnostic stack trace as a business audit event.

04 / WATCH OUT

Both channels can expose confidential information.

05 / INTERVIEWER FOLLOW-UP

Must an approval fail if its audit cannot be recorded?

Answer: If a required audit record is part of the business/legal invariant, approval must not commit without durable audit intent. Put both in one local transaction if possible.

Source guide p. 101 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

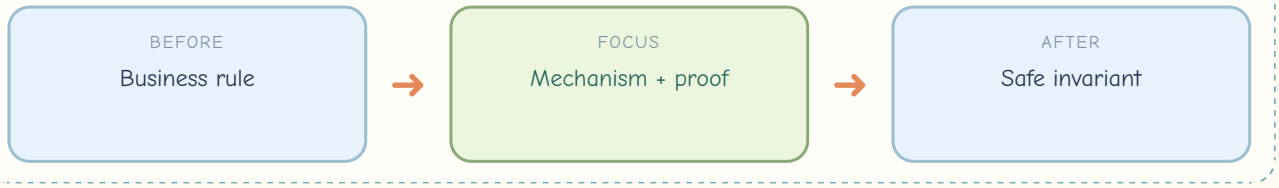
QUESTION 153 / 180

How should an idempotency-key store handle concurrent requests?

01 / THE SHORT INTERVIEW ANSWER

Atomically claim the key, bind it to the intended request, track completion, and return or reconcile the same operation outcome.

DRAW THE IDEA — idempotency-key store handle concurrent



02 / THE ORIGINAL EXAMPLE

```
Unique key -> claimed operation and payload fingerprint
Same key + same payload -> existing pending/completed outcome
Same key + different payload -> explicit conflict
```

03 / WHY THIS WORKS

A cache-only check followed by work races across requests. Store enough state to recover after a crash between external success and local completion. Define retention and pending-operation timeouts.

04 / WATCH OUT

Deleting a pending key can enable a duplicate effect.

05 / INTERVIEWER FOLLOW-UP

Who reconciles an unknown provider outcome?

Answer: The payment owner and business workflow must reconcile by the stable operation ID, using status lookup/callbacks and recorded uncertainty; a timeout alone is not a failure result.

Source guide p. 101 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

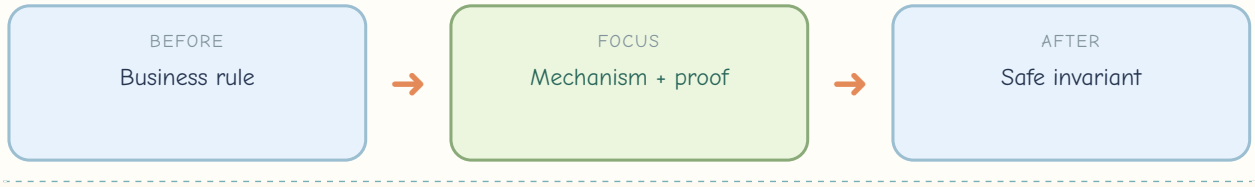
QUESTION 154 / 180

Why can optimistic retries still violate a business rule?

01 / THE SHORT INTERVIEW ANSWER

> **A retry must re-read and revalidate the full invariant, not blindly repeat arithmetic from stale state.**

DRAW THE IDEA — optimistic retries still violate



02 / THE ORIGINAL EXAMPLE

```
Read stock=1 -> conflict -> re-read stock=0
Business rule: reserve requires available >= 1
Retry -> reject now rather than force another decrement
```

03 / WHY THIS WORKS

A successful later write is not proof the original operation remained valid. Include other relevant rows and unique constraints in the invariant. Bound retry attempts and return a clear conflict or insufficient-capacity outcome.

04 / WATCH OUT

Retrying every optimistic failure can amplify contention.

05 / INTERVIEWER FOLLOW-UP

When would atomic conditional SQL be a better fit?

Answer: A conditional UPDATE … WHERE available>=? enforces a simple stock invariant atomically at the database, avoiding a stale read and repeat-update loop.

Source guide p. 102 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

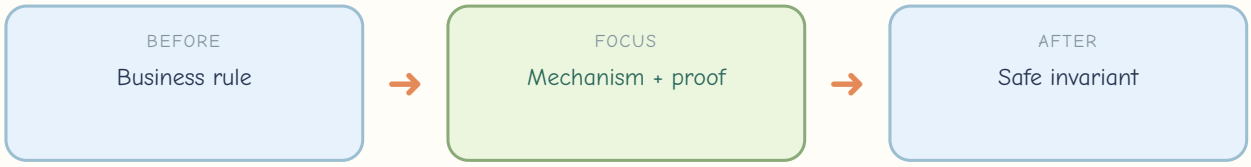
QUESTION 155 / 180

How should sensitive fields be protected during JSON serialization?

01 / THE SHORT INTERVIEW ANSWER

Use explicit DTOs and field policies, and verify every output and error path rather than relying only on one annotation.

DRAW THE IDEA — sensitive fields protected during



02 / THE ORIGINAL EXAMPLE

```
record PublicUser(long id, String name) {}
// passwordHash, token and internal flags are absent by design.
```

03 / WHY THIS WORKS

Serialization annotations can help but do not protect logs, exceptions, projections, alternate mappers, or toString. Test response fields and review caching/tracing representations.

04 / WATCH OUT

Ignoring a password field in JSON does not make an entity safe for all uses.

05 / INTERVIEWER FOLLOW-UP

How would you avoid accidentally echoing a rejected credential?

Answer: Use dedicated safe error DTOs and central exception handling. Never serialize the request object or rejected credential into logs or response ProblemDetail properties.

Source guide p. 102 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 23 • ADVANCED DESIGN AND INTERVIEW DISTINCTIONS

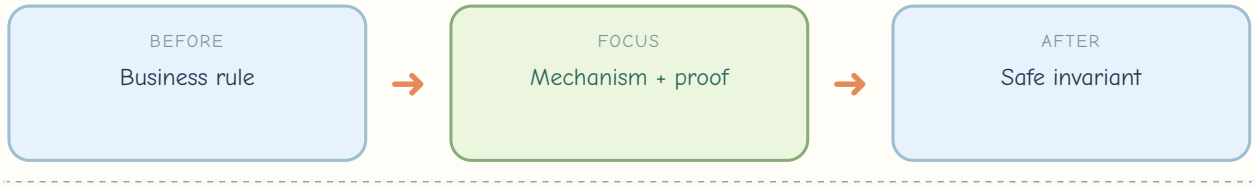
QUESTION 156 / 180

What should change when a service is deployed across regions?

01 / THE SHORT INTERVIEW ANSWER

Define data ownership, replication consistency, conflict handling, latency, and failover/recovery semantics explicitly.

DRAW THE IDEA — change deployed across regions



02 / THE ORIGINAL EXAMPLE

```
Local region routing -> lower request latency
Shared authoritative write region -> cross-region dependency
Multi-writer -> conflict and invariant policy required
```

03 / WHY THIS WORKS

Boot packaging is not the limiting design issue. Operation identity, clocks, timeouts, caches, and database consistency can behave differently across regions. Test partial partition and recovery using the chosen datastore's guarantees.

04 / WATCH OUT

Two healthy deployments do not prove one consistent business state.

05 / INTERVIEWER FOLLOW-UP

Which operations can safely remain region-local?

Answer: Read-only caches, telemetry and optional personalization may stay region-local; monetary balance, stock and other global invariants need an authority/replication protocol.

Source guide p. 103 • Senior • References: F03, D03, M03, M04, I03

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

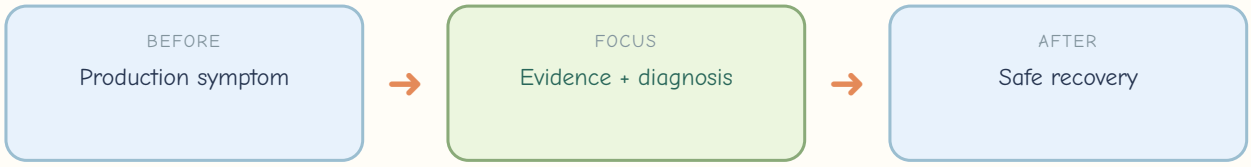
QUESTION 157 / 180

A bean exists in code but injection fails. What do you inspect?

01 / THE SHORT INTERVIEW ANSWER

Check registration, scanning, profile/condition activation, candidate type, qualifier, and the actual context before changing annotations.

DRAW THE IDEA — bean exists code but



02 / THE ORIGINAL EXAMPLE

```
Bean class -> inside scan root?
@Bean method -> configuration imported?
Condition/profile -> matched?
Dependency -> required type and qualifier?
```

03 / WHY THIS WORKS

Read whether the failure says no candidate, ambiguous candidates, or failed creation. They need different fixes. The condition report and context test can reproduce the relevant registration path.

04 / WATCH OUT

Moving the main class to the default package can create broad scanning problems.

05 / INTERVIEWER FOLLOW-UP

How would you test one conditional bean's absence?

Answer: Use ApplicationContextRunner with the exact conditions absent and assert the bean is missing. Include a positive case to prove the registration path works.

Source guide p. 104 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

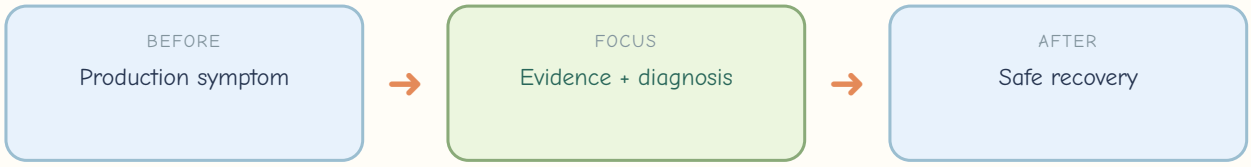
QUESTION 158 / 180

@Transactional appears on a method, but writes partially commit. Why?

01 / THE SHORT INTERVIEW ANSWER

Verify the proxy path, transaction manager, propagation, connection ownership, and failure handling rather than trusting the annotation's presence.

DRAW THE IDEA — @Transactional appears method but



02 / THE ORIGINAL EXAMPLE

```
Controller -> service proxy -> transaction active?
this.save() -> self-call, no new advice
caught checked failure -> may not trigger rollback
second data source -> may use another transaction boundary
```

03 / WHY THIS WORKS

Reproduce with an integration test that checks database state after failure. Also inspect autocommit or direct JDBC use and whether external side effects are outside the transaction.

04 / WATCH OUT

Annotated methods are not proof that every operation participates.

05 / INTERVIEWER FOLLOW-UP

Which log or test would prove the boundary?

Answer: A transaction trace/log should show begin/commit/rollback and connection ownership. An integration test that forces a checked failure can prove rollback semantics and the proxy boundary.

Source guide p. 105 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

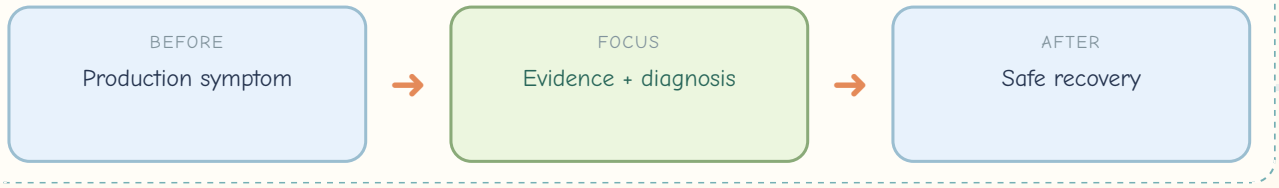
QUESTION 159 / 180

A list API runs hundreds of SQL statements. How do you fix it?

01 / THE SHORT INTERVIEW ANSWER

Measure the actual query pattern, identify lazy traversal or per-row lookups, and fetch the required shape deliberately.

DRAW THE IDEA — list API runs hundreds



02 / THE ORIGINAL EXAMPLE

```
List orders -> map each order's customer and lines
SQL count rises roughly with number of rows
Repair candidate -> projection, entity graph, or planned batch
```

03 / WHY THIS WORKS

Compare query count, rows transferred, database time, and memory after the change. A giant join can replace round trips with excessive duplication, so verify the actual result and pagination semantics.

04 / WATCH OUT

Changing every association to EAGER can worsen the endpoint.

05 / INTERVIEWER FOLLOW-UP

How would you detect the regression in a test?

Answer: Enable SQL/query counting in a real repository integration test and assert a fixed query budget as row count grows; compare before and after fetching strategy changes.

Source guide p. 105 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

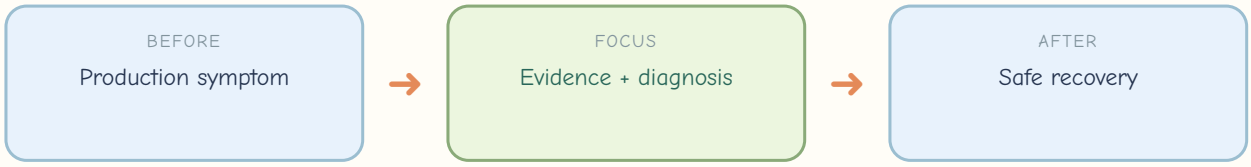
QUESTION 160 / 180

Latency rises while CPU stays low. What is your first hypothesis?

01 / THE SHORT INTERVIEW ANSWER

Investigate waits on pools, external I/O, locks, and queues using correlated thread and latency evidence.

DRAW THE IDEA — Latency rises while CPU



02 / THE ORIGINAL EXAMPLE

```
CPU low + Hikari pending high -> pool ownership/query wait
CPU low + executor queue high -> blocked workers or admission excess
CPU low + outbound spans long -> dependency slowdown
```

03 / WHY THIS WORKS

These are hypotheses, not conclusions. Time-aligned metrics and stacks determine the bottleneck. Bound waiting and repair the constrained dependency before blindly increasing threads.

04 / WATCH OUT

Adding replicas can multiply pressure on the same database.

05 / INTERVIEWER FOLLOW-UP

What would prove that connections are leaking?

Answer: Hikari active connections remain near max, pending requests grow, and connection-acquisition timeouts appear while completed request load falls. Inspect leaked-resource stack traces and slow transactions.

Source guide p. 106 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

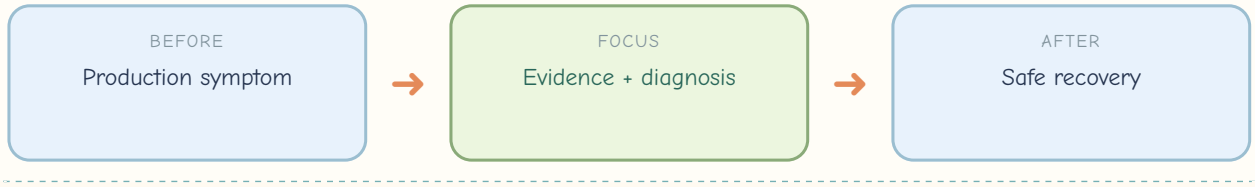
QUESTION 161 / 180

A cache serves another tenant's data. What failed?

01 / THE SHORT INTERVIEW ANSWER

> **The key or cached representation omitted a result-affecting tenant or authorization dimension, or access was not enforced at the boundary.**

DRAW THE IDEA — cache serves another tenant



02 / THE ORIGINAL EXAMPLE

```
Before: cache[productId]
After:  cache[(tenantId, productId)] for tenant-specific results
Permission check -> remains mandatory on each protected operation
```

03 / WHY THIS WORKS

Stop the leakage, assess affected entries, and invalidate or rebuild the cache. A corrected key does not retroactively remove old entries. Test cross-tenant requests and role changes.

04 / WATCH OUT

> Valid authentication does not repair a wrong cache key.

05 / INTERVIEWER FOLLOW-UP

How would a tenant id be trusted rather than taken blindly from a request?

Answer: Derive tenant scope from verified authentication, not a user-editable header alone. Compare resource ownership in the data query and test cross-tenant IDs explicitly.

Source guide p. 106 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

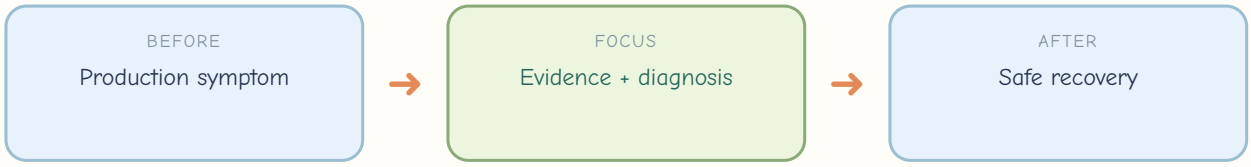
QUESTION 162 / 180

JWT requests work in Postman but fail in the browser. Why?

01 / THE SHORT INTERVIEW ANSWER

Separate CORS/preflight behavior from token validation, cookie/CSRF rules, and the browser's actual sent headers.

DRAW THE IDEA — JWT requests work Postman



02 / THE ORIGINAL EXAMPLE

```
Inspect OPTIONS status and access-control headers.
Inspect actual Authorization header on the following request.
Check Origin allowlist and credential mode.
```

03 / WHY THIS WORKS

Postman does not enforce browser CORS. A preflight may be rejected before the actual bearer request occurs. Use server and browser evidence to locate the failure instead of disabling all security.

04 / WATCH OUT

CORS errors can mask an underlying 401/403 response.

05 / INTERVIEWER FOLLOW-UP

How would you test the preflight in MockMvc?

Answer: Use MockMvc OPTIONS with Origin and Access-Control-Request-Method headers; assert allowed origins, methods, headers and no unintended 401.

Source guide p. 107 • Senior • References: D03, S04, I04, 001, 002

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

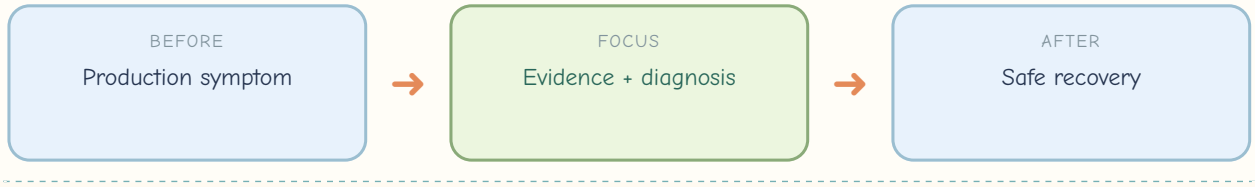
QUESTION 163 / 180

An API returns 403 after login. What should you check?

01 / THE SHORT INTERVIEW ANSWER

Check the required authority, role-prefix mapping, matched security chain, method checks, CSRF, and object ownership.

DRAW THE IDEA — API returns 403 after



02 / THE ORIGINAL EXAMPLE

```
Token scope: orders.read -> typical authority SCOPE_orders.read
Rule expects ROLE_ADMIN -> different authority
POST with cookie authentication -> may also require CSRF token
```

03 / WHY THIS WORKS

A valid identity can fail authorization, and 403 can have multiple causes. Examine which component denied access. Configure claim conversion deliberately rather than granting broad privileges.

04 / WATCH OUT

hasRole and hasAuthority are not identical string checks.

05 / INTERVIEWER FOLLOW-UP

How would you verify scope conversion?

Answer: Load a token with the expected scope, inspect JwtGrantedAuthoritiesConverter mapping, and assert the resulting SCOPE_ authority and effective SecurityFilterChain selection.

Source guide p. 107 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

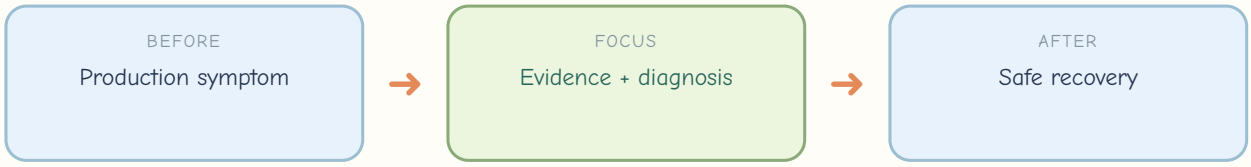
QUESTION 164 / 180

An endpoint charges twice after a retry. How should it be repaired?

01 / THE SHORT INTERVIEW ANSWER

Persist one business operation identity and coordinate deduplication with the provider or recover through reconciliation.

DRAW THE IDEA — endpoint charges twice after



02 / THE ORIGINAL EXAMPLE

```
First request: key=payment-42, result unknown after timeout
Retry: same key=payment-42
Server/provider: return existing outcome or continue same operation
```

03 / WHY THIS WORKS

A timeout cannot prove the first attempt failed. Store request identity, state, and result under a concurrency-safe protocol. Define key reuse, payload mismatch, expiry, and recovery.

04 / WATCH OUT

A local synchronized block does not deduplicate across replicas.

05 / INTERVIEWER FOLLOW-UP

What if the provider succeeded before the application stored the result?

Answer: Use provider reconciliation by the original operation ID: recover the charge result and store it once. A new idempotency key risks a second charge.

Source guide p. 108 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

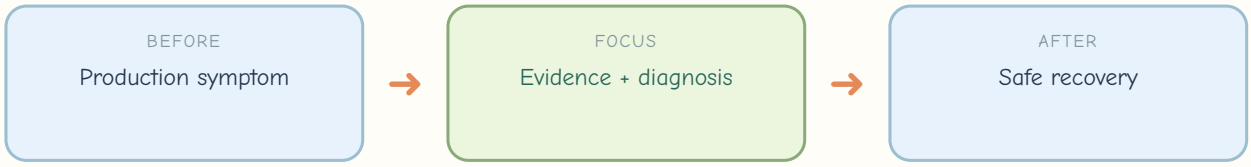
QUESTION 165 / 180

A Kafka consumer applies the same event twice. What is required?

01 / THE SHORT INTERVIEW ANSWER

Make the business effect idempotent and inspect the commit, redelivery, retry, and rebalance windows.

DRAW THE IDEA — Kafka consumer applies same



02 / THE ORIGINAL EXAMPLE

```
Effect committed -> process stops before offset committed
Restart -> record delivered again
Dedup marker + effect -> authoritative atomic policy
```

03 / WHY THIS WORKS

This is expected under at-least-once delivery. A durable unique operation/event marker helps when it shares the correct boundary with the effect. For remote effects, reuse provider idempotency identity.

04 / WATCH OUT

Acks=all does not eliminate consumer duplicate effects.

05 / INTERVIEWER FOLLOW-UP

How would replay tests validate correctness?

Answer: Replay the same ID, out-of-order IDs and restart-after-commit cases. Assert a single durable effect and correct offset/retry behavior in an integrated broker/DB test.

Source guide p. 108 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

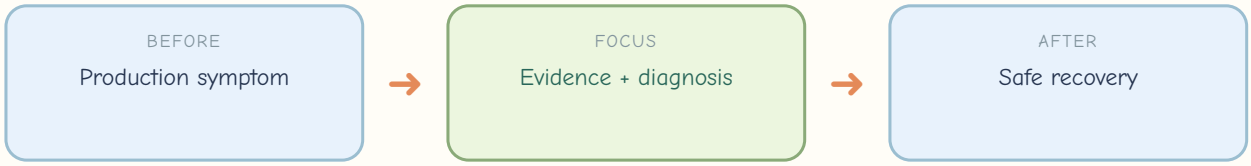
QUESTION 166 / 180

Async notifications run before an order commits. Why?

01 / THE SHORT INTERVIEW ANSWER

A separate worker does not inherit the caller's thread-bound transaction and can execute immediately after submission.

DRAW THE IDEA — Async notifications run before



02 / THE ORIGINAL EXAMPLE

```
Transaction A writes order -> submits async task
Task B reads order before A commits -> absent or stale view
Repair -> after-commit handoff or durable outbox
```

03 / WHY THIS WORKS

After-commit callbacks fit some best-effort cases but can still lose work if the process stops. An outbox records the intent with the business transaction and supports recovery.

04 / WATCH OUT

A delay or sleep is not a commit-order guarantee.

05 / INTERVIEWER FOLLOW-UP

How would you prove task loss is recovered after restart?

Answer: Persist order and outbox intent atomically, crash after commit and before send, restart the relay, and prove delivery/retry from the pending row.

Source guide p. 109 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

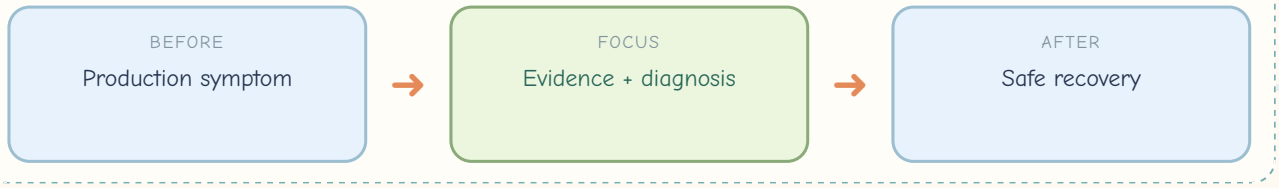
QUESTION 167 / 180

Scheduled jobs run once per pod instead of once per cluster. How do you fix it?

01 / THE SHORT INTERVIEW ANSWER

Use a deliberate distributed coordination or durable scheduling design when cluster-wide singleton execution is required.

DRAW THE IDEA — Scheduled jobs run once



02 / THE ORIGINAL EXAMPLE

```
5 replicas x local @Scheduled -> 5 trigger sources
Cluster-wide requirement -> lease/lock or external scheduler
Business effect -> still idempotent under takeover and retry
```

03 / WHY THIS WORKS

A lock can expire while an old owner continues, so consider fencing and ownership semantics. Durable scheduling and progress recording can simplify recovery. Choose based on delivery requirements rather than suppressing logs.

04 / WATCH OUT

A single initial replica is not a future coordination policy.

05 / INTERVIEWER FOLLOW-UP

How would clock skew or a network partition affect the lease?

Answer: Clock skew and partitions can permit competing lease assumptions. Use database/cluster coordination and fencing/ownership checks at the effect boundary; prefer idempotent job steps.

Source guide p. 109 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

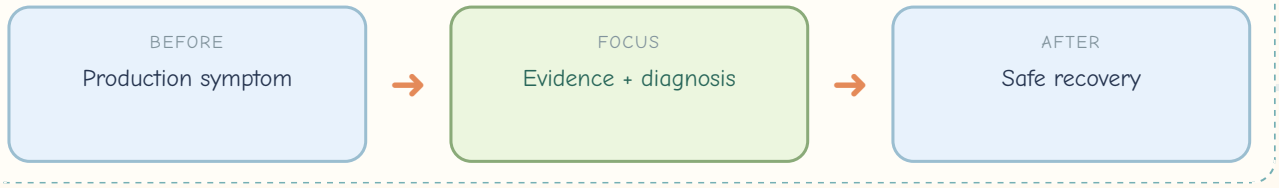
QUESTION 168 / 180

A startup runner waits forever for a remote service. What should change?

01 / THE SHORT INTERVIEW ANSWER

Bound startup waiting, define required versus optional readiness, and separate endless background work from startup completion.

DRAW THE IDEA — startup runner waits forever



02 / THE ORIGINAL EXAMPLE

```
Required dependency -> bounded verification, fail or stay unready
Optional warmup -> controlled degraded state and retry later
Infinite polling loop -> managed background lifecycle
```

03 / WHY THIS WORKS

A never-ending runner prevents successful startup completion. The policy should match whether useful requests can be served. Use backoff and limits rather than spinning or overwhelming the dependency.

04 / WATCH OUT

A listening port does not prove runners have completed.

05 / INTERVIEWER FOLLOW-UP

What would readiness report during fallback operation?

Answer: Readiness should report usable-with-documented-degradation only when the application can safely serve its essential contract; otherwise remain unready while bounded startup retry continues.

Source guide p. 110 • Senior • References: D03, S04, I04, 001, 002

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

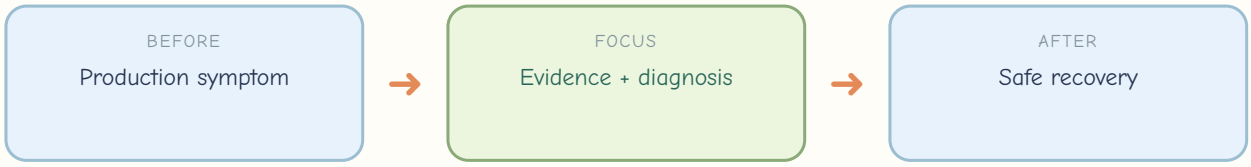
QUESTION 169 / 180

A database pool exhausts after traffic drops. What do you inspect?

01 / THE SHORT INTERVIEW ANSWER

Inspect leaked or long-held connections, transaction duration, blocked queries, and pool acquisition/ownership evidence.

DRAW THE IDEA — database pool exhausts after



02 / THE ORIGINAL EXAMPLE

```
Active connections stay high + requests low
 -> long transactions, stuck I/O, leaked ownership, lock waits
Thread stacks + database sessions -> correlate holders
```

03 / WHY THIS WORKS

Enable appropriate temporary leak diagnostics carefully, then find the ownership defect. Leak warnings can also identify a legitimately long transaction. Ensure every manual resource closes and avoid holding database transactions over remote calls.

04 / WATCH OUT

Raising the pool size can delay the next outage.

05 / INTERVIEWER FOLLOW-UP

Which code path fails to release resources on an exception?

Answer: Inspect every response/error/timeout cancellation path, especially streams and HTTP client bodies; track outstanding leases and stack traces showing acquisition without close.

Source guide p. 110 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

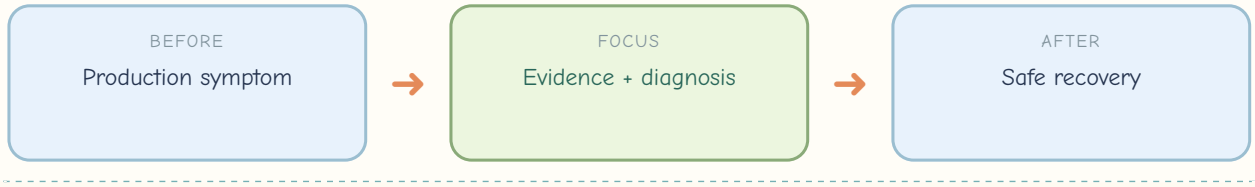
QUESTION 170 / 180

A REQUIRES_NEW audit call stalls under load. Why?

01 / THE SHORT INTERVIEW ANSWER

> **The outer transaction may hold a connection while the independent inner transaction waits for another, exhausting the pool.**

DRAW THE IDEA — REQUIRES_NEW audit call stalls



02 / THE ORIGINAL EXAMPLE

```
20 outer transactions hold 20 pool connections
Each enters inner REQUIRES_NEW and requests another
No spare connection -> blocked acquisition
```

03 / WHY THIS WORKS

Verify actual connection and propagation behavior. Redesign the audit durability boundary or budget concurrency and connections deliberately. Crossing a proper proxy is also needed for this propagation to occur.

04 / WATCH OUT

> Independent transactions are not free concurrency.

05 / INTERVIEWER FOLLOW-UP

Would an outbox satisfy the audit requirement differently?

Answer: REQUIRES_NEW may need a second pool connection while an outer transaction holds one. An outbox can persist audit intent in the outer transaction without the nested extra connection.

Source guide p. 111 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

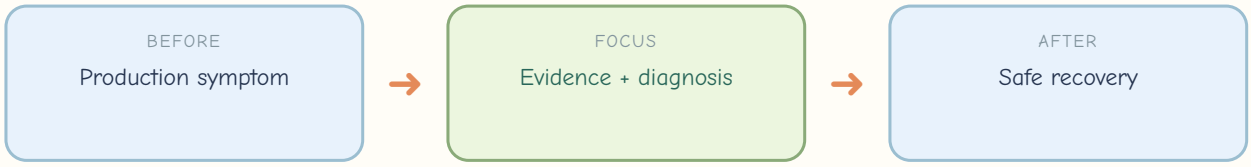
QUESTION 171 / 180

A health check causes every pod to restart during a DB outage. What failed?

01 / THE SHORT INTERVIEW ANSWER

A shared dependency was treated as a liveness failure, turning an external outage into a restart storm.

DRAW THE IDEA — health check causes every



02 / THE ORIGINAL EXAMPLE

```
Database down -> liveness DOWN -> all instances restart
Repair -> process-only liveness; deliberate readiness policy
```

03 / WHY THIS WORKS

Restarting cannot repair the database. Restore useful readiness behavior and prevent cascading reconnection surges. Test dependency outages and probe thresholds before deployment.

04 / WATCH OUT

Readiness should also consider whether partial functionality remains useful.

05 / INTERVIEWER FOLLOW-UP

How would startupProbe differ during initial boot?

Answer: startupProbe gives a slow starting process a bounded startup grace period before liveness takes over. It should not use unrelated downstream outages as a perpetual restart trigger.

Source guide p. 111 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

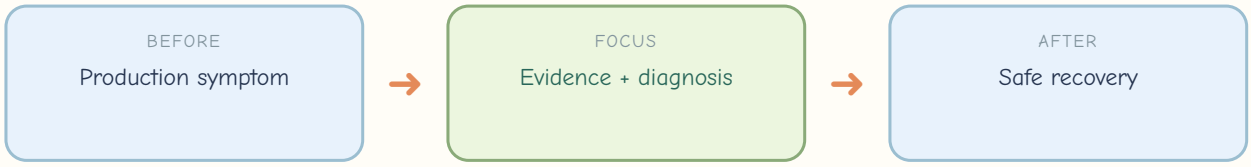
QUESTION 172 / 180

An API returns 200 with an empty list during an outage. Why is it risky?

01 / THE SHORT INTERVIEW ANSWER

It disguises retrieval failure as valid absence, potentially driving incorrect downstream decisions.

DRAW THE IDEA — API returns 200 empty



02 / THE ORIGINAL EXAMPLE

```
No customers -> successful empty result
Database unavailable -> separate failure contract
Cached fallback -> explicit degraded semantics if permitted
```

03 / WHY THIS WORKS

A client may assume there is no data and delete or create resources. Preserve the failure cause and return the defined error response. A fallback needs freshness and business approval, not just convenient typing.

04 / WATCH OUT

catch(Exception) returning empty is not universal robustness.

05 / INTERVIEWER FOLLOW-UP

How would a batch retain successes while reporting failed items?

Answer: Return per-item success/error records or a documented partial-success response, preserving stable item IDs and retryability. Empty success must not camouflage dependency failure.

Source guide p. 112 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

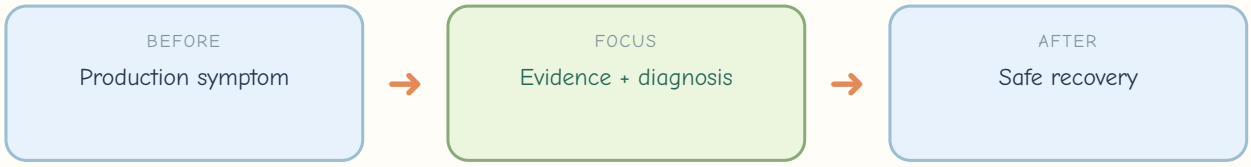
QUESTION 173 / 180

A memory leak appears after enabling metrics. What do you inspect?

01 / THE SHORT INTERVIEW ANSWER

Check high-cardinality meter tags and retained registries or listener state before attributing the issue to all observability.

DRAW THE IDEA — memory leak appears after



02 / THE ORIGINAL EXAMPLE

```
Timer tag customerId -> one series per unique customer
Meter count keeps increasing -> registry retains combinations
Fix -> normalized routes and bounded business categories
```

03 / WHY THIS WORKS

Confirm the retention path and compare series count after the change. Do not disable all telemetry without an alternative diagnostic plan. Sampling and label design solve different issues.

04 / WATCH OUT

A bounded request queue does not bound meter cardinality.

05 / INTERVIEWER FOLLOW-UP

What memory evidence would distinguish it from a cache leak?

Answer: Inspect meter count, unique tag combinations, registry allocations and heap dominators. A metrics leak retains meter/tag objects; a cache leak retains cached keys/values.

Source guide p. 112 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

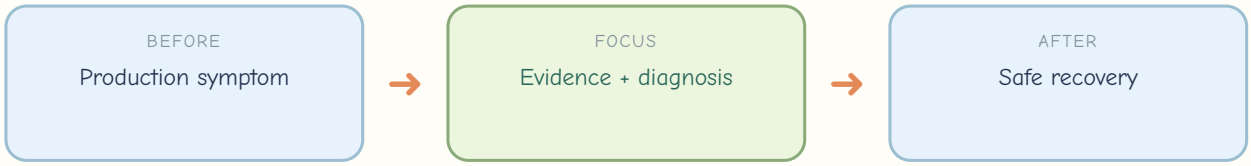
QUESTION 174 / 180

WebFlux requests stop progressing while one JDBC call runs. Why?

01 / THE SHORT INTERVIEW ANSWER

Blocking work occupies a shared event-loop thread; isolate it or choose a stack aligned with the blocking dependencies.

DRAW THE IDEA — WebFlux requests stop progressing



02 / THE ORIGINAL EXAMPLE

```
Bad: Mono.just(jdbc.load()) -> call runs before publisher assembly
Better shape: Mono.fromCallable(() -> jdbc.load())
                  .subscribeOn(Schedulers.boundedElastic())
```

03 / WHY THIS WORKS

fromCallable defers execution. The scheduler isolates the blocking operation, but downstream and in-flight concurrency still need bounds. R2DBC is a different persistence model, not a drop-in Hibernate switch.

04 / WATCH OUT

Wrapping an already-computed result in Mono is not offloading.

05 / INTERVIEWER FOLLOW-UP

Would MVC with virtual threads fit better?

Answer: MVC plus virtual threads may suit predominantly blocking JDBC workloads if the stack is compatible, but DB connections still constrain concurrency. Compare performance under bounded load.

Source guide p. 113 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

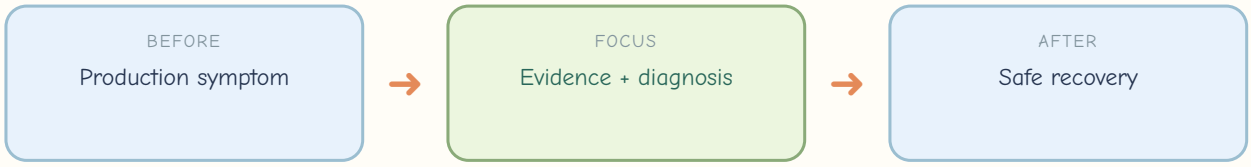
QUESTION 175 / 180

An integration test passes but production violates a constraint. Why?

01 / THE SHORT INTERVIEW ANSWER

> **The test may use a different engine, skip commit/flush, mock the repository, or omit real migrations and concurrency.**

DRAW THE IDEA — integration test passes but



02 / THE ORIGINAL EXAMPLE

```
Mock repository -> no database constraint exercised
H2 substitute -> dialect or locking differs
Rollback-only test -> commit-sensitive failure can be missed
```

03 / WHY THIS WORKS

Test the required invariant against the deployment engine, with explicit flush or completion when appropriate. Add concurrency tests when the defect depends on simultaneous requests.

04 / WATCH OUT

> More assertions on mocks do not validate SQL behavior.

05 / INTERVIEWER FOLLOW-UP

How would two concurrent inserts test a uniqueness race?

Answer: Use two independent concurrent transactions starting together, attempt the same unique key and assert one commits and the other receives the expected constraint failure.

Source guide p. 113 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

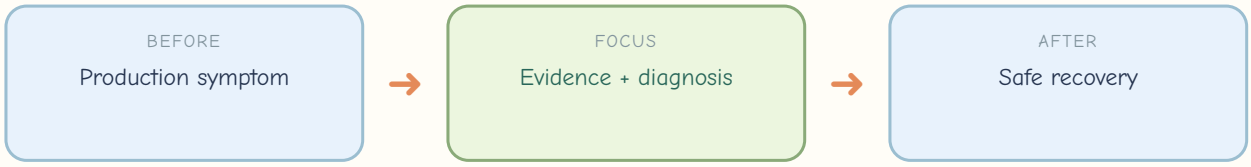
QUESTION 176 / 180

A rollout breaks old pods after a schema change. What failed?

01 / THE SHORT INTERVIEW ANSWER

> **The schema transition was not backward compatible with versions running during rollout or rollback.**

DRAW THE IDEA — rollout breaks old pods



02 / THE ORIGINAL EXAMPLE

```
Drop old column -> old pod query fails
Safer: add new shape -> dual-compatible code -> migrate -> retire old shape
```

03 / WHY THIS WORKS

Inventory consumers, migration locks, data backfill, and mixed-version behavior. Verify both the new and previous application versions against the transitional schema.

04 / WATCH OUT

Application rollback does not restore dropped data.

05 / INTERVIEWER FOLLOW-UP

Which migration should be delayed until all old consumers are gone?

Answer: Delay dropping/renaming old columns or making formerly optional fields mandatory until all old applications, jobs and consumers have migrated and rollback no longer needs the old schema.

Source guide p. 114 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

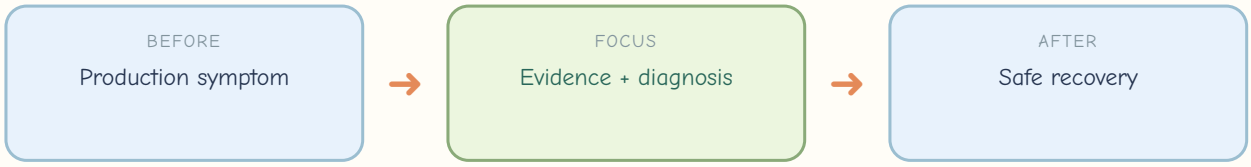
QUESTION 177 / 180

Logging context disappears in async or reactive work. How do you repair it?

01 / THE SHORT INTERVIEW ANSWER

Use supported context propagation with clear lifecycle ownership rather than assuming ordinary ThreadLocal state follows execution.

DRAW THE IDEA — Logging context disappears async



02 / THE ORIGINAL EXAMPLE

```
Caller thread MDC -> executor task may use another thread
Reactor sequence -> context belongs to the reactive chain
Cleanup -> restore or remove owned context at the boundary
```

03 / WHY THIS WORKS

Select propagation support for the actual framework and executor. Test both correct correlation and absence of cross-request leakage. Do not copy authentication blindly into work with a different trust boundary.

04 / WATCH OUT

InheritableThreadLocal is not a general pooled-task solution.

05 / INTERVIEWER FOLLOW-UP

How would task reuse expose stale context?

Answer: Reused pool threads may retain ThreadLocal/MDC data from an earlier request. Capture/restore explicitly and remove in finally; Reactor needs its supported context propagation approach.

Source guide p. 114 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

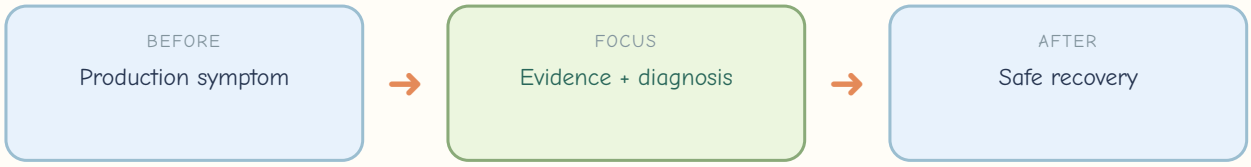
QUESTION 178 / 180

A tenant filter exists but direct repository calls bypass it. What should change?

01 / THE SHORT INTERVIEW ANSWER

> **Enforce the invariant at an authoritative service or data-access boundary reachable from every entry point.**

DRAW THE IDEA — tenant filter exists but



02 / THE ORIGINAL EXAMPLE

```
HTTP interceptor only -> messaging path can bypass
Service authorization + tenant-scoped query -> common boundary
Test HTTP, batch, and message entry points
```

03 / WHY THIS WORKS

A cross-cutting mechanism must actually cover the calls it claims to protect. Trust tenant identity from authenticated context and ensure query predicates and key selection align. Consider database-enforced isolation where required.

04 / WATCH OUT

> A client-supplied tenant header alone is insufficient authority.

05 / INTERVIEWER FOLLOW-UP

How would a cache interact with the same invariant?

Answer: Use tenant-qualified keys and recheck authorization on every sensitive hit path. A cache is part of the security boundary, not an exemption from ownership checks.

Source guide p. 115 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

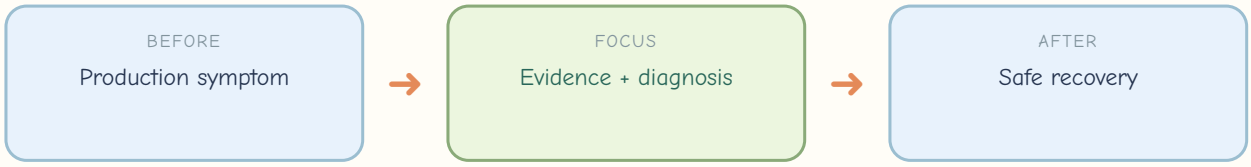
QUESTION 179 / 180

Graceful shutdown still loses queued tasks. What contract is missing?

01 / THE SHORT INTERVIEW ANSWER

Executor draining is bounded process behavior, not durable acceptance or recovery of important queued work.

DRAW THE IDEA — Graceful shutdown still loses



02 / THE ORIGINAL EXAMPLE

```
HTTP accepts job -> in-memory queue -> process terminated
Durable job record -> accepted work can be recovered
Shutdown -> stop admission, bounded drain, record unfinished state
```

03 / WHY THIS WORKS

Return acceptance only after the durability requirement is met. Pending tasks and in-flight effects need replay/idempotency rules. Tune platform grace and executor shutdown, but do not rely on an unlimited wait.

04 / WATCH OUT

shutdownNow does not atomically finish all accepted jobs.

05 / INTERVIEWER FOLLOW-UP

What would the client query after a restart?

Answer: Expose a durable job/order status endpoint keyed by the operation ID, so a client can discover whether a previously accepted task completed or needs recovery after restart.

Source guide p. 115 • Senior • References: D03, S04, I04, O01, O02

CHAPTER 24 • SENIOR PRODUCTION DIAGNOSIS

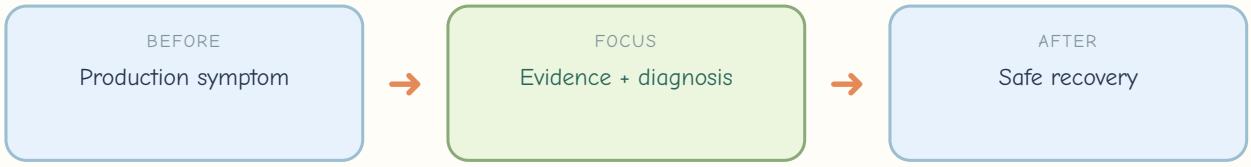
QUESTION 180 / 180

How would you review a Spring Boot service before production?

01 / THE SHORT INTERVIEW ANSWER

State its key invariants, failure boundaries, resource budgets, security rules, observability, deployment compatibility, and recovery evidence.

DRAW THE IDEA — would review before production



02 / THE ORIGINAL EXAMPLE

```
Review -> contract, authorization, transaction, idempotency
Limits -> DB, outbound pool, executor, cache, retries
Release -> readiness, probes, migrations, rollback
Evidence -> focused tests, load profile, operational signals
```

03 / WHY THIS WORKS

A successful startup proves wiring, not business correctness or capacity. Choose checks according to the actual service rather than counting annotations. Resolve risks with an owner and a verifiable decision before release.

04 / WATCH OUT

A generic checklist is not evidence that a critical payment path is safe.

05 / INTERVIEWER FOLLOW-UP

Which unresolved risk would block your release?

Answer: A critical unresolved risk such as cross-tenant access, duplicate charges, unsafe schema rollout or missing recovery of committed orders should block release regardless of passing unit tests.

Source guide p. 116 • Senior • References: D03, S04, I04, O01, O02

12 coding challenges

These are the **original Spring Boot coding exercises** from the uploaded guide. Most are implementation recipes: they require a suitable project, imports, dependencies and integration-test context.

For each: implement the core logic, state the success/failure contract, then prove the intended behavior with a test.

> The source guide explicitly states its snippets have not all been compiled as a complete application. This edition does not claim that the exercises were executed against live Spring Boot, PostgreSQL, Kafka or JWT infrastructure.

LAB C01 / 12

Bind and reject invalid client configuration

Original source page 118 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C01 Bind and reject invalid client configuration
Create typed client settings. Reject a missing URL and attempts below one during startup.
Setup: configuration
Worked solution
@Validated
@ConfigurationProperties("client")
public record ClientProperties(@NotNull URI baseUrl,
                               @Min(1) int maxAttempts) {}
@SpringBootApplication
@ConfigurationPropertiesScan
class App { /* standard main method */ }
# application.properties:
client.base-url=https://service.example
client.max-attempts=3
Verification and boundary policy
Requires validation and properties registration. Check a valid context, missing base URL, zero
attempts, and an environment override. Expected: invalid cases fail context creation, not the first
user request.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C02 / 12

Build a validated create endpoint

Original source page 119 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C02 Build a validated create endpoint
Accept a name and email, return a created DTO with 201 and Location, and reject invalid input.
Setup: web + validation
Worked solution
record CreateCustomer(@NotBlank String name, @Email String email) {}
record CustomerDto(long id, String name) {}
@RestController
class Customers {
  private final CustomerService service;
  Customers(CustomerService service) { this.service = service; }
  @PostMapping("/customers")
  ResponseEntity<CustomerDto> create(@Valid @RequestBody CreateCustomer r) {
    CustomerDto dto = service.create(r);
    return ResponseEntity.created(URI.create("/customers/" + dto.id()))
        .body(dto);
  }
}
Verification and boundary policy
CustomerService must implement the stated operation and uniqueness policy. Check 201/Location
for valid input, 400 for blank name, malformed JSON, and duplicate email under the chosen conflict
contract. @Email alone allows null; add @NotBlank if email is required.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C03 / 12

Return a safe version-conflict response

Original source page 120 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C03 Return a safe version-conflict response
Map a domain conflict into ProblemDetail without exposing persistence stack traces.
Setup: web
Worked solution
@RestControllerAdvice
class ApiErrors {
  @ExceptionHandler(OrderVersionConflict.class)
  ResponseEntity<ProblemDetail> conflict(OrderVersionConflict ex) {
    ProblemDetail p = ProblemDetail.forStatusAndDetail(
        HttpStatus.CONFLICT, "Order changed; reload before updating");
    p.setProperty("code", "ORDER_VERSION_CONFLICT");
    return ResponseEntity.status(HttpStatus.CONFLICT).body(p);
  }
}
Verification and boundary policy
The domain exception must be defined and thrown at a deliberate translation boundary. Check
status 409, stable code, and absence of SQL, credentials or raw stack traces. Do not map all
optimistic failures to a retry without understanding the operation.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C04 / 12

Prove checked-failure rollback

Original source page 121 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C04 Prove checked-failure rollback
Write one database change, throw a checked exception, and verify no change remains after the real
service boundary returns failure.
Setup: data-jpa + target database test
Worked solution
@Service
class ImportService {
  private final OrderRepository orders;
  ImportService(OrderRepository orders) { this.orders = orders; }
  @Transactional(rollbackFor=IOException.class)
  public void importOne(Order order) throws IOException {
    orders.save(order);
    throw new IOException("source incomplete");
  }
}
Verification and boundary policy
Test through the Spring-managed service proxy with a real persistence engine. The verification
should occur outside an enclosing test transaction that could hide the service's behavior. Expected:
failure propagates and row is absent. Repeat without rollbackFor to explain the default
checked-exception rule.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C05 / 12

Guard optimistic inventory updates

Original source page 122 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C05 Guard optimistic inventory updates
Use a versioned inventory row and demonstrate two writers cannot silently overwrite each other's
changes.
Setup: data-jpa
Worked solution
@Entity
class Stock {
  @Id Long id;
  @Version Long version;
  int available;
  protected Stock() {}
  void reserve(int amount) {
    if (amount < 1 || available < amount) throw new IllegalArgumentException();
    available -= amount;
  }
}
Verification and boundary policy
Two transactions load the same version, then attempt incompatible changes. Check that one
encounters the translated optimistic conflict. Retry from fresh state only when the business
operation is safe; recheck available stock. Test final quantity, not just exception occurrence.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C06 / 12

Fetch an API projection without lazy traversal

Original source page 123 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C06 Fetch an API projection without lazy traversal
Return only approved customer fields through a repository query, with deterministic bounded
pagination.
Setup: data-jpa
Worked solution
interface CustomerSummary {
  Long getId();
  String getName();
}
interface Customers extends JpaRepository<Customer, Long> {
  @Query("select c.id as id, c.name as name from Customer c where c.active=true")
  Slice<CustomerSummary> active(Pageable page);
}
// Whitelist sort fields before constructing PageRequest.
Verification and boundary policy
The entity must provide the mapped fields. Check selected columns, page boundary behavior, stable
id tie-breaking, and absence of per-row lazy queries. Slice can avoid a required total count but
still
needs another-page determination.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C07 / 12

Protect an API with JWT scopes

Original source page 124 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C07 Protect an API with JWT scopes
Require the read scope for GET order endpoints and deny callers without it. Provide issuer and
audience validation.
Setup: security + oauth2-resource-server
Worked solution
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
  http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(a -> a
          .requestMatchers(HttpMethod.GET, "/orders/**")
          .hasAuthority("SCOPE_orders.read")
          .anyRequest().authenticated())
      .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
  return http.build();
}
// Trusted issuer configured through external settings:
@Bean
JwtDecoder decoder(@Value("${security.issuer}") String issuer) {
  NimbusJwtDecoder decoder = JwtDecoders.fromIssuerLocation(issuer);
  OAuth2TokenValidator<Jwt> audience = jwt ->
      jwt.getAudience().contains("orders-api")
          ? OAuth2TokenValidatorResult.success()
          : OAuth2TokenValidatorResult.failure(
              new OAuth2Error("invalid_token", "Wrong audience", null));
  decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
      JwtValidators.createDefaultWithIssuer(issuer), audience));
  return decoder;
}
Verification and boundary policy
The custom validator composes audience with the default time and issuer checks rather than
replacing them. Configure a trusted security.issuer and appropriate identity-provider access. This
bearer-only example keeps CSRF/CORS policy explicit: select it for the actual credential design
rather
than silently disabling protection. Check missing/invalid token, wrong issuer, wrong audience,
missing scope, valid read scope, and cross-tenant access. Mocked authorization identities do not
replace decoder validation tests.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C08 / 12

Test the real MVC error contract

Original source page 125 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C08 Test the real MVC error contract
Verify request validation and a domain absence through MockMvc with a mocked service, keeping
the actual controller and advice.
Setup: test
Worked solution
@WebMvcTest(Customers.class)
class CustomersTest {
  @Autowired MockMvc mvc;
  @MockitoBean CustomerService service;
  @Test void rejectsBlankName() throws Exception {
    mvc.perform(post("/customers").with(user("tester")).with(csrf())
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"name\":\"\",\"email\":\"a@example.com\"}"))
       .andExpect(status().isBadRequest());
  }
}
Verification and boundary policy
The user and csrf helpers require spring-security-test static imports. Match the test authority to
the
actual endpoint rules; this fixture assumes an authenticated tester is allowed. A request rejected
by
security would not test validation. Verify service.create is not called. Add malformed JSON, valid
201,
advice handling, and explicit security cases.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C09 / 12

Design a tenant-safe immutable cache value

Original source page 126 • Add a compatible Boot project and dependencies

Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C09 Design a tenant-safe immutable cache value
Cache an authorization-independent projection using tenant plus object identity, and define
invalidation on a successful update.
Setup: cache + chosen provider
Worked solution
record ProductKey(String tenant, long id) {}
record ProductView(long id, String name) {}
@Service
class ProductService {
  private final ProductViews views;
  ProductService(ProductViews views) { this.views = views; }
  public ProductView load(String tenant, long id) {
    authorizeTenant(tenant); // Implement from trusted caller identity.
    return views.load(tenant, id); // Crosses another bean's proxy.
  }
}
@Component
class ProductViews {
  private final ProductRepository repository;
  ProductViews(ProductRepository r) { repository = r; }
  @Cacheable(cacheNames="products",
      key="new com.example.ProductKey(#tenant, #id)")
  public ProductView load(String tenant, long id) {
    return repository.loadView(tenant, id);
  }
}
Verification and boundary policy
Enable caching, configure the provider, and put ProductKey in the named package. Retain parameter
names for SpEL or use #p0 and #p1. Authorization runs on every outer call even when the inner
method is skipped on a cache hit. Restrict direct entry to ProductViews so callers cannot bypass
that
boundary. Check tenant isolation, hit-path authorization, update invalidation, TTL and immutable
values.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C10 / 12

Bound an async notification executor

Original source page 127 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C10 Bound an async notification executor
Define a managed executor, reject overload explicitly, and make task failure observable.
Setup: core + async support
Worked solution
@Bean("notifications")
ThreadPoolTaskExecutor notifications() {
  ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
  e.setCorePoolSize(2); e.setMaxPoolSize(4);
  e.setQueueCapacity(50); e.setThreadNamePrefix("notify-");
  e.setRejectedExecutionHandler(new ThreadPoolExecutor.AbortPolicy());
  e.setWaitForTasksToCompleteOnShutdown(true);
  e.setAwaitTerminationSeconds(10);
  return e;
}
Verification and boundary policy
Enable async support and invoke a named @Async method across its proxy. Check success,
exceptional completion, saturated admission, and bounded shutdown. These values illustrate policy,
not recommended sizing. Durable notifications need a persisted job or outbox, not just this queue.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C11 / 12

Defer and isolate a blocking client call

Original source page 128 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C11 Defer and isolate a blocking client call
Create a reactive wrapper that defers blocking execution, isolates it, and exposes a result
deadline.
Setup: webflux
Worked solution
Mono<Result> call() {
  return Mono.fromCallable(() -> blockingClient.call())
      .subscribeOn(Schedulers.boundedElastic())
      .timeout(Duration.ofSeconds(2));
}
Verification and boundary policy
Check no client call occurs before subscription, execution does not occupy the event loop, and a
slow
call produces timeout behavior. The operator does not guarantee transport cancellation; configure
the client's deadlines too. Bound in-flight work and verify actual cleanup.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

LAB C12 / 12

Persist order creation with outbox intent

Original source page 129 • Add a compatible Boot project and dependencies

> Source exercise: implement the method or project recipe, then verify its actual success and failure behavior.

```
C12 Persist order creation with outbox intent
Store the order and event intent in one local transaction, then publish with retryable, observable
ownership.
Setup: data-jpa + messaging integration
Worked solution
@Transactional
public OrderId place(CreateOrder c) {
  Order order = orders.save(Order.from(c));
  outbox.save(OutboxEvent.orderCreated(order.getId(), c.operationId()));
  return new OrderId(order.getId());
}
// Publisher later claims pending rows, publishes, and tracks outcome.
// Consumer deduplicates the stable event/operation identity.
Verification and boundary policy
This domain sketch requires actual entities and repositories. Check rollback leaves neither record,
commit leaves both, crash-after-publish can redeliver safely, concurrent publishers cannot lose
rows,
and backlog alerts fire. Describe the supported exactly-once scope without claiming the entire
HTTP/Kafka/database chain is automatically atomic.
```

After implementing, test the expected result and its failure path. Snippets may be partial project recipes.

30-day interview sprint

Use this revision loop alongside the source's original day-by-day study route (pp. 9–11). Repeat one 20-second explanation, one code walkthrough and one follow-up for every question.

DAY 01 **Boot startup + dependencies** — DI, scanning, lifecycle and proxy; answer 4 original questions and review follow-ups.

DAY 02 **Boot startup + dependencies** — DI, scanning, lifecycle and proxy; answer 4 original questions and review follow-ups.

DAY 03 **Boot startup + dependencies** — DI, scanning, lifecycle and proxy; answer 4 original questions and review follow-ups.

DAY 04 **Boot startup + dependencies** — DI, scanning, lifecycle and proxy; answer 4 original questions and review follow-ups.

DAY 05 **Configuration & auto-config** — Validation, profiles and condition reports; answer 4 original questions and review follow-ups.

DAY 06 **Configuration & auto-config** — Validation, profiles and condition reports; answer 5 original questions and review follow-ups.

DAY 07 **Configuration & auto-config** — Validation, profiles and condition reports; answer 5 original questions and review follow-ups.

DAY 08 **Configuration & auto-config** — Validation, profiles and condition reports; answer 5 original questions and review follow-ups.

DAY 09 **MVC + API contracts** — Request mapping, status and pagination; answer 5 original questions and review follow-ups.

DAY 10 **MVC + API contracts** — Request mapping, status and pagination; answer 5 original questions and review follow-ups.

DAY 11 **MVC + API contracts** — Request mapping, status and pagination; answer 5 original questions and review follow-ups.

DAY 12 **MVC + API contracts** — Request mapping, status and pagination; answer 6 original questions and review follow-ups.

DAY 13 **JPA + transactions** — Dirty checking, N+1 and isolation; answer 6 original questions and review follow-ups.

DAY 14 **JPA + transactions** — Dirty checking, N+1 and isolation; answer 6 original questions and review follow-ups.

DAY 15 **JPA + transactions** — Dirty checking, N+1 and isolation; answer 6 original questions and review follow-ups.

30-day sprint — days 16–30

Keep the same rhythm: answer, example, pitfall, follow-up, and a hands-on check.

DAY 16	**JPA + transactions** — Dirty checking, N+1 and isolation; answer 6 original questions and review follow-ups.
DAY 17	**Security** — JWT, OAuth2, CSRF and tenant authorization; answer 6 original questions and review follow-ups.
DAY 18	**Security** — JWT, OAuth2, CSRF and tenant authorization; answer 7 original questions and review follow-ups.
DAY 19	**Security** — JWT, OAuth2, CSRF and tenant authorization; answer 7 original questions and review follow-ups.
DAY 20	**Security** — JWT, OAuth2, CSRF and tenant authorization; answer 7 original questions and review follow-ups.
DAY 21	**Testing & HTTP clients** — MVC, integration, timeouts and retries; answer 7 original questions and review follow-ups.
DAY 22	**Testing & HTTP clients** — MVC, integration, timeouts and retries; answer 7 original questions and review follow-ups.
DAY 23	**Testing & HTTP clients** — MVC, integration, timeouts and retries; answer 7 original questions and review follow-ups.
DAY 24	**Testing & HTTP clients** — MVC, integration, timeouts and retries; answer 8 original questions and review follow-ups.
DAY 25	**Async + messaging** — Executor limits, outbox and Kafka; answer 8 original questions and review follow-ups.
DAY 26	**Async + messaging** — Executor limits, outbox and Kafka; answer 8 original questions and review follow-ups.
DAY 27	**Async + messaging** — Executor limits, outbox and Kafka; answer 8 original questions and review follow-ups.
DAY 28	**Async + messaging** — Executor limits, outbox and Kafka; answer 8 original questions and review follow-ups.
DAY 29	**WebFlux + operations** — Reactor, probes, metrics and rollout; answer 8 original questions and review follow-ups.
DAY 30	**WebFlux + operations** — Reactor, probes, metrics and rollout; answer 9 original questions and review follow-ups.

Use this edition responsibly

Source basis: the attached *Spring Boot Zero to Hero Interview Guide* (136 pages). Its 180 numbered questions and 180 follow-up prompts have been preserved word-for-word; the answers here are reformatted and the new follow-up answers are added.

Verification scope: source question IDs and snippets can be checked mechanically. Spring integration behavior, migrations, security policy and production outcomes require a project with appropriate dependencies and infrastructure. Code fragments that represent conceptual flows are not ready-to-run projects.

Commercial rights: before publishing on a storefront, confirm that you own or have permission to adapt, reproduce, and redistribute the underlying guide.

High-value references: Spring Boot Reference Guide, Spring Framework Reference, Spring Security Reference, Hibernate ORM documentation, Spring Data JPA documentation, Reactor Reference, RFC 9457 Problem Details, OAuth 2.0/OIDC specifications. See the source guide's primary-reference directory for its original keys.